

Angular v21 Eğitim Programı

Modül 1-4

Temeller, TypeScript & Component'lar

Tamamlanan Modüller:4 / 30

Toplam Ders: 24 ders

Seviye: Başlangıç · Sıfır → Junior

Versiyon: Angular v21 (Kasım 2025)

İçerik:

- ✓ **Modül 1** Angular & Modern Frontend Temelleri
- ✓ **Modül 2** TypeScript Refresher (Angular için)
- ✓ **Modül 3** İlk Angular Uygulaması, CLI & DevTools
- ✓ **Modül 4** Component'lar & Modern Template Syntax

Hazırlayan: Claude

İçindekiler

✓ Tamamlanan modüller | ▶ Şu an işlenen modül | ○ Henüz işlenmemiş modüller

Başlangıç · Sıfır → Junior

✓ Modül 1 — Angular & Modern Frontend Temelleri

→ Modül 1'e Giriş

→ Ders 1.1 — Frontend Frameworks: Neden Var?

- ↳ Web'in Tarihsel Hikayesi
- ↳ Vanilla JS'in Sınırları
- ↳ SPA (Single Page Application)
- ↳ Component-Based Architecture
- ↳ Reactivity Nedir?
- ↳ Vanilla JS vs Framework Karşılaştırması
- ↳ Bu Eğitimde Kullanılacak Felsefe
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.2 — Angular Tarihçesi & Felsefesi

- ↳ AngularJS — Her Şeyin Başlangıcı
- ↳ Büyük Yıkım ve Yeniden Doğuş — Angular 2
- ↳ Angular'ın Felsefesi — "Opinionated"
- ↳ Angular'ın Versiyon Tarihi
- ↳ Angular Kim Tarafından Kullanılıyor?
- ↳ Angular'ın Geleceği
- ↳ Bu Eğitimde Neden Angular v21?
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.3 — Angular vs React vs Vue (Kod Karşılaştırması)

- ↳ Karşılaştırma Senaryosu
- ↳ React Versiyonu
- ↳ Vue Versiyonu
- ↳ Angular Versiyonu (v21)
- ↳ Yan Yana Karşılaştırma
- ↳ "Bu Kod Bana Ne Söylüyor?"
- ↳ "Hangisini Öğrenmeliyim?"
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.4 — Angular v21'in "Big Picture" Yenilikleri

- ↳ v21 Bir "Geçiş Versiyonu" Değil
- ↳ Zoneless Change Detection
- ↳ Signal Forms
- ↳ Angular Aria
- ↳ Vitest — Karma'nın Sonu
- ↳ MCP Server — AI Çağına Entegrasyon
- ↳ Tailwind CSS
- ↳ Diğer Önemli Yenilikler

- ↳ Bu Yenilikler Senin İçin Ne Demek?
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.5 — Geliştirme Ortamı Kurulumu

- ↳ Geliştirme Ortamımızda Neler Olacak?
- ↳ Node.js Kurulumu
- ↳ Angular CLI Kurulumu
- ↳ VS Code Kurulumu ve Uzantılar
- ↳ VS Code Ayarları
- ↳ Git Kurulumu
- ↳ Angular DevTools Kurulumu
- ↳ İlk Angular Projemizi Oluşturma
- ↳ Hızlı Bir Düzenleme — Hot Reload Görelim
- ↳ Angular DevTools'u İlk Kez Kullanma
- ↳ Geliştirme Workflow'u
- ↳ Sık Karşılaşılan Problemler
- ↳ Pratik Komutlar Sözlüğü
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Modül 1 Özeti

✓ Modül 2 — TypeScript Refresher (Angular için)

→ Modül 2'ye Giriş

→ Ders 2.1 — Temel Tipler & Interfaces

- ↳ Neden TypeScript? (Hızlı Hatırlatma)
- ↳ Temel Tipler
- ↳ Interfaces
- ↳ Type Aliases — interface vs type
- ↳ Union & Intersection Types
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 2.2 — Generics

- ↳ Problem — Generics Olmasaydı
- ↳ Sözdizimi — "T" Nedir?
- ↳ Generic Constraints — Sınırlama
- ↳ Generic Interfaces
- ↳ Generic Classes
- ↳ Default Type Parameters
- ↳ Angular'da Generics
- ↳ Generic CRUD Service Örneği
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 2.3 — Decorators (Kavramsal)

- ↳ Decorator Nedir?
- ↳ Neden Var? Hangi Problemi Çözüyor?
- ↳ Decorator Türleri
- ↳ Modern Angular'da Decorator Trendi
- ↳ Mini Quiz
- ↳ Anti-Patterns

- ↳ Bu Dersten Çıkarılacaklar

→ Ders 2.4 — Utility Types

- ↳ Utility Types Nedir?
- ↳ Partial — Hepsi Optional
- ↳ Pick — Belirli Property'leri Seç
- ↳ Omit — Belirli Property'leri Çıkar
- ↳ Record — Object Map
- ↳ Readonly — Değiştirilemez
- ↳ Exclude ve Extract
- ↳ Function Utility Types
- ↳ Hangi Utility Hangi Senaryoda?
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 2.5 — Type Guards & Narrowing

- ↳ Narrowing Nedir?
- ↳ Built-in Type Guards
- ↳ Discriminated Union Narrowing
- ↳ Custom Type Guards (Type Predicates)
- ↳ NonNullable ile Pratik Narrowing
- ↳ Assertion Functions
- ↳ Exhaustiveness Checking
- ↳ Narrowing'in Sınırları
- ↳ Angular'da Real-World Type Guards
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 2.6 — Mini Proje: TypeScript Bootcamp

- ↳ Proje Tanımı
- ↳ Domain Modelleri
- ↳ Generic Store Pattern
- ↳ TodoService — Specific Logic
- ↳ Kullanım Örnekleri
- ↳ Bu Mini Projede Ne Öğrendik?
- ↳ Kendi Pratiğin (Egzersizler)

→ Modül 2 Özeti

✓ Modül 3 — İlk Angular Uygulaması, CLI & DevTools

→ Modül 3'e Giriş

→ Ders 3.1 — ng new ile Proje Oluşturma

- ↳ ng new Komutu — Anatomy
- ↳ CLI'nin Sorduğu Sorular
- ↳ ng new Sonrası Ne Olur?
- ↳ Yeni Projeye İlk Bakış
- ↳ Yaygın Hatalar ve Çözümleri
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 3.2 — Proje Yapısı Anatomisi

- ↳ Klasör Yapısı — Genel Bakış
- ↳ src/ Klasörü — Kalbi
- ↳ src/app/ — Component'lerin Yuvası
- ↳ Konfigürasyon Dosyaları

- ↳ public/ ve Diğer Klasörler
- ↳ Hangi Dosyaya Ne Sıklıkta Dokunmalıyım?
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 3.3 — Angular CLI Komutları

- ↳ CLI Felsefesi
- ↳ ng serve — Geliştirme Sunucusu
- ↳ ng generate — Kod Üretici
- ↳ ng build — Production Build
- ↳ ng test, ng update, ng add
- ↳ --dry-run ve --help
- ↳ Cheat Sheet
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 3.4 — app.config.ts & Bootstrapping

- ↳ Bootstrap Akışı
- ↳ Eski NgModule vs Modern Standalone
- ↳ ApplicationConfig Anatomisi
- ↳ Yaygın provideX() Fonksiyonları
- ↳ Custom Provider'lar
- ↳ Environment-Specific Config
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 3.5 — Angular DevTools

- ↳ Kurulum ve Aktivasyon
- ↳ Component Explorer
- ↳ Profiler
- ↳ Router Tree
- ↳ Injector Tree
- ↳ Pratik Workflow İpuçları
- ↳ Yaygın Sorunlar ve Çözümleri
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 3.6 — MCP Server (v21 Yeniliği)

- ↳ Problem: AI'ların Eski Bilgisi
- ↳ MCP (Model Context Protocol) Nedir?
- ↳ Angular MCP Server'ı Çalıştırma
- ↳ Cursor IDE'ye Entegrasyon
- ↳ Claude Code (Terminal) Entegrasyonu
- ↳ VS Code Copilot Entegrasyonu
- ↳ Angular MCP Tools
- ↳ Pratik İpuçları
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 3.7 — Mini Proje: Counter App

- ↳ Proje Tanımı
- ↳ Adım 1: Yeni Proje Oluştur
- ↳ Adım 2: app.ts Component'i
- ↳ Adım 3: app.html Template'i

- ↳ Adım 4: app.scss Styling
- ↳ Adım 5: Test ve DevTools
- ↳ Pekiştirme: Ne Öğrendin?
- ↳ Egzersizler
- ↳ Mini Quiz

→ Modül 3 Özeti

► Modül 4 — Component'lar & Modern Template Syntax

→ Modül 4'e Giriş

→ Ders 4.1 — Component Anatomisi

- ↳ Component Nedir?
- ↳ Component'in Tam Yapısı
- ↳ @Component Decorator Anatomisi
- ↳ Naming Conventions
- ↳ Lifecycle Hooks
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 4.2 — Standalone Components

- ↳ Hatırlatma: NgModule Problemi
- ↳ Standalone Components Çözümü
- ↳ NgModule vs Standalone — Yan Yana
- ↳ Component Composition
- ↳ imports Array'ine Ne Konabilir?
- ↳ Standalone ile Bootstrap
- ↳ Eski Projeyi Migrate Etmek
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 4.3 — Data Binding 4 Çeşidi

- ↳ Data Binding Genel Bakış
- ↳ Interpolation — {{ value }}
- ↳ Property Binding — [prop]
- ↳ Event Binding — (event)
- ↳ Two-way Binding — [(value)]
- ↳ Pratik Örnek — Hepsi Bir Arada
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 4.4 — Modern Control Flow

- ↳ Neden Yeni Syntax?
- ↳ @if — Koşullu Render
- ↳ @for — Liste Render
- ↳ @switch — Çoklu Koşul
- ↳ Pratik Örnek — Hepsi Bir Arada
- ↳ Eski Syntax'tan Migrate
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 4.5 — @let ile Template Değişkenleri

- ↳ Problem: Tekrar Eden Expression'lar
- ↳ Çözüm: @let
- ↳ Pratik Senaryolar

- ↳ @let vs computed()
- ↳ @let Sınırlamaları
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 4.6 — input(), output(), model()

- ↳ input() — Component'e Veri Geçme
- ↳ output() — Event Yayma
- ↳ model() — Two-way Binding
- ↳ Pratik Örnek — Üçü Bir Arada
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Mini Proje: Angular ToDo App

- ↳ Proje Özellikleri
- ↳ Proje Yapısı
- ↳ Adım Adım Yapım
- ↳ Bu Projede Ne Öğrendin?
- ↳ Egzersizler

→ Modül 4 Özeti

Gelişim · Junior → Mid-Level

- Modül 5 — Signals & Reactivity
 - 5.1 Signal nedir? (teori)
 - 5.2 Signal API tam referans
 - 5.3 linkedSignal (v19+)
 - 5.4 resource API (v19+)
 - 5.5 Signal queries (viewChild, viewChildren)
 - 5.6 Mini proje: Reactive filter & search
- Modül 6 — Directives & Pipes
 - 6.1 Directive türleri
 - 6.2 Custom attribute directive
 - 6.3 Host directives
 - 6.4 Built-in pipes
 - 6.5 Custom pipe
 - 6.6 Mini proje: Highlight & format library
- Modül 7 — Services & Dependency Injection
 - 7.1 Service nedir?
 - 7.2 inject() vs constructor injection
 - 7.3 Provider scope (hierarchical injection)
 - 7.4 InjectionToken & multi-providers
 - 7.5 Modern provider patterns: provideX()
 - 7.6 Mini proje: Pluggable logger service
- Modül 8 — Project Structure & Feature-Based Architecture
 - 8.1 Neden klasör yapısı önemli?
 - 8.2 Feature-based architecture
 - 8.3 core / shared / features ayrımı
 - 8.4 Bağımlılık kuralları

- 8.5 Index files (barrel exports)
- 8.6 Path aliases (tsconfig.json)
- 8.7 ESLint rules ile mimariyi korumak
- 8.8 Mini proje: Project skeleton
- Modül 9 — Routing & Navigation
 - 9.1 Temel routing
 - 9.2 Route parameters & component input binding
 - 9.3 Functional guards
 - 9.4 Functional resolvers
 - 9.5 Lazy loading routes
 - 9.6 Nested routes
 - 9.7 Mini proje: Multi-role dashboard
- Modül 10 — RxJS Deep-Dive
 - 10.1 Observable: stream over time
 - 10.2 Operators: 4 kategori
 - 10.3 Higher-order mapping operators (kritik)
 - 10.4 Subjects (Subject, BehaviorSubject, ReplaySubject)
 - 10.5 Hot vs cold observables
 - 10.6 Error handling
 - 10.7 Memory leak prevention
 - 10.8 RxJS + signals interop
 - 10.9 Mini proje: Smart search
- Modül 11 — HTTP & Server Communication
 - 11.1 HttpClient setup
 - 11.2 CRUD işlemleri
 - 11.3 Functional interceptors
 - 11.4 httpResource() (v19+ modern yaklaşım)
 - 11.5 File upload & download
 - 11.6 HTTP caching patterns
 - 11.7 Mini proje: Blog API client
- Modül 12 — State Management (NgRx Signal Store)
 - 12.1 Hangi araç ne zaman?
 - 12.2 Service + signal ile basit state
 - 12.3 NgRx Signal Store setup
 - 12.4 Signal Store anatomisi
 - 12.5 Component'ta kullanım
 - 12.6 withEntities (entity management)
 - 12.7 Feature store (component-scoped)
 - 12.8 Mini proje: Shopping cart
- Modül 13 — Angular Material & CDK
 - 13.1 Neden UI library?
 - 13.2 UI library karşılaştırması
 - 13.3 Angular Material setup
 - 13.4 Theming (Material Design 3)
 - 13.5 Sık kullanılan component'lar
 - 13.6 MatDialog (modal)
 - 13.7 Angular CDK nedir?
 - 13.8 CDK drag & drop

- 13.9 CDK overlay (tooltip, popover)
- 13.10 CDK virtual scrolling
- 13.11 CDK layout (responsive)
- 13.12 Mini proje: Material admin dashboard

Olgunlaşma · Mid → Senior

- Modül 14 — Forms (Reactive + Signal Forms)
 - 14.1 Forms yaklaşımları
 - 14.2 Reactive Forms temelleri
 - 14.3 Typed forms (v14+)
 - 14.4 FormArray (dynamic lists)
 - 14.5 Custom validators
 - 14.6 Cross-field validation
 - 14.7 Conditional validation
 - 14.8 Signal Forms (v21 experimental)
 - 14.9 Mini proje: Multi-step wizard
- Modül 15 — Animations
 - 15.1 Setup
 - 15.2 Trigger, state, transition
 - 15.3 :enter & :leave
 - 15.4 Stagger (liste animasyonları)
 - 15.5 Route animations
 - 15.6 Reduced motion desteği
 - 15.7 Mini proje: Animated card gallery
- Modül 16 — Lazy Loading & @defer
 - 16.1 3 seviyeli lazy loading
 - 16.2 @defer tetikleyicileri
 - 16.3 Tam @defer sözdizimi
 - 16.4 Prefetch stratejileri
 - 16.5 NgOptimizedImage
 - 16.6 Mini proje: E-commerce product page
- Modül 17 — WebSockets & Real-Time Communication
 - 17.1 Real-time: 3 yaklaşım
 - 17.2 RxJS WebSocket subject
 - 17.3 Reconnection strategy
 - 17.4 Server-sent events (SSE)
 - 17.5 Socket.IO entegrasyonu
 - 17.6 Mini proje: Real-time chat
- Modül 18 — i18n (Internationalization)
 - 18.1 Yaklaşımlar
 - 18.2 Transloco (önerilen)
 - 18.3 Locale-aware pipes
 - 18.4 RTL support
 - 18.5 Pluralization & ICU messages
 - 18.6 Mini proje: Multi-language blog
- Modül 19 — Accessibility (a11y) & Angular Aria

- 19.1 Neden a11y?
- 19.2 WCAG 2.2 (POUR)
- 19.3 Semantic HTML first
- 19.4 ARIA attributes
- 19.5 Angular Aria (v21 yeniliği)
- 19.6 Focus management
- 19.7 Klavye navigasyonu
- 19.8 Testing
- 19.9 Mini proje: Accessible form & modal

○ Modül 20 — Security

- 20.1 Web security temelleri (OWASP)
- 20.2 XSS koruması
- 20.3 DomSanitizer
- 20.4 Content security policy
- 20.5 CSRF protection
- 20.6 JWT storage stratejileri
- 20.7 Dependency security
- 20.8 Sensitive data handling
- 20.9 Mini proje: Secure auth flow

○ Modül 21 — Testing (Vitest + Playwright)

- 21.1 Test pyramidi
- 21.2 Vitest setup
- 21.3 Component testing
- 21.4 Service mocking
- 21.5 Signal testing
- 21.6 Signal Store testing
- 21.7 E2E with Playwright
- 21.8 Mini proje: %80+ coverage

Profesyonel • Senior

○ Modül 22 — Performance & Change Detection

- 22.1 Change detection: zone vs zoneless
- 22.2 OnPush vs default
- 22.3 Memory leaks
- 22.4 Bundle size optimization
- 22.5 Track-by optimization
- 22.6 Performance profiling
- 22.7 Web vitals
- 22.8 Mini proje: Performance audit

○ Modül 23 — Error Handling & Logging

- 23.1 Neden önemli?
- 23.2 Global ErrorHandler
- 23.3 HTTP error interceptor
- 23.4 User-friendly error UI
- 23.5 Sentry entegrasyonu
- 23.6 Logging service pattern
- 23.7 Performance monitoring

23.8 Mini proje: Error-resilient app

○ Modül 24 — Environment Config & Build Configurations

- 24.1 Environment files
- 24.2 angular.json build configurations
- 24.3 Runtime configuration
- 24.4 Feature flags
- 24.5 Build optimization
- 24.6 Source maps stratejisi
- 24.7 Mini proje: Multi-environment setup

○ Modül 25 — SSR, Hydration & Web Vitals

- 25.1 Neden SSR?
- 25.2 Setup
- 25.3 Hydration
- 25.4 Component-level hydration (v21)
- 25.5 Server-only / client-only code
- 25.6 SEO meta tags
- 25.7 Mini proje: SEO-optimized blog

○ Modül 26 — PWA (Progressive Web Apps)

- 26.1 PWA nedir?
- 26.2 Setup
- 26.3 Service worker caching
- 26.4 Update notifications
- 26.5 Push notifications
- 26.6 Install prompt
- 26.7 Offline detection
- 26.8 Mini proje: Offline-first notes app

Expert · Senior → Expert

○ Modül 27 — Microfrontend Architecture

- 27.1 Microfrontend nedir?
- 27.2 Module Federation (Webpack)
- 27.3 Native Federation (modern alternatif)
- 27.4 Inter-MFE communication
- 27.5 @defer + Module Federation
- 27.6 Mini proje: E-commerce microfrontend

○ Modül 28 — Design Patterns

- 28.1 Smart vs dumb components
- 28.2 Repository pattern
- 28.3 Facade pattern
- 28.4 Strategy pattern
- 28.5 Observer pattern (built-in)
- 28.6 Mini proje: Architecture refactor

○ Modül 29 — CI/CD & Deployment

- 29.1 Build pipeline anatomisi
- 29.2 GitHub Actions
- 29.3 Docker

- 29.4 Vercel / Netlify deployment
- 29.5 Cloudflare Pages
- 29.6 Self-hosting (Nginx + VPS)
- 29.7 SSR deployment stratejileri
- 29.8 Environment variables in CI
- 29.9 Mini proje: Full CI/CD pipeline

Capstone • Final Proje

- Modül 30 — Capstone — Task Management Platform
 - H1 Mimari tasarım + project skeleton
 - H2 Shell + auth + routing + Material setup
 - H3 MFE-Projects (CRUD, formlar)
 - H4 MFE-Tasks (@defer, drag-drop, real-time)
 - H5 MFE-Reports (dashboard, virtual scroll)
 - H6 State management (NgRx Signal Store)
 - H7 i18n + a11y + security audit
 - H8 Testing + error handling + monitoring
 - H9 SSR + PWA
 - H10 CI/CD + production deployment

Modül 1'e Hoş Geldin

Bu modülde Angular dünyasının kapısını aralayacağız. Henüz çok fazla kod yazmayacağız — önceliğimiz kavramları doğru anlamak. Bir framework'ü öğrenmek, sadece syntax'ını ezberlemek değil; o framework'ün neden var olduğunu, hangi problemleri çözdüğünü ve nasıl düşündüğünü anlamaktır.

Bu modülün sonunda:

- ✓ Frontend framework'lerinin neden var olduğunu anlayacaksın
- ✓ Angular'ın felsefesini ve diğer framework'lerden farkını bileceksin
- ✓ Angular v21'in büyük yeniliklerini tanıyacaksın
- ✓ Geliştirme ortamını kurmuş ve ilk uygulamayı çalıştırmış olacaksın



Öğrenme Felsefesi

Bu eğitimde "Neden?" sorusunu sürekli soracağız. Bir konsept gördüğünde sadece "nasıl yazılır" değil, "neden böyle yazılır, alternatifleri ne, ne zaman kullanılır" sorularını da kavramaya çalışacağız.

MODÜL 1 — DERS 1.1

Frontend Frameworks: Neden Var?

*Bu derste Angular'a kod yazmadan önce asıl soruyu cevaplayacağız:
"Neden framework'lere ihtiyaç var? Vanilla JS ile yapamaz mıyız?"*



Hedef

Bu sorunun cevabı, sonraki tüm derslerin temelini oluşturuyor. Framework'lerin neden var olduğunu, hangi problemleri çözdüklerini kavrayacağız.

1

Web'in Tarihsel Hikayesi

Angular'ı anlamak için önce nereden geldiğimizi anlamalıyız.

❑ Taş Devri: Static HTML (1990'lar)

İlk web siteleri sadece HTML dosyalarıydı. Sunucu sana bir HTML gönderiyordu, sen okuyordun, başka bir sayfaya gitmek için yeni bir HTML talep ediyordun.

```
Kullanıcı: "Anasayfayı göster"  
Sunucu: <html>...</html> gönderir  
Kullanıcı bağlantıya tıklar  
Sunucu: Yeni <html>...</html> gönderir
```

Sorun: Her tıklamada tüm sayfa yeniden yükleniyor. Çok yavaş, çok kötü deneyim.

❑ Demir Devri: jQuery & DOM Manipulation (2000'ler)

JavaScript geldi. Artık sayfayı yeniden yüklemekten DOM'u (Document Object Model) değiştirebiliyoruz.

```
// jQuery dönemi  
$('#user-name').text('John');  
$('#user-list').append('<li>Yeni kullanıcı</li>');  
$('.delete-btn').click(function() {  
  $(this).parent().remove();  
});
```

Bu büyük bir devrimdi! Ama bir sorun var...

❑ Cehennem: Spaghetti Code (2010'lar)

Uygulamalar büyüdükçe DOM manipülasyonu kabusla dönüştü. Senaryo: Bir e-ticaret sitesi düşün. Kullanıcı sepete ürün ekliyor. Sen şunları manuel

güncellemek zorundasın:

```
function addToCart(product) {  
  // 1. Sepet sayısını güncelle (header'da)  
  $('#cart-count').text(parseInt($('#cart-count').text()) + 1);  
  
  // 2. Sepet listesine ekle (sidebar'da)  
  $('#cart-list').append('<li>${product.name}</li>');  
  
  // 3. Toplam fiyatı hesapla (footer'da)  
  let total = parseFloat($('#total').text()) + product.price;  
  $('#total').text(total.toFixed(2));  
  
  // 4-8. Notification, empty mesajı, button enable, localStorage, anal  
  ytics...  
}
```

Görüyor musun problemi? Tek bir state değişikliği için, DOM'un 8 farklı yerini manuel olarak senkronize ediyorsun. Hangi birini unutursan UI bozulur. Buna "imperative programming" deniyor: "Şunu yap, şunu yap, şunu yap..."

□ Aydınlanma: Declarative UI (2013+)

Birisi şu fikri attı: "Ya UI, state'in bir fonksiyonu olsa? Ben state'i değiştirsem, UI kendi kendini güncellese?"

```
UI = f(state)
```

İşte modern frontend framework'leri (React, Vue, Angular) bu fikrin ürünü.

2 Vanilla JS'in Sınırları (Somut Örneklerle)

Belki "Bence vanilla JS ile her şey yapılabilir" diye düşünüyor olabilirsin. Doğru, yapılabilir. Ama maliyeti ne?

Sorun 1: State Senkronizasyonu

Tek state değişikliği = onlarca DOM güncellemesi. Vanilla JS'in en büyük problemi.

Sorun 2: Component Reusability

Bir component'i 5 sayfada kullanırsan, kopyala-yapıştır yapmak zorundasın. Tasarım değişince hepsini tek tek güncelle.

Sorun 3: Event Handling Cehennemi

AJAX ile DOM'a yeni element eklendiğinde event listener'ları yeniden attach etmek gerekiyor. Karmaşık.

Sorun 4: Routing

Tek sayfalık uygulama yapmak için URL hash'leri, History API, component switching — hepsini kendin yazmalısın.

Sorun 5: Form Validation

10 alanlı bir form için 300+ satır kod. Framework ile 30 satır.

3

SPA (Single Page Application) Konsepti

MPA (Multi Page Application) — Geleneksel

Anasayfa.html → Tıkla → Profil.html → Tıkla → Ayarlar.html
(yenile) (yenile) (yenile)

Her tıklamada: tüm HTML, CSS, JS yeniden indirilir. Tüm sayfa render edilir. Yavaş.

SPA — Single Page Application

Anasayfa → URL değişir → Profil bileşenini yükle → Ayarlar bileşenini yükle
(sayfa yenilenmez!)

Tek bir HTML dosyası var (index.html). JavaScript, hangi bileşenin gösterileceğine karar veriyor. Sayfalar arası geçiş anlık. Mobil uygulama gibi hissettirir.



SPA'nın Avantajları

- Hızlı navigasyon
- Mobil app benzeri deneyim
- Background'da veri çekme (loading state)
- Smooth transitions



SPA'nın Dezavantajları

- İlk yükleme yavaş (tüm JS bundle'ı indirilir)
- SEO problemleri (bot'lar JS çalıştıramayabilir)
- İlk paint geç



Spoiler

Bu dezavantajları çözmek için SSR (Server-Side Rendering) var. Modül 25'te işleyeceğiz.

4 Component-Based Architecture

Modern framework'lerin temeli component'ler. Analoji: Lego tuğlası. Her tuğla kendi başına bir şey, ama birleştirence ev olur.

Component'in 3 Bileşeni

1. State (veri):

```
class UserCardComponent {  
  name = 'John Doe';  
  age = 30;  
  isOnline = true;  
}
```

2. Template (görünüm):

```
<div class="card">  
  <h3>{{ name }}</h3>  
  <p>Yaş: {{ age }}</p>  
  @if (isOnline) {  
    <span class="badge">Online</span>  
  }  
</div>
```

3. Style (stil):

```
.card {  
  padding: 1rem;  
  border-radius: 8px;  
}  
.badge {  
  background: green;  
  color: white;  
}
```

Component'lerin Süper Güçleri

Reusability: 5 farklı yerde kullan. Tasarım değişti? Tek dosyayı güncelle.

Encapsulation: Component'in CSS'i sadece o component'i etkiler.

Composition: Küçük bileşenler birleşerek karmaşık UI'lar oluşturur. Her parça bağımsız test edilebilir.

5 Reactivity Nedir?

Bu modern framework'lerin kalbi. Bu kavramı doğru anlamak çok kritik.

Klasik Programlama (Imperative)

```
let a = 5;
let b = 10;
let total = a + b; // total = 15

a = 20;
console.log(total); // 15! Çünkü total'i tekrar hesaplamadık
```

Reactive Programlama

```
let a = signal(5);
let b = signal(10);
let total = computed(() => a() + b()); // total = 15

a.set(20);
console.log(total()); // 30! Otomatik güncellendi.
```



Excel Analojisi

Reactivity'yi anlamak için Excel düşün:

A1 = 5, A2 = 10, A3 = =A1+A2 (formül)

A3 hücresi 15 olur. A1'i 20 yaparsan, A3 otomatik 30 olur. Çünkü A3, A1'e bağımlı.

İşte reactivity bu. UI'nın state'e Excel hücrelerinin formüllerine bağlı olduğu gibi bağlı olması.

Angular'da Reactivity

```
@Component({
  template: `
    <input [(ngModel)]="name" />
    <p>Merhaba, {{ name() }}!</p>
    <p>Adın {{ name().length }} karakter.</p>
  `,
})
export class GreetingComponent {
  name = signal('Dünya');
}
```

Kullanıcı input'a yazdığı anda: signal güncellenir, UI otomatik güncellenir. Sen tek satır DOM kodu yazmadın!

6 Vanilla JS vs Framework — Karşılaştırma

Aynı küçük "Counter" uygulamasını iki şekilde yazalım:

Vanilla JS Versiyonu

```
<div id="app">
  <p>Sayı: <span id="count">0</span></p>
  <button id="increment">+1</button>
  <button id="decrement">-1</button>
</div>

<script>
  let count = 0;
  const countEl = document.getElementById('count');

  document.getElementById('increment').addEventListener('click', () =>
  {
    count++;
    countEl.textContent = count; // Manuel DOM update
  });

  document.getElementById('decrement').addEventListener('click', () =>
  {
    count--;
    countEl.textContent = count; // Yine manuel
  });
</script>
```

Sorunları: 3 farklı yerde DOM update kodu var. Yeni buton ekleyince yine manuel update gerekecek. "Sayı negatifse kırmızı" gibi kural eklenince DOM güncelleme dağınık.

Angular Versiyonu

```
@Component({
  selector: 'app-counter',
  template: `
    <p [class.negative]="count() < 0">Sayı: {{ count() }}</p>
    <button (click)="count.update(n => n + 1)">+1</button>
    <button (click)="count.update(n => n - 1)">-1</button>
    <button (click)="count.set(0)">Reset</button>
  `,
  styles: [`.negative { color: red; }`]
})
export class CounterComponent {
  count = signal(0);
}
```

Avantajları: State ve UI net şekilde ayrı. Manuel DOM update yok. "Negatifse kırmızı" kuralı declarative. Test edilmesi çok kolay.

7

Bu Eğitim Boyunca Kullanacağımız Felsefe

Bu eğitim boyunca 3 prensibi içselleştireceksin:

1. State'i Tek Yerde Tut

```
// ❌ Kötü – state DOM'da saklı
const count = parseInt(document.getElementById('count').textContent);

// ✅ İyi – state component'te, DOM sadece görünüm
count = signal(0);
```

2. UI = f(state)

UI, state'in bir fonksiyonu olmalı. State değişince UI otomatik güncellensin.

3. Component = Bağımsız Birim

Her component:

- Kendi state'ini yönetir
- Kendi template'ine sahiptir
- Kendi CSS'i kendisini etkiler
- Tek başına test edilebilir
- Tek başına anlamlıdır

□ Mini Quiz

- **Vanilla JS ile yapılabilecek bir şey neden bir framework gerektirir?**

Cevap: Yapılabilir ile verimli, sürdürülebilir, hatasız yapılabilir farklı şeylerdir. Framework'ler state senkronizasyonu, component reusability, routing, forms gibi karmaşık işleri standardize eder.

- **SPA ile MPA arasındaki temel fark nedir?**

Cevap: MPA'da her sayfa geçişi yeni bir HTML talebi yapar (sayfa yenilenir). SPA'da tek bir HTML var; navigasyon JavaScript ile yönetilir, sayfa yenilenmez.

- **Imperative ve declarative programlama arasındaki fark nedir?**

Cevap: Imperative: "Şunu yap, şunu yap." (DOM'u manuel güncelle). Declarative: "Şu state şu UI'a karşılık gelir." (State'i değiştir, UI otomatik güncellenir).

- **Reactivity'yi Excel analojisiyle anlat.**

Cevap: Excel'de bir hücreye $=A1+A2$ formülü yazınca, A1 veya A2 değişince o hücre otomatik güncellenir. Reactivity de tam olarak bu.

- **Component'in 3 ana bileşeni nedir?**

Cevap: State (veri/logic — TypeScript), Template (görünüm — HTML), Style (stil — CSS/SCSS).

⚠ Anti-Patterns

Daha kod yazmadık ama düşünme şekli olarak şunlardan kaçın:

- ✗ "Framework gereksiz, vanilla JS ile yapılır" — Yapılır, ama maliyeti devasa.
- ✗ DOM'u state olarak kullanmak — DOM görünüm. State component'te tutulur.
- ✗ "Manuel her şeyi kontrol ederim" — Reactive sistem 1000 kat daha güvenilir.
- ✗ jQuery alışkanlıklarını taşımak — `$('#elem').click(...)` mantığı modern Angular'da yok.

□ Bu Dersten Çıkarman Gerekenler

- ✓ Frontend framework'lerinin neden var olduğunu biliyorum
- ✓ SPA'nın ne olduğunu ve geleneksel web'den farkını biliyorum
- ✓ Component-based mimarinin avantajlarını anlıyorum
- ✓ Reactivity konseptini Excel analojisiyle açıklayabiliyorum
- ✓ "UI = f(state)" felsefesini içselleştirdim

MODÜL 1 — DERS 1.2

Angular Tarihçesi & Felsefesi

Angular'ın neden bu hale geldiğini, hangi problemleri çözmek için tasarlandığını ve diğer framework'lerden nasıl ayrıştığını anlayacağız.



Hedef

Bu ders biraz "tarih dersi" gibi gelebilir, ama emin ol — kodun arkasındaki felsefeyi anlamadan, kod yazarken sürekli "neden böyle yapılmış?" sorusuyla mücadele edeceksin.

1

AngularJS — Her Şeyin Başlangıcı (2010)

Hikaye

2009 yılında Google'da çalışan Miško Hevery, ufak bir yan proje yapmaya başladı. Amacı: web tasarımcılarının HTML'i daha güçlü kullanmasını sağlamak. Ortaya çıkan şey: AngularJS.

Devrim Niteliğindeki Fikirler

```
<!-- AngularJS 2010 -->
<div ng-controller="UserCtrl">
  <input ng-model="name" />
  <p>Merhaba {{ name }}!</p>
</div>
```

Bu kod 2010'da BÜYÜK bir devrimdi. Çünkü:

- Two-way data binding — Input değişince name değişiyor, otomatik!
- Directives — HTML'i extend etme imkanı (ng-model, ng-repeat...)
- Dependency Injection — Backend kavramı frontend'e ilk kez geldi
- Two-way binding magic — DOM ve JS senkron, manuel update yok

AngularJS'in Sorunları

☐ Performance: Digest cycle her event sonrası tüm watcher'ları kontrol ediyordu. 1000 watcher'lı sayfada çok yavaştı.

☹ Karmaşık API: \$scope, \$rootScope, \$apply, \$digest, \$watch... Yeni başlayan için kabus.

☐ TypeScript Yok: JavaScript'te yazılıyordu. Büyük projelerde tip güvenliği yok.

□ Modular Değil: Tüm uygulama tek bir bundle. Lazy loading karmaşık.

2 Büyük Yıkım ve Yeniden Doğuş — Angular 2 (2016)

□ Cesur Karar

Google ekibi şu kararı verdi: "AngularJS'i fırlat. Sıfırdan yaz. Geriye uyumluluk yok." Bu çok tartışmalı bir karardı. Topluluk ikiye bölündü.

Felsefe Değişiklikleri

Konu	AngularJS	Angular 2
Dil	JavaScript	TypeScript
Mimari	MVC + Scope	Component-based
Module	angular.module(...)	NgModule
Change Detection	Digest cycle	Zone.js + zone-aware CD
Mobile	Zayıf	Mobile-first
DI	\$injector	Hierarchical Injector
Tooling	Yetersiz	Angular CLI

Neden TypeScript?

1. Ölçeklenebilirlik: 100K satırlık projede typo yazdığın anda IDE uyarır.
2. IntelliSense / Autocomplete: IDE tüm property'leri gösterir.
3. Refactoring: Property adı değişince TypeScript tüm kullanım yerlerini gösterir.
4. Dokümantasyon: `function getUserId(id: number): Promise<User | null>` — kod okuyarak ne yaptığını biliyorsun.

3 Angular'ın Felsefesi — "Opinionated" Olmak

Angular'ı diğer framework'lerden ayıran en büyük şey: opinionated olması.

"Opinionated" Ne Demek?

Opinionated framework: "Şunu şöyle yap, şu şekilde değil." Standartları dayatır.

Unopinionated framework: "Sen bilirsin, istediğin gibi yap."

Karşılaştırma

Soru	React	Angular
Routing nasıl?	React Router, Wouter... seç	Built-in Angular Router
State management?	Redux, Zustand... seç	NgRx Signal Store (resmi)
Forms?	react-hook-form, Formik... seç	Built-in Reactive Forms
HTTP?	fetch, axios... seç	Built-in HttpClient
Testing?	Jest, Vitest, Cypress... seç	Vitest (resmi)
CSS?	CSS modules, styled... seç	Component-scoped CSS
CLI?	Yok	Angular CLI (resmi)

Hangisi Daha İyi?

Bu felsefi bir soru. İkisinin de avantajları var:



React'in Esnekliği

Pro: Her ihtiyaca özel araç seç • Innovation hızlı • Hafif öğrenme eğrisi
Con: Her takım farklı stack • "Best practice" değişken • Junior'lar yolunu kaybedebilir



Angular'ın Standardı

Pro: Tüm projelerde tutarlı yapı • Senior her projeye hızlı adapte • "Doğru yol" belli • Enterprise için ideal
Con: Esneklik az • Innovation yavaş • Daha büyük initial bundle

Angular kurumsal odaklı. Büyük ekipler, uzun ömürlü projeler, yüksek code quality için tasarlandı.

4

Angular'ın Versiyon Tarihi — Önemli Dönüm Noktaları

Versiyon	Yıl	Önemli Yenilik
Angular 2	2016	Yeniden doğuş, TypeScript zorunlu
Angular 9	2020	Ivy renderer (büyük performance kazancı)
Angular 14-15	2022	Standalone components
Angular 16	2023	Signals (preview)
Angular 17	2023	@if/@for/@switch, @defer
Angular 19	2024	linkedSignal, resource API
Angular 20	2025	Zoneless stable
Angular 21	2025	Zoneless DEFAULT, Signal Forms, Vitest, MCP

Anahtar Noktalar

- Angular 9 — Ivy Renderer: Bundle size %40 küçüldü, build hızı 2x oldu.
- Angular 14-15 — Standalone Components: NgModule'ün ölüm fermanı. Boilerplate %50 azaldı.
- ✂ Angular 16 — Signals (Preview): Yeni reaktivite sistemi. Zone.js'in sonunun başlangıcı.
- Angular 17 — Modern Template Syntax: @if/@for, %90 daha okunabilir.
- Angular 21 — Zoneless Default: Sen şu anda bu versiyonu öğreniyorsun!

5

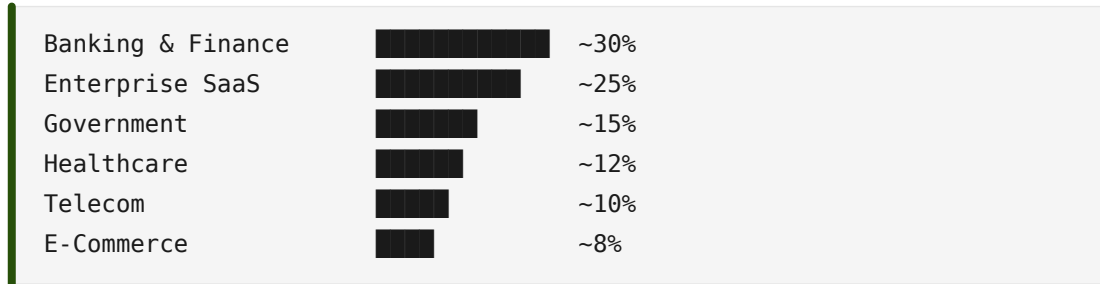
Angular Kim Tarafından Kullanılıyor?

Belki şöyle düşünebilirsin: "React her yerde, Angular niye öğreneyim?" Cevap: Angular kurumsal dünyanın kralı.

Angular Kullanan Büyük Şirketler

-  Google — Gmail, AdWords, Maps console
-  Microsoft — Office 365 web, Outlook web
-  Forbes — Forbes.com
-  Delta — Booking sistemi
-  Upwork — Freelance platformu
-  Deutsche Bank — Internal tools
-  BNP Paribas — Banking platforms
-  Wealthsimple — FinTech

Sektör Dağılımı



Çıkarım: Stabil, uzun vadeli, iyi maaşlı işler için Angular çok güçlü.

6

Angular'ın Geleceği

Kısa Vadeli (1-2 yıl)

-  Signal Forms stable olacak (şu an experimental)
-  Zoneless her yerde standart olacak
-  MCP Server ile AI-assisted development güçlenecek
-  Native Federation Module Federation'ın yerini alacak

Orta Vadeli (3-5 yıl)

-  Server Components (React'tan ilham, Angular yorumu)
-  Streaming SSR olgunlaşacak
-  Edge Runtime desteği (Cloudflare Workers, Deno Deploy)
-  Mobile-first improvements

7

Bu Eğitimde Neden Angular v21?

Angular v21, 5 yıl önceki Angular ile karşılaştırılamayacak kadar farklı. Modern, hızlı, type-safe, AI-friendly.

"Eski Kodu Öğrenmek Gerekmez Mi?"

Gerekirse öğrenirsin. Ama modern ile başlamak çok daha kolay. Çünkü:

- Eski Angular bilgisi modern Angular'a tam transfer olmaz
- Migration'ı zaten Modül 4'te göreceğiz
- Yeni kod yazarken modern best practice'leri direkt uygulayacaksın



Felsefe

"İyi bir Angular developer eski kodu okuyabilir, ama yeni kod yazarken modern olanı yazar."

□ Mini Quiz

- **AngularJS ve Angular 2+ arasındaki en büyük fark nedir?**

Cevap: AngularJS (1.x) ve Angular 2+ tamamen farklı framework'ler. Angular 2 sıfırdan yazıldı, TypeScript zorunlu hale geldi, MVC yerine component-based mimariye geçildi.

- **"Opinionated framework" ne demek?**

Cevap: Standartları dayatan, "doğru yolu" belirleyen framework. Angular routing, forms, HTTP, testing için resmi çözümler sunar — sen seçim yapmazsın.

- **Angular neden TypeScript zorunluluğu getirdi?**

Cevap: Büyük projelerde tip güvenliği, IDE desteği, refactoring kolaylığı ve self-documenting code için. Enterprise odaklı bir framework için TypeScript şart.

- **Angular 21'in en önemli yeniliği nedir?**

Cevap: Zoneless Change Detection'ın artık default olması. Bunun yanında Signal Forms (experimental), Vitest, Angular Aria, MCP Server.

- **Angular ne tür projeler için en uygundur?**

Cevap: Büyük takımlar, uzun ömürlü kurumsal projeler. Banking, healthcare, government, enterprise SaaS gibi sektörlerde özellikle güçlü.

- **NgModule'den Standalone'a geçiş neden önemliydi?**

Cevap: Boilerplate'in azaltılması, daha kolay tree-shaking, lazy loading'in basitleşmesi ve modern JavaScript module sistemine yaklaşma.

⚠ Anti-Patterns

- ✗ AngularJS ile Angular'ı karıştırmak — İki ayrı framework, bilgi transfer etmiyor
- ✗ "Angular = React'in alternatifi" sanmak — Hayır, ikisi farklı kategoriler
- ✗ Angular 1.x tutorial'larını takip etmek — Tamamen yanlış paradigma
- ✗ "Angular ölüyor" söylemine inanmak — Kurumsal dünyada hala büyüyor
- ✗ Modern Angular'ı eski Angular gibi yazmak — *ngIf, NgModule, class-based guard

□ Bu Dersten Çıkarman Gerekenler

- ✓ AngularJS ve Angular'ın iki ayrı framework olduğunu biliyorum
- ✓ Angular'ın opinionated olduğunu ve avantajlarını anlıyorum
- ✓ TypeScript'in Angular için neden zorunlu olduğunu kavradım
- ✓ Angular'ın enterprise odaklı felsefesini benimsiyorum
- ✓ Angular vs React vs Vue arasındaki bağlamsal farkları biliyorum
- ✓ v21'in neden özel bir versiyon olduğunu anladım

MODÜL 1 — DERS 1.3

Angular vs React vs Vue — Kod Karşılaştırması

"Angular, React, Vue arasındaki farklar nedir?" sorusunu kod yazmadan anlamak imkansız. Aynı uygulamayı üç framework'te yazıp karşılaştıracamız.



Hedef

Ders sonunda, ne zaman hangisini seçeceğini kendi gözünle göreceksin.

1

Karşılaştırma Senaryosu

Üç framework'te de aynı uygulamayı yazacağız: Mini Todo List.

Özellikler:

- ➡ Yeni todo ekleme
- ☐ Todo'yu tamamlandı işaretleme
- ☐ Todo silme
- ☐ İstatistik gösterimi
- ☐ Tamamlanmış todo'lar üstü çizili

Bu uygulama küçük ama her framework'ün temel API'lerini kullanıyor: state management, event handling, list rendering, conditional rendering, computed values.

2

React Versiyonu

```
import { useState } from 'react';

function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [newTodoText, setNewTodoText] = useState('');

  const addTodo = () => {
    if (!newTodoText.trim()) return;
    setTodos([...todos, {
      id: Date.now(),
      text: newTodoText,
      completed: false
    }]);
    setNewTodoText('');
  };

  const toggleTodo = (id) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    ));
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  const completedCount = todos.filter(t => t.completed).length;

  return (
    <div>
      <input
        value={newTodoText}
        onChange={(e) => setNewTodoText(e.target.value)}
        onKeyDown={(e) => e.key === 'Enter' && addTodo()}
      />
      <button onClick={addTodo}>Ekle</button>

      <ul>
        {todos.map(todo => (
          <li key={todo.id} style={{
            textDecoration: todo.completed ? 'line-through' : 'none'
          }}>
            <input type="checkbox" checked={todo.completed}
              onChange={() => toggleTodo(todo.id)} />
            <span>{todo.text}</span>
            <button onClick={() => deleteTodo(todo.id)}>Sil</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```
    )})  
  </ul>  
  
  {todos.length > 0 && (  
    <p>{completedCount} / {todos.length} tamamlandı</p>  
  )}  
</div>  
);  
}
```

React'in Felsefesi

"Sadece state'ini değiştir, ben re-render'ı hallederim. Ama dikkat: state immutable, side effect'ler useEffect içinde."

Dikkat Edilecek Noktalar

- useState hook — State yönetimi, her setState re-render tetikler
- JSX — JavaScript içinde HTML benzeri syntax, attribute'lar camelCase
- Immutable updates — Yeni array oluşturmak zorundasın, mutation yasak
- Inline expressions — JS doğrudan template'te
- {condition && <element/>} — JS truthy/falsy ile conditional
- key prop zorunlu — Her item için unique key

3 Vue Versiyonu


```
<script setup>
import { ref, computed } from 'vue';

const todos = ref([]);
const newTodoText = ref('');

const addTodo = () => {
  if (!newTodoText.value.trim()) return;
  todos.value.push({
    id: Date.now(),
    text: newTodoText.value,
    completed: false
  });
  newTodoText.value = '';
};

const toggleTodo = (id) => {
  const todo = todos.value.find(t => t.id === id);
  if (todo) todo.completed = !todo.completed;
};

const deleteTodo = (id) => {
  todos.value = todos.value.filter(t => t.id !== id);
};

const completedCount = computed(() =>
  todos.value.filter(t => t.completed).length
);
</script>

<template>
  <input v-model="newTodoText" @keydown.enter="addTodo" />
  <button @click="addTodo">Ekle</button>

  <ul>
    <li v-for="todo in todos" :key="todo.id"
      :class="{ completed: todo.completed }">
      <input type="checkbox" v-model="todo.completed" />
      <span>{{ todo.text }}</span>
      <button @click="deleteTodo(todo.id)">Sil</button>
    </li>
  </ul>

  <p v-if="todos.length > 0">
    {{ completedCount }} / {{ todos.length }} tamamlandı
  </p>
</template>
```

Vue'nun Felsefesi

"Geliştirici deneyimi her şeyden önemli. Template'in temiz olsun, mutation'la state değiştir, v- directive'leriyle güç al."

Dikkat Edilecek Noktalar

- `ref(...)` — Reactive state, `.value` ile okuma/yazma
- `computed(...)` — Türetilmiş değer
- `v-model` — Two-way binding
- `v-for`, `v-if` — Liste/conditional rendering
- `@click`, `@keydown.enter` — Event binding ve modifier'lar
- `<style scoped>` — Component-scoped CSS, otomatik
- Mutation OK — Vue'da mutation yapabilirsin (React'te yapamazsın)

4

Angular Versiyonu (v21 — Şu An Öğrendiğimiz)

```
import { Component, signal, computed } from '@angular/core';
import { FormsModule } from '@angular/forms';

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

@Component({
  selector: 'app-todo',
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="newTodoText"
      (keydown.enter)="addTodo()" />
    <button (click)="addTodo()">Ekle</button>

    <ul>
      @for (todo of todos(); track todo.id) {
        <li [class.completed]="todo.completed">
          <input type="checkbox" [checked]="todo.completed"
            (change)="toggleTodo(todo.id)" />
          <span>{{ todo.text }}</span>
          <button (click)="deleteTodo(todo.id)">Sil</button>
        </li>
      }
    </ul>

    @if (todos().length > 0) {
      <p>{{ completedCount() }} / {{ todos().length }} tamamlandı</p>
    }
  `,
})
export class TodoComponent {
  todos = signal<Todo[]>([]);
  newTodoText = '';

  completedCount = computed(() =>
    this.todos().filter(t => t.completed).length
  );

  addTodo() {
    if (!this.newTodoText.trim()) return;
    this.todos.update(list => [...list, {
      id: Date.now(),
      text: this.newTodoText,
      completed: false
    }]);
  }
}
```

```
    }]);  
    this.newTodoText = '';  
  }  
  
  toggleTodo(id: number) {  
    this.todos.update(list =>  
      list.map(todo => todo.id === id  
        ? { ...todo, completed: !todo.completed }  
        : todo)  
    );  
  }  
  
  deleteTodo(id: number) {  
    this.todos.update(list => list.filter(t => t.id !== id));  
  }  
}
```

Angular'ın Felsefesi

"Yapı, standardizasyon, type safety. Component nasıl yazılır net belli. 5 kişilik takım da, 500 kişilik takım da aynı şekilde yazar."

Dikkat Edilecek Noktalar

- signal() — Reactive state
- computed() — Türetilmiş değer
- TypeScript zorunlu — interface Todo ile şekil garanti
- Template syntax: [(ngModel)], (click), [class.x], [checked]
- Modern control flow: @for (track ZORUNLU), @if
- Component'in 3 parçası net — Decorator, class, template
- Decorator-based — @Component({...})

5 Yan Yana Karşılaştırma — Tablo

Konu	React	Vue	Angular
State definition	useState(0)	ref(0)	signal(0)
State okuma	count	count.value / count	count()
State yazma	setCount(5)	count.value = 5	count.set(5)
Computed	useMemo()	computed()	computed()
Two-way binding	value + onChange	v-model	[(ngModel)]
Event	onClick={fn}	@click="fn"	(click)="fn()"
Conditional	{cond && X}	v-if="cond"	@if (cond) { }
List	.map() + key	v-for + key	@for + track
Mutation izinli mi?	✗ Hayır	✓ Evet	✗ Hayır
CSS scoping	3rd party	scoped (built-in)	styles (built-in)
TypeScript	Opsiyonel	Opsiyonel	✓ Zorunlu

6 "Bu Kod Bana Ne Söylüyor?" — Felsefi Çıkarımlar

□ React: "JavaScript First"

React, JavaScript'in kendisine çok yakın. JSX bile aslında JavaScript. Conditional? && operatörü. List? .map(). State update? setState(...).

Avantaj: İyi JS biliyorsan, React'i 1 günde öğrenirsin.

Dezavantaj: Best practice'ler community-driven, "doğru yol" tartışmalı.

□ Vue: "Template First"

Vue, template syntax'a yatırım yapmış. v-model, v-if, v-for, :class... her şeyin direkt template'te bir karşılığı var.

Avantaj: HTML/CSS bilen biri için çok hızlı öğrenilir.

Dezavantaj: Vue'ya özel syntax çok (DSL).

❑ Angular: "Structure First"

Angular, yapı ve tutarlılık üstüne kurulu. Decorator, separate template, type-safe state, opinionated structure.

Avantaj: 100K satırlık projede hala temiz kalır.

Dezavantaj: Küçük proje için overkill, başlangıçta daha çok concept.

7

"Hangisini Öğrenmeliyim?" Kararı

Eğer Sen...	Tercih
❑ Kurumsal projelerde çalışacaksan	Angular
❑ Startup'ta hızlı iterasyon yapacaksan	React veya Vue
❑ Mobile app da yapacaksan	React (React Native)
❑ UI-heavy ürün geliştireceksen	React veya Vue
❑❑ İlk framework olarak öğreniyorsan	Vue (yumuşak eğri)
❑ İş bulma odaklıysan	Bölgeye bakar — Avrupa: Angular, ABD: React, Asya: Vue

Sen Şu An Angular'ı Öğreniyorsun. İyi mi Karar?

Evet, çok iyi karar. Çünkü:

- ✓ Senior Angular developer'lar çok aranıyor
- ✓ Kurumsal sektör stabil ve iyi maaşlı
- ✓ Angular kullananların çoğu uzun ömürlü projelerde
- ✓ TypeScript'in derinine ineceksin (her yerde işe yarar)
- ✓ Backend developer'larla çalışmak kolaylaşır
- ✓ Bir framework'ü gerçekten anlayan biri, diğerlerini hızlı öğrenir



Bilmen Gereken Sır

Bir framework'te uzmanlaştığında, diğerlerini öğrenmek 1-2 hafta. Konseptler aynı (state, component, reactivity), sadece syntax değişiyor.

□ Mini Quiz

- **React, Vue ve Angular arasındaki en temel felsefi fark nedir?**

Cevap: React = JavaScript first (esnek). Vue = Template first (HTML-centric). Angular = Structure first (opinionated, enterprise).

- **Angular'da neden mutation yapamıyoruz, Vue'da yapabiliyoruz?**

Cevap: Angular signals immutable update bekler. Vue'nun reactivity sistemi proxy-based, mutation'ı tracking edebiliyor. Angular daha "predictable", Vue daha "magic".

- **@for ... track, .map() + key, v-for + :key neden var?**

Cevap: List rendering yaparken framework'ün hangi item'ın hangisi olduğunu bilmesi gerekiyor (DOM diffing için). Unique identifier olmadan, item silindiğinde/eklendiğinde framework yanlış DOM update yapabilir.

- **Angular'ın bundle size'ı diğerlerinden daha büyük. Bu Angular'ı kötü yapıyor mu?**

Cevap: Hayır. Bundle size küçük projede önemli, büyük projede önemsizleşir. Angular zaten built-in çok şey getiriyor (router, forms, http, di). Ayrıca v21 zoneless ile bundle ~%50 küçüldü.

- **Senior Angular developer, React öğrenmek isterse ne kadar sürer?**

Cevap: Yaklaşık 1-2 hafta. Konseptler aynı, sadece syntax ve ekosistem farklı.

⚠ Anti-Patterns

- ✗ "Hangisi en iyi?" sorusunu sormak — Yanlış soru. Bağlama göre doğru framework değişir.
- ✗ Birden fazla framework'ü aynı projede kullanmak — Çok özel senaryolar dışında kabus.
- ✗ React'i Angular gibi yazmak veya tersi — Her birinin felsefesi farklı.
- ✗ Bundle size obsesyonu — Modern web'de 50KB fark çoğu zaman önemsiz.
- ✗ Performans karşılaştırma testlerine takılmak — Çoğu sentetik, gerçek projeleri yansıtmıyor.

□ Bu Dersten Çıkarman Gerekenler

- ✓ Üç framework'ün kod düzeyinde farklarını gördüm
- ✓ Her birinin felsefesini anlıyorum (JS-first, Template-first, Structure-first)
- ✓ Hangisinin hangi senaryoda uygun olduğunu biliyorum
- ✓ Angular'ın opinionated olmasının kod üzerinde etkilerini gördüm
- ✓ Bir framework'te uzmanlaşınca diğerlerinin kolay öğrenileceğini biliyorum
- ✓ Angular'ı öğrenmenin kariyer açısından doğru bir karar olduğunu hissediyorum

MODÜL 1 — DERS 1.4

Angular v21'in "Big Picture" Yenilikleri

v21'in getirdiği büyük yenilikleri kavramsal düzeyde tanıyacağız. Her birinin ne olduğu, neden önemli olduğu ve hangi problemi çözdüğü.



Önemli Not

Bu ders bir "tanıştırma" dersi. Kodda derinlemesine kullanmayı kendi modüllerinde göreceğiz. Şimdilik amaç: Angular v21 hakkında konuşurken hangi terimin neye karşılık geldiğini bilmek.

1

v21 Bir "Geçiş Versiyonu" Değil

Angular v21, 5 yıl önceki Angular ile karşılaştırılamayacak kadar farklı. Versiyonlar bazen küçük güncellemeler getirir. v21 öyle değil. Angular'ın mental modelini değiştiren birçok büyük yeniliği bir araya getiren versiyon.

Bu derste 6 büyük yeniliği inceleyeceğiz:

- ⚡ Zoneless Change Detection (artık default)
- 📄 Signal Forms (experimental)
- ♿ Angular Aria (accessibility primitives)
- 🧪 Vitest (Karma yerine)
- 🖥️ MCP Server (AI destekli geliştirme)
- 🎨 Tailwind CSS (yeni proje scaffold'ında)

2

Zoneless Change Detection — Devrimin Kalbi

Bu yenilik en önemlisi. Angular'ın çalışma mantığını temelinden değiştiriyor.

Önce: Zone.js Nedir?

Zone.js, 2014'ten beri Angular'ın gizli kalbi. JavaScript'in tüm async API'lerini monkey-patch eder, Angular'a haber verir.

```

setTimeout(() => {
  this.count++;
}, 1000);

// 1. Zone.js araya girer
// 2. "Async işlem başladı" Angular'a haber verir
// 3. Bittiğinde "şimdi her şeyi kontrol et" der
// 4. Angular tüm component'leri tekrar render eder

```

Bu sihir gibi çalışıyordu, ama 3 problemi vardı:



Sorun 1: Performance

Zone.js her async olayda tüm change detection cycle'ı tetikliyor. 100 component için 99 gereksiz kontrol.



Sorun 2: Bundle Size

Zone.js ~100KB. Her Angular uygulamasının yanında zorunlu olarak geliyor. Özellikle mobil için önemli.



Sorun 3: Predictability

Zone.js "magic" çalışıyor. Bir şey beklediğin gibi olmadığında debug etmek çok zor. Third-party library uyumsuzlukları.

Şimdi: Zoneless Dünya

Angular ekibi şu fikre çalıştı: "Ya değişiklikleri signals ile takip etsek? Tüm async olayları monkey-patch etmek yerine, gerçekten değişen yerleri bilelim?"

```

@Component({
  template: `<p>{{ count() }}</p>`
})
export class CounterComponent {
  count = signal(0);

  increment() {
    this.count.update(n => n + 1);
    // 1. Signal'in değerini değiştirir
    // 2. Signal "hey, ben değiştim" der
    // 3. Angular SADECE bu signal'i kullanan component'i günceller
    // 4. Diğer hiçbir component check edilmez
  }
}

```

Avantajları

- ✓ Daha hızlı — Sadece gerçekten değişen kısımlar render edilir
- ✓ Daha küçük bundle — Zone.js gereksiz, ~100KB tasarruf
- ✓ Daha öngörülebilir — "Magic" yok, signal değişince render olur
- ✓ Daha kolay debug — Component neden render oldu? net cevap
- ✓ Better SSR — Hydration daha hızlı

v21'de Ne Değişti?

```
// app.config.ts (v21 default)
export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(), // ← v21 default
    provideRouter(routes),
    provideHttpClient(),
  ]
};
```

v21'de: `provideZonelessChangeDetection()` artık stable. Yeni proje oluştururken default olarak soruluyor.

3 Signal Forms — Forms'un Geleceği

Hikaye

Angular'da forms her zaman bir konu olmuş. Şu anda 2 yaklaşım var: Template-Driven Forms (eski, basit ama güçsüz) ve Reactive Forms (şu anki standart, güçlü ama verbose).

Reactive Forms (Şu Anki Standart)

```
form = this.fb.nonNullable.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(8)]]
});
```

✓ Type-safe, ✓ Karmaşık formlar için ideal, ✗ Verbose, ✗ Signals ile uyumsuz

Gelecek: Signal Forms (v21'de Experimental)

"Form'lar zaten state. Signals state için. Bunları birleştirelim."

```
import { form, required, email, minLength } from '@angular/forms/signal
s';

@Component({...})
export class LoginComponent {
  // 1. State signal olarak
  data = signal({
    email: '',
    password: ''
  });

  // 2. Form, signal'den oluşturuluyor
  loginForm = form(this.data, (path) => {
    required(path.email);
    email(path.email);
    required(path.password);
    minLength(path.password, 8);
  });
}
```

Neden Devrimsel?

- ✓ Type-safe by default — TypeScript signal'in tipini biliyor
- ✓ Daha az boilerplate — FormControl, FormGroup yok
- ✓ Signal-native — Diğer signals ile sorunsuz çalışır

✓ Validation declarative — `required(path.email)` net

**Önemli Uyarı: v21'de Experimental**

Bu özellik henüz production-ready değil. API değişebilir. Strateji: Yeni projede Reactive Forms kullan, Signal Forms'u göz at. 1-2 yıl içinde stable olacak.

4

Angular Aria — Accessibility Devrimi

Hikaye

Accessibility (a11y) bugüne kadar Angular dünyasında 3rd party library'lerle çözülmüyordu. Material kullanmadan accessible bir combobox veya dropdown yazmak çok zordu.

Çözüm: Angular Aria

v21 ile Angular ekibi @angular/aria paketini çıkardı. Headless accessibility primitives sağlıyor: "Sana accessible component'ların mantığını vereyim. Görünüşü sen tasarla."

```
import { CdkAccordion } from '@angular/aria';

@Component({
  imports: [CdkAccordion],
  template: `
    <div cdkAccordion>
      <div cdkAccordionItem>
        <button cdkAccordionTrigger>Section 1</button>
        <div cdkAccordionPanel>Content...</div>
      </div>
    </div>
  `,
  styles: [`
    /* Sen styling'i hallediyorsun */
    [cdkAccordionPanel] { padding: 1rem; }
  `]
})
export class AccordionComponent {}
```

Headless Component Nedir?

Headless = Sadece davranış, görünüş yok.

Yaklaşım	Davranış	Görünüş	Esneklik
Angular Material	Built-in	Built-in (Material)	Düşük
Headless (Angular Aria)	Built-in	Sen yazıyorsun	Yüksek
Sıfırdan yazma	Sen yazıyorsun	Sen yazıyorsun	Çok yüksek (riskli)

Sağlanan Primitives

- ☐ Accordion — Açılır/kapanır panel'ler
- ☐ Combobox — Autocomplete input
- ☐ Listbox — Selectable list
- ☐ Menu — Dropdown menu
- ☐ Tabs — Tab navigation
- ☐ Tree — Hierarchical tree

Hepsi: Klavye navigasyonu, ARIA attributes otomatik, Screen reader uyumlu, Focus management.

5 Vitest — Karma'nın Sonu

Hikaye

Angular her zaman Karma + Jasmine ile test edilirdi. Sorunları: yavaş (browser başlatma), eski (2014'ten beri update yok), karmaşık config, kötü watch mode.

v21 Çözümü: Vitest Default

Yeni Angular projeleri artık Vitest ile geliyor. Karma artık opt-in.

```
import { TestBed } from '@angular/core/testing';
import { describe, it, expect } from 'vitest';

describe('CounterComponent', () => {
  it('should increment count', () => {
    const fixture = TestBed.createComponent(CounterComponent);
    fixture.componentInstance.setInput('initial', 0);
    fixture.detectChanges();

    fixture.componentInstance.increment();
    expect(fixture.componentInstance.count()).toBe(1);
  });
});
```

Avantajları

- ✓ 5-10x daha hızlı — Browser yerine jsdom
- ✓ Hot reload — Test değişince anında çalışır
- ✓ Modern API — Jest'e benzer (büyük community)
- ✓ Better mocking — vi.fn(), vi.mock(), vi.useFakeTimers()
- ✓ Built-in coverage — Ekstra plugin gerek yok

Migration

```
ng generate @angular/core:vitest-migration
```

Otomatik olarak karma.conf.js siler, vitest.config.ts ekler, test dosyalarını günceller.

6

MCP Server — AI Çağına Entegrasyon

Hikaye

2024-2025 AI'nın yazılım geliştirmeye girdiği yıllar. Anthropic Model Context Protocol (MCP) standartını çıkardı. AI agent'lar ile araçlar arasında iletişim için.

Angular ekibi: "Angular CLI'yi MCP Server yapalım. AI'lar Angular'a doğrudan bağlanabilsin, doğru Angular kodu yazabilsin."

Kullanımı

```
# Angular MCP Server'ı başlat
ng mcp

# Cursor'a entegre et:
# .cursor/mcp.json
{
  "mcpServers": {
    "angular": {
      "command": "npx",
      "args": ["-y", "@angular/cli", "mcp"]
    }
  }
}
```

Neden Devrimsel?



Eski Dünya (MCP'siz)

Cursor'a yazıyorsun: "Angular'da reactive form yap." Cursor 2 yıl eski training data'dan kod üretiyor — NgModule, *ngIf kullanan eski kod.



Yeni Dünya (MCP ile)

Cursor MCP üzerinden Angular CLI'ye bağlanıyor. find_examples ile kürate edilmiş örnekleri alıyor. v21 standartlarında kod üretiyor (signals, standalone, @if/@for).

Kullanılabilir MCP Tools

- `find_examples` — Angular ekibinin kürate ettiği örnekler
- `get_best_practices` — En güncel best practice'ler
- `search_documentation` — Resmi dokümandan arama
- `ai_tutor` — Etkileşimli AI öğretmen

- 🔄 modernize — Eski Angular kodunu v21'e dönüştürme



Sen Şu Anda Claude ile Çalışıyorsun

Angular MCP Server'ı kurarsan: Claude Code (terminal), Cursor (IDE), VS Code Copilot — hepsi doğrudan Angular'a bağlanır ve modern, doğru kod üretir.

7 Tailwind CSS — Yeni Default Styling

Tailwind Felsefesi

"Önceden tanımlanmış utility class'lar ile doğrudan HTML'de styling yap. Custom CSS dosyasına gitme."

```
<!-- Eski yaklaşım -->
<div class="user-card">
  <img class="user-avatar" />
  <h3 class="user-name">John</h3>
</div>
<style>
  .user-card { ... }
  .user-avatar { ... }
  .user-name { ... }
</style>

<!-- Tailwind yaklaşımı -->
<div class="p-4 bg-white rounded-lg shadow flex items-center gap-3">
  <img class="w-12 h-12 rounded-full" />
  <h3 class="text-lg font-semibold">John</h3>
</div>
```

Neden Default'a Eklendi?

Önceki versiyonlarda Tailwind kurmak karmaşıktı. v21'de: `ng new my-app` → "Use Tailwind CSS?" → Yes → Hepsi otomatik kurulur.

Avantaj-Dezavantaj

- ✓ Hızlı prototyping
- ✓ Tutarlı design system (spacing, colors auto)
- ✓ Küçük production CSS (purging)
- ✓ Responsive design kolaylığı
- ✗ HTML kalabalıklaşır (uzun class listeleri)
- ✗ Öğrenme eğrisi (class isimlerini ezberlemek)

☐ **Senin Stratejin**

Modül 1-12: Vanilla SCSS (temellere odaklanmak için)

Modül 13+: Angular Material (UI library)

Capstone: Tailwind opsiyonu açık. Şu an Tailwind öğrenmene gerek yok — öncelik Angular.

8

Diğer Önemli ama Daha Küçük Yenilikler

httpResource() Stable

Service yazmadan HTTP isteği:

```
users = httpResource<User[]>(() => ({  
  url: '/api/users',  
  params: { q: this.search() }  
}));
```

Component-Level Hydration Control

```
@Component({  
  hydrate: 'never' // veya 'always', 'whenIdle', 'whenVisible'  
})
```

@switch Fall-Through (v21.1)

```
@switch (status) {  
  @case ('pending')  
  @case ('processing') { <app-loading /> }  
  @case ('completed') { <app-success /> }  
}
```

Improved Error Messages

Compiler hataları artık çok daha açıklayıcı.

9

Bu Yenilikler Senin İçin Ne Demek?

Pratik Anlamda

- ✓ Daha hızlı uygulamalar — Zoneless ile bundle ~%50 küçük
- ✓ Daha temiz kod — Signal Forms ile boilerplate az
- ✓ Daha kolay test — Vitest ile %5-10x hızlı
- ✓ Daha iyi a11y — Angular Aria ile sıfırdan accessible
- ✓ AI ile çalışma — MCP Server ile modern kod
- ✓ Hızlı styling — Tailwind opsiyonel

Mental Model Değişimi



Eski Angular Düşüncesi

Async olay → Zone.js → Tüm app güncellenir
NgModule → declare et → export et → import et
*ngIf → ng-template → Else case



Yeni Angular Düşüncesi

Signal değişir → Sadece o yerler güncellenir
Standalone component → Import et → Kullan
@if → @else → @else if (HTML benzeri)

Migration Endişesi

Eski projeler ne olacak? Geriye uyumluluk var. Migration tool'larla:

```
ng update @angular/cli @angular/core          # Versiyona güncelle
ng generate @angular/core:zoneless-migration    # Zoneless'a geç
ng generate @angular/core:control-flow          # *ngIf → @if
ng generate @angular/core:standalone            # NgModule → Standalone
ng generate @angular/core:signal-input-migration # @Input → input()
ng generate @angular/core:vitest-migration      # Karma → Vitest
```

Bu komutlar otomatik refactor ediyor. Manuel iş minimum. Bu yüzden modern Angular öğrenmek eski Angular'ı öğrenmekten daha akıllıca.

□ Mini Quiz

- **Zone.js'in 3 ana sorunu neydi?**

Cevap: 1) Performance — gereksiz change detection. 2) Bundle size — ~100KB ek yük. 3) Predictability — "magic" davranış, debug zor.

- **Zoneless Change Detection'da render ne zaman tetiklenir?**

Cevap: Bir signal değiştiğinde. Angular sadece o signal'e bağlı component'leri günceller.

- **Signal Forms ile Reactive Forms arasındaki temel fark nedir?**

Cevap: Reactive Forms ayrı bir state sistemi. Signal Forms zaten signal'lerden oluşan state'i form olarak kullanır — daha az boilerplate, signals ile native uyum.

- **Angular Aria neden "headless" diye anılıyor?**

Cevap: Sadece accessibility davranışını sağlar (klavye, ARIA, focus). Görünüşü (CSS) sen yazarsın. Her tasarıma uyar.

- **Vitest, Karma'ya göre hangi avantajları sunar?**

Cevap: 5-10x daha hızlı, modern API, built-in coverage, daha iyi watch mode, daha kolay mocking.

- **MCP Server'ın temel amacı nedir?**

Cevap: AI agent'ların Angular CLI'ye bağlanıp güncel ve doğru Angular kodu üretmesini sağlamak. Training data eskimişlik sorununu çözer.

⚠ Anti-Patterns

- ✗ Zoneless'ı zone-aware library'lerle karıştırmak — Eski library'ler sorun çıkarabilir
- ✗ Production'da Signal Forms kullanmak — Henüz experimental, Reactive Forms tercih et
- ✗ MCP Server'sız AI ile Angular kodu üretmek — Eski kod alma riski yüksek
- ✗ Karma'da takılı kalmak — Vitest migration'ı yap, %5-10x hız kazanırsın
- ✗ Sıfırdan accessible component yazmaya çalışmak — Angular Aria primitives kullan

□ Bu Dersten Çıkarman Gerekenler

- ✓ Zoneless Change Detection'ın ne olduğunu ve neden devrim olduğunu biliyorum
- ✓ Signal Forms'un Reactive Forms'tan farkını anlıyorum
- ✓ Angular Aria'nın headless component yaklaşımını kavradım
- ✓ Vitest'in Karma'ya göre avantajlarını biliyorum
- ✓ MCP Server'ın AI çağındaki rolünü anladım

- ✓ Tailwind CSS'in v21'de opsiyonel olarak geldiğini biliyorum
- ✓ v21 mental model'ini hissetmeye başladım — signals merkez

MODÜL 1 — DERS 1.5

Geliştirme Ortamı Kurulumu

Bu derste kodlamaya hazır bir geliştirme ortamı kuracağız. Node.js, Angular CLI, VS Code, gerekli uzantılar ve ilk Angular projemizi başlatacağız.



Pratik Ders

Bu pratik bir ders. Anlatacağım, sen yapacaksın. Her adımı bilgisayarında uygulamayı bekliyorum.

1

Geliştirme Ortamımızda Neler Olacak?

Bir Angular geliştiricisinin olmazsa olmaz araçları:

- ☐ Node.js (LTS) — JavaScript runtime, npm dahil
- ☐ npm — Package manager (Node ile gelir)
- ☐ Angular CLI — Angular komut satırı aracı
- ☐ VS Code — Code editor
- ☐ Angular Language Service — IDE desteği
- ☐ ESLint + Prettier — Code quality + formatting
- ☐ Git — Version control
- ☐ Angular DevTools — Browser extension
- ☐ Modern bir tarayıcı — Chrome / Edge / Firefox

2

Node.js Kurulumu

Neden Node.js?

Angular bir frontend framework olsa da, build araçları Node.js üzerinde çalışır. Angular CLI, npm paketleri, dev server — hepsi Node.js gerektirir.

Hangi Sürüm?

Angular v21 için Node.js 20.x veya 22.x (LTS) öneriliyor.

i LTS Nedir?

LTS = Long Term Support. Stabil, güvenilir, uzun vadeli destek alan sürüm. Mutlaka LTS yükle, "Current" sürümü değil.

Kontrol: Node.js Yüklü mü?

```
node --version
```

Çıktı yorumlama:

- ✓ v20.18.0 ✓ Yüklü ve uygun
- ⚠ v18.19.0 ⚠ Çalışır ama eski
- ✗ v16.20.0 ✗ Çok eski
- ✗ command not found ✗ Yüklü değil

Windows Kurulum

Yöntem 1: Resmi Installer (Kolay)

- <https://nodejs.org> adresine git
- LTS versiyonu indir (yeşil button)
- .msi dosyasını çalıştır, varsayılanları kabul et

Yöntem 2: nvm-windows (Önerilen)

```
# https://github.com/coreybutler/nvm-windows/releases adresinden installer indir
nvm install lts
nvm use lts
node --version
```

macOS Kurulum

```
# Homebrew ile (önerilen)
brew install node@20

# Veya nvm ile
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.zshrc
nvm install --lts
nvm use --lts
```

Linux Kurulum (Ubuntu/Debian)

```
# NodeSource repo (önerilen)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Kontrol
node --version
npm --version
```

Doğrulama

```
node --version    # v20.x.x veya v22.x.x görmeli
npm --version     # 10.x.x görmeli
```

3 Angular CLI Kurulumu

Angular CLI Nedir?

Komut satırı aracı. Yeni proje oluşturma, component generate etme, build alma, test çalıştırma — hepsi Angular CLI ile.

Kurulum

```
npm install -g @angular/cli@latest
```

-g flag'i global demek — terminal'de her yerden ng komutu kullanabileceksin.

Doğrulama

```
ng version
```

Şöyle bir çıktı görmelisin:

```
Angular CLI: 21.0.0  
Node: 20.18.0  
Package Manager: npm 10.x.x  
OS: ...
```



Hata: "ng: command not found"

Bu olursa terminal'i kapat ve yeniden aç. Hala olmuyorsa PATH'e ekleme yapman gerekiyor:

macOS/Linux: `export PATH="$PATH:$(npm config get prefix)/bin"` satırını `.zshrc/.bashrc`'ye ekle.

4 VS Code Kurulumu ve Uzantılar

Neden VS Code?

Angular için en yaygın IDE. Microsoft'un ücretsiz editörü. Angular Language Service ve diğer uzantılarla mükemmel uyum.

Kurulum

<https://code.visualstudio.com> adresinden işletim sistemine uygun versiyonu indir ve kur.

Olmazsa Olmaz Uzantılar

VS Code aç → `Ctrl+Shift+X` → Aşağıdakileri arayıp yükle:

1 Angular Language Service

Template'lerde autocomplete, hata kontrolü, "Go to definition"

2 ESLint

Kod kalitesi kontrolü, real-time linting

3 Prettier - Code Formatter

Otomatik kod formatlama

4 EditorConfig for VS Code

Takım çalışmalarında tutarlı formatlama

Faydalı Uzantılar

- Material Icon Theme — Dosya/klasör ikonları
- Error Lens — Hataları satır içinde gösterir
- Path Intellisense — Import path autocomplete
- Auto Rename Tag — HTML tag otomatik yeniden adlandırma
- GitLens — Git geçmişi, blame, satır bazında commit
- Angular Snippets — Hızlı Angular template'leri

Toplu Kurulum

```
code --install-extension Angular.ng-template
code --install-extension dbaeumer.vscode-eslint
code --install-extension esbenp.prettier-vscode
code --install-extension EditorConfig.EditorConfig
code --install-extension PKief.material-icon-theme
code --install-extension usernamehw.errorlens
code --install-extension christian-kohler.path-intellisense
code --install-extension formulahendry.auto-rename-tag
code --install-extension eamodio.gitlens
code --install-extension johnpapa.Angular2
```

5 VS Code Ayarları

VS Code'un default ayarları iyi, ama Angular için birkaç custom ayar öneririm. Ctrl+Shift+P → "Preferences: Open User Settings (JSON)" → Aşağıdakini ekle:

```
{
  // Editor
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.tabSize": 2,
  "editor.wordWrap": "on",
  "editor.bracketPairColorization.enabled": true,

  // Files
  "files.eol": "\n",
  "files.insertFinalNewline": true,
  "files.trimTrailingWhitespace": true,
  "files.autoSave": "onFocusChange",

  // ESLint
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },

  // Prettier
  "prettier.singleQuote": true,
  "prettier.printWidth": 100,
  "prettier.semi": true,

  // Material Icons
  "workbench.iconTheme": "material-icon-theme",

  // TypeScript
  "typescript.preferences.importModuleSpecifier": "relative",
  "typescript.updateImportsOnFileMove.enabled": "always"
}
```

Ne yaptık?

- ✓ Format on save — Save'e bastığında kod otomatik formatlanır
- ✓ ESLint auto-fix — Save'de düzeltilebilir hatalar otomatik düzeltilir
- ✓ Single quote — 'abc' standardı
- ✓ Tab size 2 — Angular topluluğu standardı
- ✓ Auto save on focus change — Pencere değiştirdiğinde otomatik kaydet

6

Git Kurulumu

Version control olmadan profesyonel geliştirme olmaz.

Git Yüklü mü?

```
git --version
```

Yüklü Değilse

```
# Windows: https://git-scm.com adresinden indir
# macOS:
brew install git

# Linux:
sudo apt-get install git
```

İlk Konfigürasyon

```
git config --global user.name "Adın Soyadın"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main
```



Email Önemli

Email, GitHub hesabınla aynı olsun (commit'lerin GitHub'da sana atanır).

7

Angular DevTools Browser Extension

Bu olmazsa olmaz. Component tree görmek, signal graph incelemek, profiling yapmak için.

Chrome / Edge için

- Chrome Web Store: <https://chromewebstore.google.com>
- "Angular DevTools" ara
- "Add to Chrome" tıkla

Test

- Bir Angular sitesi aç (örn: <https://angular.dev>)
- F12 ile DevTools aç
- Üst sekme barında "Angular" sekmesi varsa kurulum başarılı ✓

8

İlk Angular Projemizi Oluşturma

Şimdi tüm aletlerimiz hazır. Hadi gerçek bir proje yaratıp çalıştıralım!

Adım 1: Proje Klasörü Hazırla

```
# macOS / Linux
cd ~/Documents
mkdir angular-egitim
cd angular-egitim

# Windows
cd C:\Users\YourName\Documents
mkdir angular-egitim
cd angular-egitim
```

Adım 2: Yeni Angular Projesi Oluştur

```
ng new hello-angular
```

Angular CLI sana sorular soracak:

? **Stylesheet format?**

SCSS seç. Vanilla CSS'in tüm özellikleri + nesting, variables, mixins.

? **Server-Side Rendering (SSR)?**

No seç. SSR'ı Modül 25'te göreceğiz.

? **Zoneless Change Detection?**

Yes seç! v21'in default'u ve gelecekteki standart bu.

? **Tailwind CSS?**

No seç. Vanilla SCSS ile başlayacağız, Material'a Modül 13'te geçeceğiz.

Adım 3-5: Projeyi Aç ve Başlat

```
cd hello-angular
code .          # VS Code'da aç
ng serve        # Dev server başlat
```

Birkaç saniye bekle. Şöyle bir çıktı göreceksin:

```
Application bundle generation complete. [1.234 seconds]
```

```
Watch mode enabled. Watching for file changes...
```

```
→ Local: http://localhost:4200/
```

Adım 6: Tarayıcıda Aç

Tarayıcıda `http://localhost:4200` adresine git.



Tebrikler!

"hello-angular app is running!" yazısını görmelisin. İlk Angular uygulamanız çalışıyor!

9

Hızlı Bir Düzenleme — Hot Reload Görelim

Şimdi gerçek bir geliştirici gibi davranalım.

Adım 1: `src/app/app.html` dosyasını aç

TÜMÜNÜ silin. Yerine şunu yapıştırın:

```
<main style="text-align: center; padding: 4rem;">
  <h1 style="font-size: 3rem; color: #dd0031;">
    Merhaba Angular v21!
  </h1>
  <p>Bu benim ilk Angular uygulamam.</p>
  <button (click)="count.set(count() + 1)">
    Tıkla beni: {{ count() }}
  </button>
</main>
```

Adım 2: `src/app/app.ts` dosyasını düzenle


```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.scss']
})
export class App {
  protected readonly title = signal('hello-angular');
  protected readonly count = signal(0); // ← Ekle bunu
}
```

Adım 3: Save (Ctrl+S / Cmd+S)

Tarayıcıya bak — otomatik refresh olduğunu göreceksin! Butona tıkla, sayı artıyor.



Az önce kullandığın şey:

- Bir signal (count = signal(0))
- Event binding ((click))
- Interpolation ({{ count() }})
- Hot Module Replacement (HMR) — kod değişince otomatik refresh

Modül 4 ve 5'te bunları derinlemesine işleyeceğiz.

10 Angular DevTools'u İlk Kez Kullanma

Tarayıcı hala açık. Şimdi DevTools'u açalım.

- F12 ile DevTools aç
- Üst sekme barında "Angular" sekmesine geç
- Solda component tree görürsün: App → RouterOutlet
- App'e tıkla. Sağda state görünür: title, count
- Butona tıkla. DevTools'taki count değeri otomatik güncellenir!

Profiler'ı Dene

- Üstte "Profiler" sekmesi → "Start recording"
- Birkaç kez butona tıkla
- "Stop recording"
- Karşına flame graph gelir — render time analizi



Profil Etmek

Bu büyük projelerde performance bottleneck'leri bulmak için altın değerinde bir araç.

11 Geliştirme Workflow'u

Tipik bir Angular geliştirme akışın şöyle olacak:

1. Terminal aç
2. cd proje-klasörü
3. ng serve → Dev server başlat
4. VS Code'da kod yaz → Save → Otomatik refresh
5. Browser'da test et
6. Angular DevTools ile debug et
7. git commit, git push → Versiyon kontrolü

12 Sık Karşılaşılan Problemler

Problem 1: "Port 4200 is already in use"

Sorun: Başka Angular projesi çalışıyor.

Çözüm: ng serve --port 4201

Problem 2: "Cannot find module '@angular/...'"

Sorun: Bağımlılıklar tam yüklenmedi.

Çözüm: `rm -rf node_modules package-lock.json && npm install`

Problem 3: VS Code'da kırmızı çizgiler ama derleme başarılı

Sorun: TypeScript Server takılmış.

Çözüm: `Ctrl+Shift+P` → "TypeScript: Restart TS Server"

Problem 4: "ng: command not found" (kurulumdan sonra)

Sorun: PATH'e eklenmemiş.

Çözüm: Yeni terminal aç. Hala olmuyorsa Bölüm 4'teki PATH ayarını yap.

13 Bonus — Pratik Komutlar Sözlüğü

Eğitim boyunca sık kullanacağın komutlar:

Proje yönetimi

```
ng new <proje-adı>          # Yeni proje
ng serve                    # Dev server (default 4200)
ng serve --port 4201        # Custom port
ng serve --open             # Otomatik tarayıcı aç
ng build                    # Development build
ng build --configuration production
ng test                     # Test runner
```

Code generation

```
ng generate component <name> # ng g c <name>
ng generate service <name>   # ng g s <name>
ng generate directive <name> # ng g d <name>
ng generate pipe <name>      # ng g p <name>
ng generate guard <name>     # ng g g <name>
ng generate interceptor <name>
```

Bağımlılık yönetimi

```
npm install                # Tüm paketleri yükle
npm install <paket>        # Yeni paket ekle
npm install -D <paket>     # Dev dependency
npm uninstall <paket>      # Paket sil
npm update                 # Güncelle
```

Angular güncelleme

```
ng update                  # Hangileri güncellenebilir
ng update @angular/cli @angular/core # Angular'ı güncelle
ng mcp                     # MCP Server (v21)
```

📝 Mini Quiz

- **Angular v21 için minimum Node.js sürümü nedir?**

Cevap: Node.js 20.x veya 22.x (LTS). Eski sürümler çalışmayabilir.

- **Angular CLI nasıl global olarak yüklenir?**

Cevap: `npm install -g @angular/cli@latest`

- **VS Code'da Angular için olmazsa olmaz uzantı hangisidir?**

Cevap: Angular Language Service (Angular ekibinin resmi uzantısı). Template autocomplete ve hata kontrolü için şart.

- **ng serve komutu ne yapar ve default port nedir?**

Cevap: Development server'ı başlatır, hot reload aktif eder. Default port: 4200.

- **Angular DevTools nedir, neden gerekli?**

Cevap: Browser extension. Component tree görmek, state inspect, change detection profiling için. Debug etmek olmazsa olmaz.

- **ng new sırasında "Zoneless Change Detection?" sorusuna ne diyeceksin?**

Cevap: Yes! v21'in default'u ve gelecekteki standart bu.

- **Hot Module Replacement (HMR) ne demek?**

Cevap: Kod değişikliklerinin tarayıcıda otomatik yansması. Save'e bastığında manuel refresh gerekmez.

⚠️ Anti-Patterns

- ✗ Global Angular CLI versiyonu ile proje versiyonu uyumsuzluğu — Hep güncel tut.
- ✗ node_modules ve dist klasörlerini git'e commit etmek — .gitignore'da zaten var.
- ✗ VS Code uzantısı kurmadan kod yazmaya çalışmak — Angular Language Service şart.
- ✗ Format on save kapalı — Kod tutarsızlığı, takım çalışmasında sorun.
- ✗ Angular DevTools'u görmezden gelmek — Console.log primitive bir yaklaşım.
- ✗ Eski Node sürümü kullanmak (v16 ve altı) — Güvenlik açıkları.

📝 Bu Dersten Çıkarman Gerekenler

- ✓ Node.js LTS kuruldu, sürümü doğruladım
- ✓ Angular CLI v21 global olarak yüklü
- ✓ VS Code ve önemli uzantılar hazır
- ✓ Git konfigüre edildi
- ✓ Angular DevTools browser extension'ı yüklü

- ✓ İlk Angular projem çalışıyor (hello-angular)
- ✓ Hot reload'un nasıl çalıştığını gördüm
- ✓ Temel Angular CLI komutlarını biliyorum

☐ Modül 1 Tamamlandı!

Tebrikler! Modül 1'i başarıyla tamamladın. Bu modülde Angular dünyasının kapısını araladık.

Neler Öğrendin?

Ders 1.1

Frontend framework'lerinin neden var olduğunu, vanilla JS'in sınırlarını, SPA konseptini, component-based mimariyi ve reactivity'yi öğrendin.

Ders 1.2

Angular'ın AngularJS'ten v21'e olan yolculuğunu, opinionated felsefesini, neden TypeScript zorunlu olduğunu öğrendin.

Ders 1.3

Angular vs React vs Vue'yu kod düzeyinde karşılaştırdın, her birinin felsefesini gördün.

Ders 1.4

v21'in büyük yeniliklerini (Zoneless, Signal Forms, Vitest, MCP, Angular Aria, Tailwind) kavramsal olarak öğrendin.

Ders 1.5

Geliştirme ortamını kurdun, ilk Angular uygulamayı çalıştırdın, hot reload'u gördün, DevTools'u kullandın.



Sıradaki Modül: TypeScript Refresher

Modül 2 — TypeScript'in Angular'da sıkça kullanılan özelliklerini hızlıca gözden geçireceğiz: Tipler, interfaces, generics, decorators, utility types, type guards. Bu konuları zaten biliyor olabilirsin, ama Angular bağlamında nasıl kullanılacağına odaklanacağız.

Modül 2'ye Hoş Geldin

Bu modülde Angular için kritik olan TypeScript konularını refresh edeceğiz. TypeScript'i biliyorsun, ama amacımız: Angular bağlamında nasıl kullanıldığını anlamak. Generics, decorators, utility types — Angular kodunun her satırında karşına çıkacak yapılar.

Bu modülün sonunda:

- ✓ TypeScript'in Angular'da nasıl kullanıldığını biliyor olacaksın
- ✓ Generics'in mantığını ve Angular'daki kullanımlarını kavrayacaksın
- ✓ Decorator'ların ne işe yaradığını ve modern alternatiflerini bileceksin
- ✓ Utility types ile günlük işlerini hızlandırabileceksin



Modül Yapısı

Bu modül 6 dersten oluşuyor: Temel Tipler, Generics, Decorators, Utility Types, Type Guards & Narrowing, ve son olarak Mini Proje ile öğrendiklerimizi pekiştireceğiz.

MODÜL 2 — DERS 2.1

Temel Tipler & Interfaces

TypeScript'in Angular için kritik olan temel tip sistemini gözden geçireceğiz. Sen TypeScript'i biliyorsun, ama amacımız: Angular bağlamında nasıl kullanıldığını anlamak.



Hedef

Bu modül bir TypeScript kursu değil. Hızlı bir refresher. Eğer TypeScript'e tamamen yabancıysan, önce TypeScript'in resmi dokümantasyonunu okuman önerilir.

1

Neden TypeScript? (Hızlı Hatırlatma)

Modül 1'de Angular'ın TypeScript'i zorunlu kıldığını söylemiştik. Şimdi pratik anlamda neden böyle olduğunu görelim.

Vanilla JavaScript Problemi

```
function getUser_name(user) {  
  return user.name;  
}  
  
getUser_name({ name: 'John' });           // 'John'  
getUser_name({ username: 'John' });      // undefined – bug!  
getUser_name('John');                    // hata!  
getUser_name(null);                       // hata!
```

JavaScript'te runtime'a kadar bu hataları göremezsin. Production'da bug çıkar.

TypeScript Çözümü

```
interface User {
  name: string;
}

function getUserName(user: User): string {
  return user.name;
}

getUserName({ name: 'John' });           // ✓ OK
getUserName({ username: 'John' });       // ✗ Compile error
getUserName('John');                     // ✗ Compile error
getUserName(null);                       // ✗ Compile error
```

Compile time'da hatalar yakalanıyor. Daha kod çalışmadan!

Angular'da Bunun Anlamı

```
@Component({
  template: `<p>{{ user.name }}</p>`
})
export class UserCardComponent {
  user = { name: 'John' }; // TypeScript otomatik {name: string} infer
  eder
}
```

Eğer template'te `{{ user.naem }}` (typo) yazsaydın, Angular Language Service anında uyarırdı.

2 Temel Tipler

Primitive Types

```
let isActive: boolean = true;
let age: number = 30;
let name: string = 'John';
let nothing: null = null;
let notDefined: undefined = undefined;
let big: bigint = 100n;
let unique: symbol = Symbol('id');
```

Type Inference (Çıkarım)

TypeScript çoğu zaman tipi otomatik çıkarır:

```
let count = 0;           // TypeScript: number
let name = 'John';       // TypeScript: string
let isActive = true;     // TypeScript: boolean
```



Best Practice

Tip çıkarımı belliyse, açıkça yazma. Daha okunabilir kod olur.

Arrays

```
// Her ikisi de aynı şey
let scores: number[] = [85, 92, 78];
let names: Array<string> = ['Alice', 'Bob'];

// Mixed values için union type
let mixed: (string | number)[] = ['John', 30, 'Jane', 25];
```

Object Types

İki yol var: inline ve interface.

```
// Inline (küçük objeler için)
let user: { name: string; age: number } = {
  name: 'John',
  age: 30
};

// Interface (yeniden kullanılacaksa) – önerilen
interface User {
  name: string;
  age: number;
}

let user: User = {
  name: 'John',
  age: 30
};
```

any ve unknown — Tehlikeli Kuzenler

```
let foo: any = 'hello';
foo.someRandomMethod();    // ✓ TypeScript şikayet etmez (TEHLİKELİ!)

let bar: unknown = 'hello';
bar.someRandomMethod();    // ✗ Error (unknown güvenli)

if (typeof bar === 'string') {
  bar.toUpperCase();        // ✓ OK (narrowed to string)
}
```

**Anti-Pattern**

any kullanmak. TypeScript'in tüm avantajını yok eder. Eğer tipi bilmiyorsan unknown kullan, güvenli.

3 Interfaces

Angular'da en sık kullanacağın yapı interface. Veri modellerini, API response'larını, component prop'larını tanımlar.

Temel Interface

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  isActive: boolean;  
}  
  
const user: User = {  
  id: 1,  
  name: 'John',  
  email: 'john@example.com',  
  isActive: true  
};
```

Optional Properties

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  phone?: string;      // ← Optional (? işaretli)  
  avatar?: string;     // ← Optional  
}  
  
const user1: User = { id: 1, name: 'John', email: 'j@x.com' }; // ✓ OK  
const user2: User = { id: 2, name: 'Jane', email: 'j@x.com', phone: '55  
5' };
```

Readonly Properties

```
interface User {
  readonly id: number;      // Sadece okunur
  name: string;
  email: string;
}

const user: User = { id: 1, name: 'John', email: 'j@x.com' };
user.name = 'Jane';        // ✓ OK
user.id = 2;                // ✗ Error: id is readonly
```

Index Signatures

Dinamik property'ler için:

```
interface FormErrors {
  [fieldName: string]: string;
}

const errors: FormErrors = {
  email: 'Invalid email',
  password: 'Too short',
  username: 'Already taken'
};
```

Interface Extension (Inheritance)

```
interface Person {
  name: string;
  age: number;
}

interface Employee extends Person {
  employeeId: string;
  department: string;
}

const emp: Employee = {
  name: 'John',           // Person'dan
  age: 30,                 // Person'dan
  employeeId: 'E001',     // Employee'den
  department: 'IT'        // Employee'den
};
```

Multiple Inheritance

```
interface Timestamped {  
  createdAt: Date;  
  updatedAt: Date;  
}  
  
interface Identifiable {  
  id: number;  
}  
  
interface User extends Identifiable, Timestamped {  
  name: string;  
  email: string;  
}
```


4 Type Aliases — interface vs type

İkisi de aynı şeyi yapıyor gibi. Ama farklılıklar var:

Sadece Interface Yapabilir

```
// 1. Declaration merging
interface Window {
  myCustomProp: string;
}
interface Window {
  anotherProp: number; // Aynı interface'e ekleme yapar
}
// Window artık her iki property'ye sahip

// 2. Class implements
class MyClass implements User {
  name = 'John';
  age = 30;
}
```

Sadece Type Yapabilir

```
// 1. Union types
type Status = 'pending' | 'active' | 'inactive';
type ID = string | number;

// 2. Tuple types
type Coordinate = [number, number];

// 3. Complex compositions
type User = { name: string } & { age: number }; // Intersection

// 4. Conditional types
type IsString<T> = T extends string ? true : false;
```



Angular Topluluğu Konvansiyonu

Interface → Modeller, DTO'lar, component prop'ları (90% kullanım)

Type → Union types, complex types, utility types

5 Union & Intersection Types

Union Types (|)

"Bu veya bu" demek:

```
type ID = string | number;

let userId: ID = 123;           // ✓
userId = 'abc-123';           // ✓ OK
userId = true;                 // x Error
```

Angular'da çok kullanılır:

```
// API response loading state
type LoadState = 'idle' | 'loading' | 'success' | 'error';

@Component({...})
export class UserListComponent {
  loadState = signal<LoadState>('idle');
}
```

Intersection Types (&)

"Bu ve bu" demek:

```
type Person = { name: string };
type Employee = { employeeId: string };

type EmployeePerson = Person & Employee;
// = { name: string; employeeId: string }

const emp: EmployeePerson = {
  name: 'John',
  employeeId: 'E001'
};
```

Discriminated Unions (Çok Önemli!)

Bu pattern Angular'da sıkça karşına çıkar:

```
type ApiResponse<T> =  
  | { status: 'success'; data: T }  
  | { status: 'error'; error: string }  
  | { status: 'loading' };  
  
function handleResponse(response: ApiResponse<User>) {  
  switch (response.status) {  
    case 'success':  
      console.log(response.data.name);    // ✓ data var  
      break;  
    case 'error':  
      console.log(response.error);        // ✓ error var  
      break;  
    case 'loading':  
      console.log('Loading...');          // data ve error YOK  
      break;  
  }  
}
```

TypeScript, status field'ına bakarak hangi property'lerin olduğunu otomatik bilir.
Çok güçlü!

□ Mini Quiz

- **any ve unknown arasındaki fark nedir?**

Cevap: any her şeye izin verir, type safety yoktur. unknown ise narrowing yapmadan kullanılamaz, daha güvenlidir.

- **interface ile type arasındaki temel fark nedir?**

Cevap: interface declaration merging yapabilir, class'lar implement edebilir. type ise union, intersection, tuple gibi advanced compositions için kullanılır. Object shape için interface (Angular konvansiyonu).

- **Discriminated union nedir?**

Cevap: Bir property'nin (genelde type veya status) değerine göre TypeScript'in hangi property'lerin var olduğunu otomatik belirleyebildiği union type pattern'i.

- **readonly modifier ne işe yarar?**

Cevap: Bir property'nin sadece okunabilir, değiştirilemez olduğunu belirtir. Angular'da ID'ler, immutable data modeller için sıkça kullanılır.

- **Hangi durumda interface extend ederiz?**

Cevap: Common field'ları (id, timestamps, vs.) bir kez tanımlayıp birden fazla model'de yeniden kullanmak için. DRY prensibi.

⚠ Anti-Patterns

- ✗ any kullanmak — TypeScript'i kullanmıyorsun demektir, unknown tercih et
- ✗ Inline object types her yerde — Tekrar kullanılacaksa interface yap
- ✗ Object tipini kullanmak — Çok generic, anlamsız. Spesifik interface yaz
- ✗ Backend response'unu UI'da direkt kullanmak — DTO ile Domain model'i ayır
- ✗ Type assertion (as) ile compile'ı susturmak — Önce gerçekten doğru olduğunu doğrula

□ Bu Dersten Çıkarman Gerekenler

- ✓ TypeScript'in neden Angular için zorunlu olduğunu anlıyorum
- ✓ Temel tipleri (primitive, array, object) doğru kullanabiliyorum
- ✓ interface ile veri modelleri tanımlayabiliyorum
- ✓ interface ve type arasındaki farkı biliyorum
- ✓ Union ve intersection type'larını anlıyorum
- ✓ Discriminated union pattern'ini Angular'da nasıl kullanacağımı görüyorum
- ✓ readonly, optional (?) modifier'larını biliyorum

MODÜL 2 — DERS 2.2

Generics

Generics'i gerçekten anlamak. "T harfi nedir?" sorusunun cevabını bulmak. Angular'da Signal, Observable, input() gibi yapıların arkasındaki mantığı kavramak.



Önemli

Generics ilk başta soyut gelir. Bu derste gerçek dünya örnekleriyle kavrayacağız.

1

Problem — Generics Olmasaydı

Generics'i anlamamanın en iyi yolu, olmadığı dünyayı hayal etmek.

Senaryo: Bir Function Yazıyoruz

Diyelim ki bir array'in ilk elemanını döndüren bir fonksiyon yazacaksınız:

```
function firstElement(arr: number[]): number {
  return arr[0];
}

const nums = [1, 2, 3];
const first = firstElement(nums); // first: number ✓
```

Şimdi bir de string array için yazmak istiyoruz. Her tip için ayrı ayrı fonksiyon mu yazacağız?

Çözüm Denemesi 1: any Kullan

```
function firstElement(arr: any[]): any {
  return arr[0];
}

const first = firstElement([1, 2, 3]);

// Sorun: first artık `any` tipinde
first.toUpperCase(); // ✓ TypeScript şikayet etmez (ama runtime'da h
ata!)
first + 1;           // ✓ TypeScript şikayet etmez (ama runtime'da N
aN!)
```

any ile type safety'yi kaybettik. Bu kötü.

Çözüm: Generics

```
function firstElement<T>(arr: T[]): T {  
  return arr[0];  
}  
  
const nums = [1, 2, 3];  
const first = firstElement(nums);    // first: number ✓ (otomatik!)  
  
const names = ['Alice', 'Bob'];  
const firstName = firstElement(names); // firstName: string ✓  
  
const users = [{name: 'John'}, {name: 'Jane'}];  
const firstUser = firstElement(users); // firstUser: {name: string} ✓
```



Sonuç

Tek fonksiyon, sınırsız tip! Type safety korundu.

2

<T> Sözdizimi — "T" Nedir?

Aslında Bir "Tip Parametresi"

Düşün ki normal bir fonksiyonda değer parametresi var. Generic'te ise tip parametresi var:

```
function firstElement<T>(arr: T[]): T {  
  return arr[0];  
}  
  
firstElement<number>([1, 2, 3]); // T = number (açık)  
firstElement([1, 2, 3]);         // T = number (otomatik infer)
```



Mental Model

bir placeholder. "Bu fonksiyonu çağırırken hangi tipi kullanacağını sonra söyleyeceğim, sen ona göre davran."

Neden "T"?

T = "Type". Bir konvansiyon. Aslında istediğin harfi/ismi kullanabilirsin.

Harf	Anlamı
T	Type (genel)
U, V	İkinci, üçüncü tipler
K	Key
V	Value
E	Element
R	Return
P	Props

3 Generic Constraints — Sınırlama

Bazen T her şey olamamalı. Sadece belirli bir özelliğe sahip tipler olmalı.

Çözüm: extends ile Constraint

```
interface HasLength {
  length: number;
}

function printLength<T extends HasLength>(item: T) {
  console.log(item.length); // ✓ Artık T'nin length'i olduğunu biliyoruz
}

printLength('Hello');           // ✓ string'in length'i var
printLength([1, 2, 3]);         // ✓ array'in length'i var
printLength({ length: 5 });     // ✓ kendimiz tanımladık
printLength(42);                // ✗ Error: number'ın length'i yok
```

keyof ile Constraint

Bir object'in property'lerinden birini seçtirmek istiyoruz:

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user = { name: 'John', age: 30, email: 'j@x.com' };

const name = getProperty(user, 'name'); // string
const age = getProperty(user, 'age');    // number
const wrong = getProperty(user, 'username'); // ✗ Error: 'username' user'da yok
```



Bu pattern süper güçlü

TypeScript, geçerli property isimlerini autocomplete ile gösterir!

4 Generic Interfaces

Generic'ler sadece function'larda değil, interface'lerde de kullanılır.

Klasik Örnek: API Response


```
interface ApiResponse<T> {  
  data: T;  
  status: 'success' | 'error';  
  message?: string;  
}  
  
// User listesi response'u  
const usersResponse: ApiResponse<User[]> = {  
  data: [{ name: 'John' }, { name: 'Jane' }],  
  status: 'success'  
};  
  
// Tek user response'u  
const userResponse: ApiResponse<User> = {  
  data: { name: 'John' },  
  status: 'success'  
};
```

Tek interface, sonsuz şekilde kullanılabilir!

Pattern: Pagination

```
interface PaginatedResponse<T> {  
  items: T[];  
  total: number;  
  page: number;  
  pageSize: number;  
  hasNext: boolean;  
}  
  
type UsersPage = PaginatedResponse<User>;  
type PostsPage = PaginatedResponse<Post>;  
type CommentsPage = PaginatedResponse<Comment>;
```

5 Generic Classes

Class'lar da generic olabilir.

Örnek: Generic Stack

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  get size(): number {
    return this.items.length;
  }
}

// Number stack
const numStack = new Stack<number>();
numStack.push(1);
numStack.push(2);

// String stack
const strStack = new Stack<string>();
strStack.push('hello');
```

6 Default Type Parameters

Tip parametrelerine default değer verilebilir.

```
interface ApiResponse<T = unknown> {  
  data: T;  
  status: 'success' | 'error';  
}  
  
// T belirtmedik, default unknown  
const response1: ApiResponse = { data: 'whatever', status: 'success' };  
  
// T'yi belirttik  
const response2: ApiResponse<User> = { data: { name: 'John' }, status:  
  'success' };  

```

Angular'da çok yaygın pattern. Örnek:

```
// HttpClient'ten örnek  
get<T = unknown>(url: string): Observable<T>;  
  
// Kullanım  
this.http.get('/api/data'); // Observable<unknown>  
this.http.get<User>('/api/users/1'); // Observable<User>  
this.http.get<User[]>('/api/users'); // Observable<User[]>  

```

7

Angular'da Generics — Gerçek Dünya

Şimdi Angular'da gördüğümüz generic'leri anlayalım.

Signal

```
import { signal, Signal } from '@angular/core';

// signal() generic bir fonksiyon
const count = signal<number>(0);
const name = signal<string>('John');
const user = signal<User>({ name: 'John', age: 30 });

// Type inference de çalışır
const count2 = signal(0);           // Signal<number>
const items = signal<string[]>([]); // Signal<string[]>
```

Observable (RxJS)

```
import { Observable, of } from 'rxjs';

const numStream: Observable<number> = of(1, 2, 3);
const userStream: Observable<User> = this.userService.getUser(1);
const usersStream: Observable<User[]> = this.userService.getAll();
```

HttpClient.get()

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
    //           ↑
    //           Burada T'yi belirtiyoruz
  }

  getUserById(id: number): Observable<User> {
    return this.http.get<User>(`/api/users/${id}`);
  }
}
```

**Anti-Pattern**

`this.http.get('/api/users')` (T belirtmeden) — response unknown olur, hiçbir şey yapamazsın.

input() ve input.required()

```
@Component({...})
export class UserCardComponent {
  // Optional input
  user = input<User | null>(null);

  // Required input
  userId = input.required<number>();

  // Default value ile
  size = input<'small' | 'medium' | 'large'>('medium');
}
```

8

Generic Constraint'lerle Pratik Örnek

Generic CRUD service yazıyoruz:

```
// 1. Tüm entity'ler bu interface'i implement etmeli
interface Entity {
  id: number;
}

// 2. Generic CRUD service
@Injectable()
export abstract class CrudService<T extends Entity> {
  protected abstract endpoint: string;
  private http = inject(HttpClient);

  getAll(): Observable<T[]> {
    return this.http.get<T[]>(`/api/${this.endpoint}`);
  }

  getById(id: number): Observable<T> {
    return this.http.get<T>(`/api/${this.endpoint}/${id}`);
  }

  create(item: Omit<T, 'id'>): Observable<T> {
    return this.http.post<T>(`/api/${this.endpoint}`, item);
  }

  update(id: number, item: Partial<T>): Observable<T> {
    return this.http.put<T>(`/api/${this.endpoint}/${id}`, item);
  }

  delete(id: number): Observable<void> {
    return this.http.delete<void>(`/api/${this.endpoint}/${id}`);
  }
}

// 3. User için kullan
interface User extends Entity {
  name: string;
  email: string;
}

@Injectable({ providedIn: 'root' })
export class UserService extends CrudService<User> {
  protected endpoint = 'users';
}
```

Tek base class, sonsuz CRUD service. Her biri type-safe.

□ Mini Quiz

- **Generics'in temel amacı nedir?**

Cevap: Type safety'yi kaybetmeden tek bir fonksiyon/class/interface'i farklı tiplerle kullanabilmek. any alternatifi olarak.

- **function firstElement(arr: T[]): T ifadesindeki ne işe yarar?**

Cevap: Tip parametresi tanımlar. "Bu fonksiyonu çağırırken hangi tipi kullanacaksan, T onun yerine geçecek." Placeholder gibi düşün.

- **ne demek?**

Cevap: Generic constraint. T herhangi bir tip olabilir AMA HasLength interface'ini implement etmeli (yani length property'si olmalı).

- **ne demek?**

Cevap: Default type parameter. T belirtilmezse, varsayılan olarak unknown kullanılır.

- **HttpClient.get('/api/users') ifadesinde neyi belirtiyor?**

Cevap: Response'un beklenen tipini. TypeScript'e "bu endpoint User array döndürecek" diyoruz.

- **Neden signal(0) Signal döndürür?**

Cevap: Type inference. TypeScript, parametreye bakıp T'yi otomatik çıkarır. 0 number olduğu için Signal döner.

⚠ Anti-Patterns

- ✗ Her yerde any kullanmak — Generic'leri öğrenmek bunu önler
- ✗ HttpClient.get('/api/users') — Response'u type-safe yapmak için belirt
- ✗ Constraint olmadan property'lere erişmek — T extends ... kullan
- ✗ Çok fazla generic parametre — Karmaşıklaşır, 2-3'ü geçmemeli
- ✗ Signal/Observable tip belirtmemek — Yanlış tiplendirme problemleri

□ Bu Dersten Çıkarman Gerekenler

- ✓ Generics'in problemi nasıl çözdüğünü anlıyorum
- ✓ syntax'ını mental model olarak kavradım
- ✓ Generic constraint (extends) yazabiliyorum
- ✓ Generic interface ve class yazabiliyorum
- ✓ Multiple type parameters ve default values kullanabiliyorum
- ✓ Angular'daki Signal, Observable, input() gibi yapıları tanıyorum
- ✓ Partial, Pick, Omit gibi utility generic'lerin ne işe yaradığını biliyorum

MODÜL 2 — DERS 2.3

Decorators (Kavramsal)

@Component, @Injectable, @Input... Angular'da decorator'lar her yerde. Ne yapıyorlar? Hadi anlayalım.



Önemli Not

Bu ders kavramsal — decorator'ların nasıl çalıştığını anlayacağız. Implementasyon detayları (decorator factory yazma) ileri seviye konu.

1

Decorator Nedir? Genel Bakış

Bir Angular component'i hatırlayalım:

```
@Component({
  selector: 'app-user-card',
  template: `<h1>{{ name() }}</h1>`
})
export class UserCardComponent {
  name = signal('John');
}
```

Buradaki `@Component({...})` satırı bir decorator.

Mental Model: "Etiket"

Bir analogi: Bir kutuya "Kırılgan! Dikkat!" etiketi yapıştırırsın. Kutuyu değiştirmedin, sadece bilgi eklemiş oldun. Decorator'lar da öyle:

```
@Component({...}) // ← Etiket: "Bu sınıf bir component'tir"
export class UserCardComponent {
  // Sınıfın kendisi
}
```

Daha Teknik Tanım

Decorator: Bir sınıfa, metoda, property'ye veya parametreye metadata eklemek için kullanılan özel bir fonksiyondur.

- ☐ Metadata ekler — Sınıfa "bu Angular component'i" bilgisi gibi
- ☐ Davranış değiştirebilir — Method'u sarmalayıp loglama ekleyebilir
- ☐ Declarative syntax sağlar — Kod yerine bilgi yazıyorsun

2 Neden Var? Hangi Problemi Çözüyor?

Decorator Olmadan Bir Dünya Hayal Et

Diyelim ki Angular decorator'ları olmasaydı:

```
// Hayal: Decorator olmadan
class UserCardComponent {
  name = signal('John');
}

// Sonra Angular'a "bu component'tir" demek için bir şekilde
// metadata sağlamak zorundasın:
ComponentRegistry.register(UserCardComponent, {
  selector: 'app-user-card',
  template: `<h1>{{ name() }}</h1>`
});
```

Bunlar çalışır, ama:

- ✗ Sınıfın dışında ayrı bir kayıt kodu var
- ✗ Bilgi uzakta, sınıfla ilişkisi net değil

Decorator İle

```
@Component({
  selector: 'app-user-card',
  template: `<h1>{{ name() }}</h1>`
})
export class UserCardComponent {
  name = signal('John');
}
```

- ✓ Bilgi sınıfın hemen üstünde — bağlantısı net
- ✓ Declarative — "şu özellikler" diyorsun
- ✓ Okunabilir — bir Angular geliştiricisi 2 saniyede anlar
- ✓ TypeScript-friendly — IDE autocomplete, type checking

3 Decorator Türleri

TypeScript'te 4 tür decorator var:

1. Class Decorator

Sınıfın üzerinde kullanılır:

```
@Component({...})          // ← Class decorator
export class UserCardComponent { }

@Injectable({...})           // ← Class decorator
export class UserService { }

@Directive({...})           // ← Class decorator
export class HighlightDirective { }

@Pipe({...})                // ← Class decorator
export class CapitalizePipe { }
```

2. Property Decorator

Bir property'nin üstünde kullanılır:

```
export class UserCardComponent {
  @Input() user!: User;          // ← Property decorator (eski API)
  @Output() select = new EventEmitter<User>();
  @ViewChild('myEl') el!: ElementRef;
  @HostBinding('class.active') isActive = false;
}
```



Modern Angular

Modern Angular'da (v17+) bu decorator'lar yerine signal-based API'ler önerilir:

@Input() → input()

@Output() → output()

@ViewChild → viewChild()

3. Method Decorator

```
export class UserCardComponent {
  @HostListener('click', ['$event']) // ← Method decorator
  onClick(event: MouseEvent) {
    console.log('clicked');
  }
}
```

4. Parameter Decorator

```
export class UserCardComponent {
  constructor(
    @Inject(LOG_LEVEL) private logLevel: string, // ← Parameter deco
    rator
    @Optional() private logger?: LoggerService
  ) {}
}
```

4 Modern Angular'da Decorator Trendi

Eski Yaklaşım: Decorator Her Yerde

```
@Component({...})
export class UserCardComponent {
  @Input() user!: User;
  @Input() showAvatar = true;
  @Output() selected = new EventEmitter<User>();
  @ViewChild('avatar') avatarEl!: ElementRef;
  @HostBinding('class.active') isActive = false;
  @HostListener('click') onClick() { /* ... */ }
}
```

Modern Yaklaşım: Function-Based API'ler

```
@Component({
  host: {
    '[class.active]': 'isActive()',
    '(click)': 'onClick()'
  }
})
export class UserCardComponent {
  user = input.required<User>(); // @Input() yerine
  showAvatar = input(true); // @Input() yerine
  selected = output<User>(); // @Output() yerine
  avatarEl = viewChild<ElementRef>('avatar'); // @ViewChild yerine

  isActive = signal(false);
  onClick() { /* ... */ }
}
```

Modern Yaklaşımın Avantajları

- ✓ Type-safe by default — `input.required()` tipini bilir, undefined olamaz
- ✓ Signal entegrasyonu — Otomatik reactivity
- ✓ Tree-shakable — Kullanılmayan kod build'e girmez
- ✓ Test etmesi kolay — Function call'lar mock'lanabilir

Hala Geçerli Decorator'lar

Decorator	Modern Alternatif
@Component	✓ Hala kullanılır (alternatifi yok)
@Injectable	✓ Hala kullanılır
@Directive	✓ Hala kullanılır
@Pipe	✓ Hala kullanılır
@Input()	△ input() veya input.required()
@Output()	△ output()
@ViewChild	△ viewChild()
@HostListener	△ host: { '(click)': '...' }
@HostBinding	△ host: { '[class.x]': '...' }
@NgModule	✗ Standalone components ile gereksiz
@Inject	△ inject() function

📝 Mini Quiz

- **Decorator nedir, en basit anlamıyla?**

Cevap: Bir sınıfa, metoda, property'ye veya parametreye metadata eklemek için kullanılan özel fonksiyon. "Etiket" gibi düşün.

- **@Component decorator'ı ne işe yarar?**

Cevap: Bir TypeScript sınıfını Angular component'i olarak işaretler ve metadata'sını (selector, template, styles vs.) Angular'a bildirir.

- **TypeScript'te kaç tür decorator vardır?**

Cevap: 4 tür: Class decorator, Property decorator, Method decorator, Parameter decorator.

- **@Input() yerine modern Angular'da ne kullanılır?**

Cevap: input() veya input.required() function. Signal-based, daha type-safe.

- **Decorator'lar runtime'da mı, build time'da mı işlenir?**

Cevap: Çoğunlukla build time'da. Angular compiler decorator'ları okur, optimize edilmiş runtime kodu üretir.

- **Modern Angular'da @NgModule decorator'ı kullanılır mı?**

Cevap: Hayır. Standalone components ile birlikte NgModule sistemi gereksiz hale geldi. Yeni projelerde kullanılmaz.

⚠️ Anti-Patterns

- ✗ Decorator'a "korkulacak bir şey" gibi bakmak — Sadece metadata
- ✗ @NgModule ile yeni component yazmak — Standalone kullan
- ✗ @Input() decorator'ı yeni projede kullanmak — input() fonksiyonunu tercih et
- ✗ @Inject decorator'ını her yerde kullanmak — inject() function daha modern
- ✗ Custom decorator yazmaya hevesli olmak — Çoğu zaman gereksiz complexity

📝 Bu Dersten Çıkarman Gerekenler

- ✓ Decorator'ın ne olduğunu ve neden var olduğunu anlıyorum
- ✓ "Etiket" mental modeli ile decorator'lara bakıyorum
- ✓ 4 türü (class, property, method, parameter) tanıyorum
- ✓ Angular'daki sık kullanılan decorator'ları biliyorum
- ✓ Modern API'lerin (input(), output(), inject()) decorator alternatifleri olduğunu görüyorum
- ✓ Decorator'ların build time'da işlendiğini biliyorum
- ✓ Artık @ işareti gördüğümde korkmuyorum — sadece metadata

MODÜL 2 — DERS 2.4

Utility Types

TypeScript'in built-in utility type'larını tanımak ve kullanabilmek. *Partial, Pick, Omit, Required, Readonly...* Bu utility'ler senin TypeScript hayatını gerçekten kolaylaştıracak.



Önemli

Bu utility'leri günlük Angular kodunda çok sık kullanacaksın. Form modelleri, API response'ları, immutable update'ler — hepsinde işine yarar.

1

Utility Types Nedir?

Utility Types: TypeScript'in standart kütüphanesinde gelen, var olan tipleri dönüştürmeye yarayan generic type'lar.

Mental Model: Tip Dönüştürücüler

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  age: number;  
}
```

Şimdi farklı senaryolarda farklı tip lazım:

- Update endpoint: Tüm field'lar optional olsun
- Create endpoint: id field'ı olmasın
- Read-only API: Hiçbir field değiştirilemesin
- User preview: Sadece id ve name göstereceğiz

Çözüm: Utility Types

```
type UserUpdate = Partial<User>;  
type UserCreate = Omit<User, 'id'>;  
type UserReadonly = Readonly<User>;  
type UserPreview = Pick<User, 'id' | 'name'>;
```

4 satır. Tek kaynak (User), 4 farklı kullanım. Eğer User'a yeni field eklersen, hepsi otomatik güncellenir.

2 Partial<T> — Hepsi Optional

En sık kullanılan utility type.

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
type PartialUser = Partial<User>;  
// {  
//   id?: number;  
//   name?: string;  
//   email?: string;  
// }
```

Pratik Örnek 1: Update Endpoint

```
@Injectable({ providedIn: 'root' })  
export class UserService {  
  private http = inject(HttpClient);  
  
  // Tüm field'lar optional - kullanıcı sadece değiştirmek istediği field'ı gönderir  
  updateUser(id: number, changes: Partial<User>): Observable<User> {  
    return this.http.patch<User>(`/api/users/${id}`, changes);  
  }  
}  
  
// Kullanım  
this.userService.updateUser(1, { name: 'New Name' }); // ✓  
this.userService.updateUser(1, { email: 'new@example.com' }); // ✓
```

Pratik Örnek 2: Default Config Merge

```
interface AppConfig {
  apiUrl: string;
  timeout: number;
  retryCount: number;
  debug: boolean;
}

const defaultConfig: AppConfig = {
  apiUrl: '/api',
  timeout: 5000,
  retryCount: 3,
  debug: false
};

function createApp(userConfig: Partial<AppConfig> = {}): AppConfig {
  return { ...defaultConfig, ...userConfig };
}

const app1 = createApp(); // Tüm default değerler
const app2 = createApp({ debug: true }); // Sadece debug değışti
```

**Dikkat**

Partial sadece birinci seviye property'leri optional yapar. Nested objelerin içine girmez. Nested için DeepPartial custom utility yazmak gerekiyor.

3 Pick<T, K> — Belirli Property'leri Seç

```
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
  password: string;
}

type UserCard = Pick<User, 'id' | 'name' | 'email'>;
// {
//   id: number;
//   name: string;
//   email: string;
// }
```

Pratik Örnek: Public API

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;          // Hassas
  internalNotes: string;     // Hassas
  createdAt: Date;
}

// API'den dönen, public olarak güvenli versiyon
type PublicUser = Pick<User, 'id' | 'name' | 'email' | 'createdAt'>;
```

4 Omit<T, K> — Belirli Property'leri Çıkar

Pick'nın tersi.

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

type SafeUser = Omit<User, 'password'>;
// {
//   id: number;
//   name: string;
//   email: string;
// }
```

Pratik Örnek: Create Endpoint

Backend ID'yi otomatik üretir, sen göndermek zorunda değilsin:

```
interface User {
  id: number;
  name: string;
  email: string;
  createdAt: Date;
  updatedAt: Date;
}

// "Create" payload – id ve timestamps backend'den
type CreateUserDto = Omit<User, 'id' | 'createdAt' | 'updatedAt'>;
// { name: string; email: string }

// Kullanım
this.userService.createUser({ name: 'John', email: 'j@x.com' }); // ✓
this.userService.createUser({ id: 1, name: 'X' });                // ✗
id giremezsin
```

5 Record<K, V> — Object Map

Key'leri K, value'ları V olan bir object type oluşturur.

```
type Roles = 'admin' | 'user' | 'guest';

type RolePermissions = Record<Roles, string[]>;
// {
//   admin: string[];
//   user: string[];
//   guest: string[];
// }

const permissions: RolePermissions = {
  admin: ['read', 'write', 'delete'],
  user: ['read', 'write'],
  guest: ['read']
};
```

Pratik Örnek: Translation Files

```
type Language = 'tr' | 'en' | 'de' | 'fr';

interface Translations {
  greeting: string;
  farewell: string;
}

const i18n: Record<Language, Translations> = {
  tr: { greeting: 'Merhaba', farewell: 'Hoşçakal' },
  en: { greeting: 'Hello', farewell: 'Goodbye' },
  de: { greeting: 'Hallo', farewell: 'Auf Wiedersehen' },
  fr: { greeting: 'Bonjour', farewell: 'Au revoir' }
};
```

Pratik Örnek: Form Errors

```

type FormErrors<T> = Partial<Record<keyof T, string>>;

interface LoginForm {
  email: string;
  password: string;
}

const errors: FormErrors<LoginForm> = {
  email: 'Invalid email',
  // password: optional
};

```



Record vs Map

Record<K, V> sadece bir tip etiketi — runtime'da normal object. JavaScript'in Map class'ı ise gerçek bir runtime yapısı (set, get, has, delete metodları var).

6

Readonly<T> — Değiştirilemez

```

interface User {
  id: number;
  name: string;
}

type FrozenUser = Readonly<User>;
// {
//   readonly id: number;
//   readonly name: string;
// }

const user: FrozenUser = { id: 1, name: 'John' };
user.name = 'Jane'; // x Error: Cannot assign

```

7 Exclude<T, U> ve Extract<T, U>



Önemli Ayrım

Pick/Omit object type'larla çalışır.
Extract/Exclude union type'larla çalışır.
İsimleri benzer ama farklı şeyler için.

Exclude — Tipi Çıkar

```
type AllStatus = 'pending' | 'active' | 'inactive' | 'banned';

type ActiveStatus = Exclude<AllStatus, 'banned'>;
// 'pending' | 'active' | 'inactive'

type FinalStatus = Exclude<AllStatus, 'pending'>;
// 'active' | 'inactive' | 'banned'
```

Extract — Tipi Al (Tersi)

```
type AllStatus = 'pending' | 'active' | 'inactive' | 'banned';

type LiveStatus = Extract<AllStatus, 'active' | 'pending'>;
// 'active' | 'pending'
```

8 Function Utility Types

Bu utility'ler fonksiyonlar için.

Parameters — Parametreleri Al

```
function createUser(name: string, age: number, email: string) {
  return { name, age, email };
}

type CreateUserParams = Parameters<typeof createUser>;
// [string, number, string]

const params: CreateUserParams = ['John', 30, 'j@x.com'];
createUser(...params); // ✓
```

ReturnType — Return Tipini Al

```
function getUser() {  
  return { id: 1, name: 'John' };  
}  
  
type User = ReturnType<typeof getUser>;  
// { id: number; name: string }
```

Awaited — Promise Çözümü

```
async function fetchUser(): Promise<User> {  
  return /* ... */;  
}  
  
type FetchedUser = Awaited<ReturnType<typeof fetchUser>>;  
// User (Promise<User> değil!)
```


9

Hangi Utility Hangi Senaryoda?

Senaryo	Utility
Update endpoint payload	Partial
Create endpoint (id'siz)	Omit
Read-only data	Readonly
Public/safe version	Omit
Liste view (az field)	Pick
Object map (key→value)	Record
Form errors	Partial>
Status/role union	Exclude
Function return type	ReturnType
Function params	Parameters
Async return	Awaited>

□ Mini Quiz

- **Partial ne yapar?**

Cevap: User interface'inin tüm property'lerini optional (?) yapar. Update endpoint'leri için ideal.

- **Pick ile Omit arasındaki fark nedir?**

Cevap: İkisi de aynı sonucu üretir. Pick: hangileri istediğini söyler. Omit: hangileri istemediğini söyler. Az sayıda field istiyorsan Pick, çoğunluğu istiyorsan Omit kullan.

- **Record<'admin' | 'user', string[]> ne demek?**

Cevap: { admin: string[]; user: string[] } tipinde bir object oluşturur. Key'ler 'admin' ve 'user', value'ları string[].

- **Pick/Omit ile Extract/Exclude aynı işi mi yapar?**

Cevap: HAYIR. Pick/Omit object type'larla, Extract/Exclude union type'larla çalışır. İsimleri benzer ama farklı şeyler için.

- **Partial nested object'lerin içine giriyor mu?**

Cevap: Hayır. Sadece birinci seviye property'leri optional yapar. Nested için custom DeepPartial yazmak gerekir.

- **ReturnType ne işe yarar?**

Cevap: getUser fonksiyonunun return tipini alır. Fonksiyonun return tipi değişirse bağlı olan tipler otomatik güncellenir.

⚠ Anti-Patterns

- ✗ Her senaryo için yeni interface yazmak — Utility types ile DRY kalın
- ✗ Partial ile derin nested update — DeepPartial gerekli
- ✗ any kullanmak çünkü utility hatırlamıyor — Önemlileri ezberle
- ✗ Pick ile çoğu field'ı seçmek — Az field istiyorsan Pick, çok istiyorsan Omit kullan
- ✗ Custom utility'leri inline yazmak — utils/types.ts'de topla, reusable yap

□ Bu Dersten Çıkarman Gerekenler

- ✓ Utility types'ın amacını ve mantığını biliyorum
- ✓ Partial ile update endpoint payload'ları yazabiliyorum
- ✓ Omit ve Pick arasındaki farkı biliyorum
- ✓ Record ile object map'leri tip-safe yapabiliyorum
- ✓ Readonly ile immutable data oluşturabiliyorum
- ✓ Exclude/Extract ile union type'ları manipüle edebiliyorum
- ✓ ReturnType/Parameters ile function tiplerini türetebiliyorum
- ✓ Utility'leri kombinleme'yi anladım — gerçek güç burada

MODÜL 2 — DERS 2.5

Type Guards & Narrowing

TypeScript'in gizli süper gücü: *type narrowing*. *if (typeof x === 'string')* yazdığında TypeScript x'in tipini otomatik daraltıyor — bu nasıl çalışıyor? Custom type guard nasıl yazılır?



Hedef

Bu derste TypeScript'i bir üst seviyeye çıkaracağız. Narrowing, tip güvenliğini hem geliştirir hem de manuel cast yapma ihtiyacını ortadan kaldırır.

1

Narrowing Nedir?

Narrowing: TypeScript'in bir tipi, kontrol akışına göre daraltması.

Basit Örnek

```
function processValue(value: string | number) {  
  // value: string | number ← şu an her ikisi de olabilir  
  
  if (typeof value === 'string') {  
    // value: string ← TypeScript otomatik daraltıldı!  
    console.log(value.toUpperCase());  
  } else {  
    // value: number ← burada da doğru daralttı  
    console.log(value.toFixed(2));  
  }  
}
```

TypeScript şunu anladı:

- if bloğunda → value kesinlikle string
- else bloğunda → value kesinlikle number

Bu narrowing. Süper güçlü, çünkü manuel cast yapmak zorunda değilsin.

2

Built-in Type Guards

TypeScript birkaç built-in type guard'ı tanır:

1. typeof Operator

Primitive'ler için:

```
function format(value: string | number | boolean) {
  if (typeof value === 'string') {
    return value.toUpperCase();          // string
  }
  if (typeof value === 'number') {
    return value.toFixed(2);            // number
  }
  return value ? 'Yes' : 'No';          // boolean (kalan)
}
```

typeof döndürebileceği değerler:

```
typeof x === 'string'
typeof x === 'number'
typeof x === 'boolean'
typeof x === 'undefined'
typeof x === 'object'      // null, array, object
typeof x === 'function'
typeof x === 'symbol'
typeof x === 'bigint'
```



Dikkat

typeof null === 'object' — JavaScript'in tarihi bug'ı. null kontrolü için ayrı kontrol gerekir.

2. instanceof Operator

Class'lar için:

```
class HttpError extends Error {
  constructor(public status: number, message: string) {
    super(message);
  }
}

function handleError(error: Error | HttpError) {
  if (error instanceof HttpError) {
    // error: HttpError
    console.log(`HTTP ${error.status}: ${error.message}`);
  } else {
    // error: Error
    console.log(`Generic error: ${error.message}`);
  }
}
```

Angular'da çok kullanılır:

```
catchError((error: unknown) => {
  if (error instanceof HttpResponse) {
    // error: HttpResponse – status, headers vs. var
    return of({ error: `Status ${error.status}` });
  }
  return of({ error: 'Unknown' });
});
```

3. in Operator

Bir property'nin object'te var olup olmadığını kontrol eder:

```
interface Cat { meow(): void; }
interface Dog { bark(): void; }

function speak(animal: Cat | Dog) {
  if ('meow' in animal) {
    animal.meow();    // animal: Cat
  } else {
    animal.bark();    // animal: Dog
  }
}
```

4. Equality Narrowing

```
function example(x: string | null) {
  if (x !== null) {
    // x: string ← null çıkarıldı
    console.log(x.length);
  }
}
```

5. Truthy/Falsy Narrowing

```
function getValue(value: string | undefined) {
  if (value) {
    // value: string ← undefined falsy, çıkarıldı
    console.log(value.toUpperCase());
  }
}
```



Dikkat

Boş string "" da falsy! if (value) her zaman doğru cevap olmayabilir. Daha kesin için value !== undefined.

3 Discriminated Union Narrowing

Modül 2.1'de gördüğümüz discriminated union pattern'i hatırlıyor musun? Narrowing ile harika çalışır:

```
type ApiResponse<T> =  
  | { status: 'success'; data: T }  
  | { status: 'error'; error: string }  
  | { status: 'loading' };  
  
function handleResponse<T>(response: ApiResponse<T>) {  
  // status field'ı discriminator (ayırt edici)  
  
  switch (response.status) {  
    case 'success':  
      // response: { status: 'success'; data: T }  
      console.log(response.data);          // ✓ data var  
      break;  
  
    case 'error':  
      // response: { status: 'error'; error: string }  
      console.log(response.error);          // ✓ error var  
      break;  
  
    case 'loading':  
      // response: { status: 'loading' }  
      console.log('Loading...');  
      break;  
  }  
}
```

TypeScript, status değerine bakarak hangi property'lerin var olduğunu otomatik biliyor. Süper güçlü!

Angular'da Pratik Örnek

```
type RemoteData<T> =
  | { state: 'idle' }
  | { state: 'loading' }
  | { state: 'success'; data: T }
  | { state: 'error'; error: string };

@Component({
  template: `
    @switch (users().state) {
      @case ('idle') { <p>Veri yok</p> }
      @case ('loading') { <p>Yükleniyor...</p> }
      @case ('success') {
        <ul>
          @for (user of users().data; track user.id) {
            <li>{{ user.name }}</li>
          }
        </ul>
      }
      @case ('error') { <p>Hata: {{ users().error }}</p> }
    }
  `,
})
export class UserListComponent {
  users = signal<RemoteData<User[]>>({ state: 'idle' });
}
```

4

Custom Type Guards (Type Predicates)

Bazen built-in guard'lar yeterli olmaz. Kendi guard'ını yazarsın.

Problem

```
interface User {
  id: number;
  name: string;
  email: string;
}

function isUser(obj: unknown): boolean {
  return (
    typeof obj === 'object' &&
    obj !== null &&
    'id' in obj &&
    'name' in obj &&
    'email' in obj
  );
}

function processData(data: unknown) {
  if (isUser(data)) {
    console.log(data.name); // x Error: data hala unknown
  }
}
```

isUser doğru kontrol etti, ama TypeScript hala data'nın User olduğunu bilmiyor. Çünkü isUser boolean döndürüyor — anlam yok.

Çözüm: Type Predicate (is keyword)


```
function isUser(obj: unknown): obj is User {
  //           ^^^^^^^^^^^
  //           Bu bir type predicate
  return (
    typeof obj === 'object' &&
    obj !== null &&
    'id' in obj &&
    'name' in obj &&
    'email' in obj
  );
}

function processData(data: unknown) {
  if (isUser(data)) {
    // data: User ← TypeScript artık narrowing yapıyor
    console.log(data.name); // ✓ OK
  }
}
```

obj is User şunu söylüyor: "Bu fonksiyon true dönerse, parametre kesinlikle User'dır."

Daha Sağlam Bir User Type Guard

```
function isUser(obj: unknown): obj is User {
  if (typeof obj !== 'object' || obj === null) return false;

  const candidate = obj as Record<string, unknown>;

  return (
    typeof candidate.id === 'number' &&
    typeof candidate.name === 'string' &&
    typeof candidate.email === 'string'
  );
}
```

Bu, sadece property'lerin var olduğunu değil, doğru tipte olduğunu da kontrol eder.

Pratik Örnek: Array Filtering

Type guard, .filter() ile inanılmaz güzel çalışır:

```
function isString(value: unknown): value is string {
  return typeof value === 'string';
}

const mixed: (string | number | undefined)[] = ['a', 1, 'b', undefined, 'c'];

const onlyStrings: string[] = mixed.filter(isString);
//      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//      string[], number ve undefined çıkarıldı!
```

5

NonNullable ile Pratik Narrowing

null/undefined çıkarma için klasik pattern:

```
function isDefined<T>(value: T | null | undefined): value is T {
  return value !== null && value !== undefined;
}

const items = ['a', null, 'b', undefined, 'c'];
const filtered = items.filter(isDefined);
//      filtered: string[] ← null ve undefined çıktı!
```

Angular'da:

```
@Component({...})
export class UserListComponent {
  users = toSignal(
    this.userService getUsers().pipe(
      filter(isDefined) // null response'ları yok say
    )
  );
}
```

6 Assertion Functions

Type predicate'in yakın akrabası: assertion function.

Type Predicate vs Assertion

```
// Type Predicate – boolean döner
function isUser(obj: unknown): obj is User {
  return /* boolean */;
}

if (isUser(data)) {
  data.name; // Narrowed
}

// Assertion Function – hata fırlatır
function assertIsUser(obj: unknown): asserts obj is User {
  if (!isUser(obj)) {
    throw new Error('Not a User');
  }
}

assertIsUser(data);
data.name; // Narrowed (assertion'dan sonra)
```

Pattern	Ne Zaman Kullanılır?
Type Predicate (is)	Conditional logic (if/else, .filter)
Assertion (asserts)	"Olması zorunlu" durumlar, validation

Pratik Örnek

```
function assertNonNull<T>(  
  value: T | null | undefined,  
  message = 'Value is null or undefined'  
) : asserts value is T {  
  if (value === null || value === undefined) {  
    throw new Error(message);  
  }  
}  
  
@Component({...})  
export class UserDetailsComponent {  
  @Input() user: User | null = null;  
  
  greet() {  
    assertNonNull(this.user, 'User must be set before greet()');  
    console.log(this.user.name); // ✓ Narrowed to User  
  }  
}
```

7 Exhaustiveness Checking (Eksiksizlik Kontrolü)

Discriminated union'larda tüm durumları işlediğinden emin olmak istersin. TypeScript bunu derleyici düzeyinde yapabilir.

Çözüm: never ile Exhaustive Check

```
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'square'; size: number }
  | { kind: 'triangle'; base: number; height: number };

function assertNever(value: never): never {
  throw new Error(`Unhandled case: ${JSON.stringify(value)}`);
}

function area(shape: Shape): number {
  switch (shape.kind) {
    case 'circle':
      return Math.PI * shape.radius ** 2;
    case 'square':
      return shape.size ** 2;
    case 'triangle':
      return 0.5 * shape.base * shape.height;
    default:
      return assertNever(shape);
  }
  //          ^^^^^
  //          shape: never
  //          Tüm case'ler ele alınınca buraya gelinmez
}
```

Eğer Shape union'ına yeni bir tip eklersen (örn. {kind: 'rectangle'}), TypeScript compile-time'da sana hatırlatır: "Bu tipi handle etmedin!"

Bu pattern Angular'da sıkça kullanılır

Form state, API response, application state gibi yerlerde tüm case'leri işlediğinden emin olursun. Yeni state ekledikçe TypeScript seni uyarır.

8 Narrowing'in Sınırları

Narrowing güçlü, ama bazı durumlarda çalışmaz:

Sınır 1: Async Boundaries

```
@Component({...})
export class UserComponent {
  user: User | null = null;

  async loadAndGreet() {
    if (this.user) {
      // user: User
      await someAsyncOperation();
      // user: User | null ← Async sonrası narrowing kaybolur!
      console.log(this.user.name); // ✗ Possibly null
    }
  }
}
```

Çözüm: Local variable kullan:

```
async loadAndGreet() {
  const user = this.user; // local'e al
  if (user) {
    await someAsyncOperation();
    console.log(user.name); // ✓ Narrowed
  }
}
```

Sınır 2: Functions

```
function example(value: string | null) {
  if (value !== null) {
    callback();
    // value: string | null ← function call sonrası narrowing kaybolab
ilir
  }
}
```

TypeScript "callback value'yu değiştirmiş olabilir" diye düşünür.



Pratik İpucu

Narrowing'i kaybetmemek için immutable local variable'lara kopyala.

9

Angular'da Real-World Type Guards

Senaryo 1: HTTP Error Handling

```
function isHttpError(error: unknown): error is HttpErrorResponse {
  return error instanceof HttpErrorResponse;
}

function isClientError(error: HttpErrorResponse): boolean {
  return error.status >= 400 && error.status < 500;
}

function isServerError(error: HttpErrorResponse): boolean {
  return error.status >= 500;
}

// Kullanım
this.userService getUsers().pipe(
  catchError((error: unknown) => {
    if (isHttpError(error)) {
      if (isClientError(error)) return of({ message: 'Bad request' });
      if (isServerError(error)) return of({ message: 'Server down' });
    }
    return of({ message: 'Unknown error' });
  })
).subscribe();
```

Senaryo 2: Discriminated State Pattern

```
type AsyncState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };

// Type guards
function isLoading<T>(state: AsyncState<T>): state is { status: 'loading' } {
  return state.status === 'loading';
}

function isSuccess<T>(state: AsyncState<T>): state is { status: 'success'; data: T } {
  return state.status === 'success';
}

@Component({
  template: `
    @if (isLoading(state())) {
      <app-spinner />
    }
    @if (isSuccess(state())) {
      <app-data-view [data]="state().data" />
    }
  `,
})
export class DataComponent {
  state = signal<AsyncState<User[]>>({ status: 'idle' });
  isLoading = isLoading;
  isSuccess = isSuccess;
}
```


□ Mini Quiz

- **Type narrowing nedir?**

Cevap: TypeScript'in kontrol akışına göre bir tipi otomatik olarak daha spesifik bir tipe daraltması.

- **typeof ve instanceof arasındaki fark nedir?**

Cevap: typeof primitive'ler (string, number, boolean) için. instanceof class instance'ları için.

- **obj is User syntax'ı ne işe yarar?**

Cevap: Type predicate. Bir fonksiyonun true dönerse parametrenin kesinlikle belirli bir tipte olduğunu TypeScript'e söyler.

- **Discriminated union narrowing nasıl çalışır?**

Cevap: Bir union'da tüm üyelerin ortak bir property'si (kind, type, status gibi) varsa, TypeScript o property'nin değerine bakarak hangi üye olduğunu otomatik anlar.

- **asserts x is User ne demek?**

Cevap: Assertion function. Fonksiyon hatasız tamamlanırsa, x'in User olduğu garanti.

- **assertNever pattern'i ne işe yarar?**

Cevap: Discriminated union'larda exhaustiveness check için. Tüm case'leri işlediğinden emin olur.

⚠ Anti-Patterns

- ✗ as ile narrowing yapmak — Type guard yaz, as runtime check değil
- ✗ Type guard'sız .filter() — Sonuç tipi hala union, narrowing kaybedilmiş
- ✗ if (value) ile string narrowing — Boş string "" falsy! value !== undefined kullan
- ✗ Async sonrası narrowing'in geçerli olduğunu varsaymak — Local variable'a kopyala
- ✗ assertNever kullanmadan exhaustive switch — Yeni case eklenince fark etmezsin

□ Bu Dersten Çıkarman Gerekenler

- ✓ Narrowing'in ne olduğunu ve nasıl çalıştığını anlıyorum
- ✓ typeof, instanceof, in built-in guard'larını kullanabiliyorum
- ✓ Discriminated union narrowing pattern'ini biliyorum
- ✓ Custom type guard (obj is User) yazabiliyorum
- ✓ Assertion function (asserts x is User) ile assertion guard yapabiliyorum
- ✓ assertNever ile exhaustive checking yapabiliyorum
- ✓ Narrowing sınırlarını (async, function, mutation) biliyorum

MODÜL 2 — DERS 2.6

Mini Proje: TypeScript Bootcamp

Modül 2'nin son dersi. Öğrendiğimiz tüm TypeScript konseptlerini birleştirip mini bir uygulama yazacağız: *Type-safe Mini ToDo Manager*. Henüz Angular yok — sadece TypeScript.



Hedef

Bu mini projede Modül 2'de öğrendiğimiz tüm kavramları kullanacağız: tipler, interfaces, generics, decorators (kavramsal), utility types, type guards. Hepsi bir araya geldiğinde gerçek bir uygulamanın temelini nasıl oluşturduğunu göreceksin.

1

Proje Tanımı

Yapacağımız: Type-safe ToDo Manager.

Özellikler

- ✓ Todo ekleme, güncelleme, silme
- ✓ Filtreleme (active, completed, all)
- ✓ Sıralama (date, priority)
- ✓ LocalStorage persist
- ✓ Type-safe API (input/output validation)
- ✓ Generic store pattern

NOT: Bu proje saf TypeScript. Angular yok. Modül 3-4'te aynı projeyi Angular ile baştan yazacağız ve karşılaştıracacağız.

2

Domain Modelleri

Önce data shape'lerini tanımlayalım. Modül 2.1'deki interface ve readonly:

```
// types/todo.ts

export type Priority = 'low' | 'medium' | 'high';
export type FilterMode = 'all' | 'active' | 'completed';

export interface Todo {
  readonly id: string;
  title: string;
  description?: string;
  completed: boolean;
  priority: Priority;
  readonly createdAt: Date;
  updatedAt: Date;
}

// Modül 2.4'teki utility types – Create endpoint için id ve timestamps
// yok
export type CreateTodoInput = Omit<Todo, 'id' | 'createdAt' | 'updatedAt'>;

// Update için sadece bazı alanlar (Partial)
export type UpdateTodoInput = Partial<Omit<Todo, 'id' | 'createdAt'>>;

// Listeleme/filtreleme için
export interface TodoFilters {
  mode: FilterMode;
  priority?: Priority;
  searchQuery?: string;
}
```

3 Generic Store Pattern

Modül 2.2'de gördüğümüz generics ile reusable bir store class yazalım:

```
// store/generic-store.ts

interface Identifiable {
  id: string;
}

export class Store<T extends Identifiable> {
  private items: T[] = [];
  private listeners: Set<() => void> = new Set();

  // Subscribe pattern (publish-subscribe)
  subscribe(listener: () => void): () => void {
    this.listeners.add(listener);
    return () => this.listeners.delete(listener);
  }

  private notify() {
    this.listeners.forEach(l => l());
  }

  getAll(): readonly T[] {
    return this.items; // Immutable view
  }

  getById(id: string): T | undefined {
    return this.items.find(item => item.id === id);
  }

  add(item: T): void {
    this.items = [...this.items, item];
    this.notify();
  }

  update(id: string, changes: Partial<T>): T | null {
    const index = this.items.findIndex(item => item.id === id);
    if (index === -1) return null;

    this.items = [
      ...this.items.slice(0, index),
      { ...this.items[index], ...changes },
      ...this.items.slice(index + 1)
    ];
    this.notify();
    return this.items[index];
  }

  delete(id: string): boolean {
```

```
const initialLength = this.items.length;
this.items = this.items.filter(item => item.id !== id);
const deleted = this.items.length < initialLength;
if (deleted) this.notify();
return deleted;
}
}
```

Dikkat ettin mi? `T extends Identifiable` constraint ile her item'ın id'si olduğunu garanti ettik (Modül 2.2). `Partial<T>` ile update'te sadece değişen alanları kabul ediyoruz (Modül 2.4).

4 **TodoService — Specific Logic**

Generic store'u extend ederek Todo'ya özel logic ekleyelim:

```
// services/todo.service.ts

import { Store } from '../store/generic-store';
import {
  Todo, CreateTodoInput, UpdateTodoInput,
  TodoFilters, FilterMode
} from '../types/todo';

export class TodoService extends Store<Todo> {
  private static readonly STORAGE_KEY = 'todos';

  constructor() {
    super();
    this.loadFromStorage();
  }

  // Domain-specific create – UUID üretir, timestamp ekler
  create(input: CreateTodoInput): Todo {
    const todo: Todo = {
      id: crypto.randomUUID(),
      ...input,
      createdAt: new Date(),
      updatedAt: new Date(),
    };
    this.add(todo);
    this.saveToStorage();
    return todo;
  }

  // Type-safe update – id ve createdAt değiştirilemez
  updateTodo(id: string, changes: UpdateTodoInput): Todo | null {
    const result = this.update(id, {
      ...changes,
      updatedAt: new Date(),
    });
    if (result) this.saveToStorage();
    return result;
  }

  toggleCompleted(id: string): Todo | null {
    const todo = this.getById(id);
    if (!todo) return null;
    return this.updateTodo(id, { completed: !todo.completed });
  }

  removeTodo(id: string): boolean {
    const result = this.delete(id);
```



```

    if (result) this.saveToStorage();
    return result;
  }

  // Filtering with type guards
  filter(filters: TodoFilters): Todo[] {
    return this.getAll().filter(todo => {
      // Filter by mode
      if (filters.mode === 'active' && todo.completed) return false;
      if (filters.mode === 'completed' && !todo.completed) return false
    ;

      // Filter by priority
      if (filters.priority && todo.priority !== filters.priority) return false;

      // Filter by search query
      if (filters.searchQuery) {
        const query = filters.searchQuery.toLowerCase();
        const matchesTitle = todo.title.toLowerCase().includes(query);
        const matchesDesc = todo.description?.toLowerCase().includes(query) ?? false;
        if (!matchesTitle && !matchesDesc) return false;
      }

      return true;
    });
  }

  // LocalStorage persistence
  private saveToStorage() {
    localStorage.setItem(
      TodoService.STORAGE_KEY,
      JSON.stringify(this.getAll())
    );
  }

  private loadFromStorage() {
    const data = localStorage.getItem(TodoService.STORAGE_KEY);
    if (!data) return;

    try {
      const parsed: unknown = JSON.parse(data);
      if (Array.isArray(parsed)) {
        // Type guard ile validation (Modül 2.5)
        const validTodos = parsed
          .filter(isTodo)
          .map(todo => ({

```

```
        ...todo,  
        createdAt: new Date(todo.createdAt),  
        updatedAt: new Date(todo.updatedAt),  
    }));  
    validTodos.forEach(t => this.add(t));  
  }  
  } catch (e) {  
    console.error('Failed to load todos:', e);  
  }  
}  
  
// Type guard (Modül 2.5)  
function isTodo(obj: unknown): obj is Todo {  
  if (typeof obj !== 'object' || obj === null) return false;  
  const o = obj as Record<string, unknown>;  
  
  return (  
    typeof o.id === 'string' &&  
    typeof o.title === 'string' &&  
    typeof o.completed === 'boolean' &&  
    (o.priority === 'low' || o.priority === 'medium' || o.priority ===  
    'high')  
  );  
}
```

5

Kullanım Örnekleri

```
// main.ts

const todos = new TodoService();

// Subscribe to changes
const unsubscribe = todos.subscribe(() => {
  console.log('Todos changed:', todos.getAll().length);
});

// Create todos – type-safe (Omit<Todo, 'id' | 'createdAt' | 'updatedAt'>)
const todo1 = todos.create({
  title: 'TypeScript öğren',
  description: 'Modül 2 bitti',
  completed: false,
  priority: 'high',
});

const todo2 = todos.create({
  title: 'Angular öğren',
  completed: false,
  priority: 'medium',
});

// Toggle
todos.toggleCompleted(todo1.id);

// Update – type-safe (Partial<Omit<Todo, 'id' | 'createdAt'>>)
todos.updateTodo(todo2.id, {
  description: 'Modül 3\'e başlayacağım',
  priority: 'high',
});

// Filter – type-safe
const activeTodos = todos.filter({ mode: 'active' });
const highPriority = todos.filter({ mode: 'all', priority: 'high' });
const search = todos.filter({ mode: 'all', searchQuery: 'angular' });

// TypeScript yanlış kullanımlara izin vermez:
// todos.create({ title: 'X' }); // x Error: completed eksik
// todos.create({ title: 'X', priority: 'urgent' }); // x Error: 'urgent' Priority'de yok
// todos.updateTodo(id, { id: 'newId' }); // x Error: id readonly
// todos.updateTodo(id, { completed: 'yes' }); // x Error: boolean olmalı

// Cleanup
```

```
unsubscribe();
```

6

Bu Mini Projede Ne Öğrendik?

Modül 2'nin tüm konseptlerini birleştirdik:

Konsept	Nerede Kullandık
Interfaces (M2.1)	Todo, TodoFilters domain modelleri
readonly modifier (M2.1)	id, createdAt değiştirilemez
Optional ? (M2.1)	description?, priority?, searchQuery?
Union types (M2.1)	Priority, FilterMode
Generics (M2.2)	Store
extends constraint (M2.2)	Item'in id'si olmasını zorunlu kıldı
Partial (M2.4)	update method'unda — sadece değişen alanlar
Omit (M2.4)	CreateTodoInput — id ve timestamp yok
Type guards (M2.5)	isTodo() — localStorage'dan veri validation
readonly arrays (M2.4)	getAll() — readonly T[] döndürür

7 Kendi Pratiğin (Egzersizler)

Bu projeyi kendi başına geliştirmeyi denemeni öneriyorum:

Egzersiz 1: Sıralama Ekle

```
type SortBy = 'createdAt' | 'priority' | 'title';
type SortOrder = 'asc' | 'desc';

// TodoService'e ekle:
sort(by: SortBy, order: SortOrder): Todo[]
```

Egzersiz 2: Toplu İşlemler

```
// Hepsini tamamlandı yap
markAllCompleted(): void

// Tamamlananları sil
clearCompleted(): number // Silinen sayısı

// Birden fazla id'yi sil
deleteMany(ids: string[]): number
```

Egzersiz 3: Statistics

```
interface TodoStats {
  total: number;
  active: number;
  completed: number;
  byPriority: Record<Priority, number>; // Modül 2.4 Record!
}

// TodoService'e ekle:
getStats(): TodoStats
```

Egzersiz 4: Tag/Etiket Sistemi

```
// Todo interface'ine ekle:
tags: readonly string[];

// TodoService'e ekle:
addTag(id: string, tag: string): Todo | null
removeTag(id: string, tag: string): Todo | null
getByTag(tag: string): Todo[]
```

Egzersiz 5: Undo/Redo

```
// History stack ile undo/redo
class TodoServiceWithHistory extends TodoService {
  undo(): boolean
  redo(): boolean
  canUndo(): boolean
  canRedo(): boolean
}
```

Modül 2 Tamamlandı!

Tebrikler! Modül 2'yi başarıyla tamamladın. TypeScript'i Angular bağlamında refresh ettik. Artık temellerin sağlam.

Neler Öğrendin?

Ders 2.1 — Temel Tipler & Interfaces

Temel tipleri, interface'leri, type vs interface farkını, union/intersection ve discriminated union'ları gözden geçirdik.

Ders 2.2 — Generics

Generics'i derinlemesine anladık. T harfinin ne demek olduğunu, constraint'leri, Angular'daki Signal, Observable, input() gibi yapıları kavradık.

Ders 2.3 — Decorators

Decorator'ların ne olduğunu, neden var olduğunu, modern Angular'daki function-based API'lere geçiş trendini öğrendik.

Ders 2.4 — Utility Types

Utility types'ı (Partial, Pick, Omit, Record, Readonly, Exclude/Extract) ve günlük Angular kodunda nasıl kullanılacağını gördük.

Ders 2.5 — Type Guards & Narrowing

TypeScript'in narrowing mekanizmasını, custom type guard'ları (is keyword), assertion function'ları (asserts) ve exhaustiveness checking'i öğrendik.

Ders 2.6 — Mini Proje

Tüm öğrendiklerimizi birleştirip type-safe bir ToDo Manager yazdık. Generic store, type guards, utility types — hepsi tek projede.



Sıradaki Modül: İlk Angular Uygulaması, CLI & DevTools

Modül 3 — Artık gerçek Angular yazma vakti! ng new ile proje oluşturma, proje yapısı anatomisi, CLI komutları, app.config.ts, Angular DevTools, MCP Server kurulumu ve ilk Counter mini projemiz. Heyecanlı!

Modül 3'e Hoş Geldin

Bu modül artık gerçek Angular yazma vakti! Modül 1'de kavramları öğrendik, Modül 2'de TypeScript'i refresh ettik. Şimdi asıl olaya geliyoruz: `ng new` ile proje oluşturma, proje yapısı, CLI komutları, `app.config.ts`, Angular DevTools, ve ilk gerçek mini projemiz olan Counter App.

Bu modülün sonunda:

- ✓ Yeni Angular projesi oluşturup tüm yapıyı tanıyacaksın
- ✓ Angular CLI komutlarını profesyonelce kullanabileceksin
- ✓ `app.config.ts` ile bootstrap akışını kavrayacaksın
- ✓ Angular DevTools ile debug ve profiling yapabileceksin
- ✓ MCP Server ile AI destekli Angular geliştirme yapabileceksin
- ✓ İlk gerçek Angular projeni (Counter App) tamamlamış olacaksın



Heyecan Verici Modül

Bu modülde kod yazıyoruz! Önceki modüller kavramsaldı, bu modül pratik. Modül 1.5'te dokunduğumuz konuların hepsini şimdi derinlemesine işleyeceğiz, ve sonunda kendi Counter App'imizi yapacağız.

MODÜL 3 — DERS 3.1

ng new ile Proje Oluşturma

Angular yolculuğumuza `ng new` ile resmen başlıyoruz. Ama bu sefer "ne soruyor?" değil, "neden soruyor, ne anlama geliyor?" üzerinde duracağız.



Hedef

Modül 1'de hızlıca bir proje açtık. Şimdi her ayar seçeneğinin ne anlama geldiğini, `angular.json` ve `package.json`'da neler olduğunu derinlemesine inceleyeceğiz.

1

ng new Komutu — Anatomy

Temel kullanım:

```
ng new <proje-adi>
```

Tüm parametreleri ile:

```
ng new my-app \
  --style=scss \
  --routing=true \
  --ssr=false \
  --strict=true \
  --skip-tests=false \
  --skip-git=false \
  --package-manager=npm \
  --inline-style=false \
  --inline-template=false \
  --view-encapsulation=Emulated \
  --skip-install=false
```

CLI çoğu zaman bu parametreleri sana etkileşimli olarak sorar. Hepsini flag olarak yazmak zorunda değilsin.

2

CLI'nin Sorduğu Sorular

1. Stylesheet Format

? Which stylesheet format would you like to use?

- CSS
- › SCSS
- Sass
- Less

Format	Avantaj	Dezavantaj
CSS	Tarayıcının doğal dili, derleme yok	Variables/nesting yok
SCSS	Variables, nesting, mixins, modüler	Build adımı gerekir
Sass	SCSS gibi ama indented syntax	Daha az yaygın, indent zorlayıcı
Less	CSS'e benzer ama daha az özellik	Topluluk küçüldü



Tavsiye: SCSS

Angular ekosisteminde standart. Material, CDK, çoğu UI library SCSS kullanıyor. Ayrıca öğrenmesi kolay — sadece CSS + nesting + variables.

2. Server-Side Rendering (SSR)

? Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)?

- Yes
- › No

SSR: Sayfa sunucuda render edilir, kullanıcıya hazır HTML gelir. SEO ve ilk paint hızı için önemli.

Senaryo	Tercih
Public web sitesi, blog, e-ticaret	SSR: Yes (SEO için)
Internal dashboard, admin panel	SSR: No (login sonrası içerik)
Mobile-like app	SSR: No (gereksiz karmaşıklık)
Henüz emin değilsen	SSR: No (sonradan ekleyebilirsiniz)

**Bu Eğitimde**

No diyoruz. SSR'ı Modül 25'te detaylı işleyeceğiz. Şimdilik gereksiz karmaşıklık.

3. Zoneless Change Detection

```
? Do you want to enable Zoneless Change Detection?  
> Yes (recommended)  
No
```

Modül 1.4'te detaylı işledik: Zoneless, Zone.js'siz çalışan modern reactivity sistemi. Daha hızlı, daha küçük bundle.

**Mutlaka Yes**

v21'in default'u ve gelecekteki standart bu. Yeni projede mutlaka Yes seç.

4. Tailwind CSS

```
? Do you want to use Tailwind CSS?  
Yes  
> No
```

Modül 1.4'te konuştuk: utility-first CSS framework. Çok hızlı styling sağlar ama HTML kalabalıklaşır.

**Bu Eğitimde**

No diyoruz. İlk başta vanilla SCSS ile temelleri pekiştireceğiz. Material'ı Modül 13'te işleyeceğiz. Tailwind'e ihtiyacımız olursa sonradan ekleriz.

5. Analytics

```
? Do you want to share pseudonymous usage data  
with the Angular Team?  
Yes  
> No
```

Bu kişisel tercih. Anonim kullanım verisi Angular ekibine gider — hangi feature'ların kullanıldığını anlamak için. Privacy odaklıysan No.

3 ng new Sonrası Ne Olur?

Komutu çalıştırdıktan sonra şu adımlar otomatik gerçekleşir:

```
✓ Authentication for the Angular CLI
✓ Resolved dependencies
✓ Created new Angular project
✓ Installed packages
✓ Successfully initialized git.

Application bundle generation complete. [3.456 seconds]

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → press h + enter to show help
```

Adımları:

- □ Klasör yapısı oluşturuldu
- □ package.json + bağımlılıklar yüklendi (npm install)
- □ angular.json yapılandırma dosyası oluşturuldu
- □ tsconfig.json + tsconfig.app.json + tsconfig.spec.json
- □ styles.scss, app.scss boş template'ler
- ✱ src/app/ içine app component'i + routing yapılandırıldı
- □ Git repository başlatıldı (.git, .gitignore)
- □ Vitest test setup'ı (v21 yeniliği)

4 Yeni Projeye İlk Bakış

Klasöre girip dev server'ı başlatalım:

```
cd my-app
ng serve --open
```

--open flag'i tarayıcıyı otomatik açar. <http://localhost:4200>.

Tarayıcıda göreceğin sayfa:

- Angular logo
- "my-app app is running!" yazısı
- Documentation, CLI, Angular linkleri

**Tebrikler!**

İlk Angular uygulaması ayakta. Hot reload aktif, kod değişikliklerin tarayıcıya anında yansır. Modül 1.5'te bunu zaten denedik. Şimdi ne içerdiğini detaylı inceleyeceğiz.

5 Yaygın Hatalar ve Çözümleri

Hata 1: Node.js sürümü uyumsuzluğu

```
× Node.js version v18.x.x detected.  
  Angular requires Node.js version v20.0 or later.
```

Çözüm: nvm ile güncelle:

```
nvm install 20  
nvm use 20
```

Hata 2: Permission denied (npm)

```
× EACCES: permission denied, mkdir '/usr/lib/node_modules/...'
```

Çözüm: Sudo ile değil, npm prefix'ini değiştir:

```
mkdir ~/.npm-global  
npm config set prefix '~/.npm-global'  
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc  
source ~/.bashrc
```

Hata 3: Corporate proxy / SSL

```
× UNABLE_TO_VERIFY_LEAF_SIGNATURE  
× self signed certificate in certificate chain
```

Çözüm: Şirket sertifikasını NODE_EXTRA_CA_CERTS ile ekle:

```
export NODE_EXTRA_CA_CERTS=/path/to/cert.pem
```

Hata 4: Port 4200 already in use

```
× Port 4200 is already in use. Use '--port' to specify a different port  
.
```

Çözüm:

```
ng serve --port 4300
```


📝 Mini Quiz

- **SSR vs SPA tercihinde hangi senaryoda SSR seçilir?**

Cevap: Public web sitesi, blog, e-ticaret gibi SEO'nun önemli olduğu durumlarda. Internal dashboard'larda SSR'a gerek yok.

- **ng new komutunda Zoneless Change Detection sorusuna ne cevap verilir?**

Cevap: Yes — v21'in default'u ve gelecekteki standart bu. Modern Angular için zoneless tercih edilir.

- **Stylesheet format olarak neden SCSS tercih edilir?**

Cevap: Variables, nesting, mixins özellikleri. Angular Material ve çoğu UI library SCSS kullanıyor. Topluluk standardı.

- **ng new sonrasında Git repository otomatik başlatılır mı?**

Cevap: Evet. --skip-git flag'i kullanılmadıkça otomatik git init yapılır.

- **Tailwind CSS ng new'da ne zaman seçilir?**

Cevap: UI library kullanmayacaksan ve hızlı styling istiyorsan. Ama bu eğitimde Material'a Modül 13'te geçeceğiz.

⚠️ Anti-Patterns

- ✗ Sudo ile npm install — Permission problemleri. nvm veya npm prefix kullan
- ✗ Eski Node.js sürümü ile çalışmak — v20+ gerekli, hata verir
- ✗ SSR'ı her projede aktif etmek — Internal dashboard'larda gereksiz karmaşıklık
- ✗ Zoneless'ı No seçmek (yeni projede) — Eski paradigma, performance kaybı
- ✗ --skip-install ile kullanmadığını fark etmemek — Sonra npm install demeyi unutmak

📝 Bu Dersten Çıkarman Gerekenler

- ✓ ng new komutunun tüm parametrelerini biliyorum
- ✓ CLI'nin sorduğu her sorunun ne anlama geldiğini anlıyorum
- ✓ SSR, Zoneless, Tailwind seçimlerini bilinçli yapabiliyorum
- ✓ ng new sonrası ne olduğunu biliyorum (klasörler, paketler, git)
- ✓ Yaygın hataları (Node sürümü, permission, port) çözebiliyorum
- ✓ İlk uygulamamı çalıştırıp tarayıcıda görebiliyorum

MODÜL 3 — DERS 3.2

Proje Yapısı Anatomisi

Yeni Angular projesinde her dosya ve klasörün ne işe yaradığını öğreneceğiz. Bu, kod yazarken nereye ne yazacağını bilmek için kritik.



Hedef

Aha! dedirteceğiz: "Demek bu dosya bunun için varmış!" Her dosyanın amacını, içeriğini ve ne zaman dokunulacağını göreceğiz.

1

Klasör Yapısı — Genel Bakış

Yeni bir Angular projesi şu şekilde görünür:

```

my-app/
├── .angular/           ← Build cache (gitignore'da)
├── .vscode/           ← VS Code launcher ve settings
├── node_modules/      ← Bağımlılıklar (gitignore'da)
├── public/            ← Static asset'ler
│   └── favicon.ico
├── src/               ← KAYNAK KOD
│   ├── app/          ← Uygulamanın kalbi
│   │   ├── app.config.ts ← Bootstrap konfigürasyonu
│   │   ├── app.routes.ts ← Routing tanımları
│   │   ├── app.html    ← Root component template
│   │   ├── app.scss    ← Root component CSS
│   │   ├── app.spec.ts ← Root component test
│   │   └── app.ts       ← Root component class
│   ├── index.html     ← Tek HTML dosyası
│   ├── main.ts        ← Entry point
│   └── styles.scss    ← Global styles
├── .editorconfig      ← Editor konfigürasyonu
├── .gitignore          ← Git ignore listesi
├── angular.json        ← Angular CLI yapılandırması
├── package.json       ← npm paketleri & scriptler
├── README.md          ← Proje dokümantasyonu
├── tsconfig.app.json  ← App için TS config
├── tsconfig.json      ← Ana TS config
└── tsconfig.spec.json ← Test için TS config

```

2 src/ Klasörü — Uygulamanın Kalbi

src/ klasörü uygulamanın tüm kaynak kodunu içerir. %95 zamanını burada geçireceksin.

src/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>MyApp</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root> ← Angular buraya render eder
</body>
</html>
```

SPA'nın "tek HTML"i. Tüm sayfa geçişleri JavaScript ile burada yapılır. Angular sadece <app-root> içine render eder.

src/main.ts — Entry Point

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

Uygulamanın başlangıç noktası. Burada Angular'a "App component'ini bootstrap et, bu config ile" diyoruz.

src/styles.scss — Global Styles

```
/* Bu dosya TÜM uygulamayı etkiler */

html, body {
  margin: 0;
  font-family: 'Roboto', sans-serif;
}

/* CSS reset, global typography, theme variables */
```

**Dikkat**

Component-specific CSS'leri buraya yazma. Sadece tüm uygulamayı etkilemesi gereken şeyler — reset, typography, theme variables, vs.

3 src/app/ — Component'lerin Yuvası

src/app/ uygulamanın gerçek kodunun yaşadığı yer. Component'ler, service'ler, modeller — hepsi burada.

app.ts — Root Component

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.scss']
})
export class App {
  protected readonly title = signal('my-app');
}
```

Bu uygulamanın kök bileşeni. Diğer tüm bileşenler bunun içinde render edilir.

Decorator İçeriği

Property	Anlamı
selector	HTML'de hangi tag ile çağrılacak (app-root)
imports	Bu component hangi modülleri/component'leri kullanıyor
templateUrl	HTML template dosyası (ayrı dosyada)
template	Inline HTML template
styleUrl	CSS/SCSS dosyası
styles	Inline CSS

app.html — Root Template

```
<main>
  <h1>{{ title() }}</h1>
  <router-outlet />
</main>
```

<router-outlet>: Aktif route'a göre component buraya yerleştirilir. Sayfa geçişlerinin gerçekleştiği yer.

app.scss — Component-scoped CSS

```
/* Bu dosyada yazılan CSS sadece App component'i etkiler */  
/* Angular bunu otomatik scope ediyor */  
  
main {  
  padding: 20px;  
}  
  
h1 {  
  color: #dd0031;  
}
```

✓ View Encapsulation

Bu CSS'in sadece bu component'i etkilemesi otomatik. Angular otomatik unique attribute selector ekliyor. Modül 4'te detaylı göreceğiz.

app.config.ts — Bootstrap Konfigürasyonu

```
import { ApplicationConfig, provideZonelessChangeDetection } from '@angular/core';  
import { provideRouter } from '@angular/router';  
import { routes } from './app.routes';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideZonelessChangeDetection(),  
    provideRouter(routes)  
  ]  
};
```

Eski NgModule'ün yerini alan dosya. Application-wide provider'ları burada tanımlıyoruz. Modül 3.4'te detaylı göreceğiz.

app.routes.ts — Routing Tanımları

```
import { Routes } from '@angular/router';  
  
export const routes: Routes = [  
  // Buraya route'larımızı tanımlayacağız  
  // { path: 'home', component: HomeComponent },  
  // { path: 'about', component: AboutComponent },  
];
```

URL'lere hangi component'lerin yükleneceğini tanımlıyoruz. Modül 9'da detaylı işleyeceğiz.

4 Konfigürasyon Dosyaları

package.json — npm'in Kalbi

```
{
  "name": "my-app",
  "version": "0.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "build": "ng build",
    "watch": "ng build --watch --configuration development",
    "test": "ng test"
  },
  "dependencies": {
    "@angular/animations": "^21.0.0",
    "@angular/common": "^21.0.0",
    "@angular/compiler": "^21.0.0",
    "@angular/core": "^21.0.0",
    "@angular/forms": "^21.0.0",
    ...
  },
  "devDependencies": {
    "@angular/build": "^21.0.0",
    "@angular/cli": "^21.0.0",
    ...
  }
}
```

Bölüm	Anlamı
scripts	npm run X ile çalıştırılan komutlar
dependencies	Production'da gereken paketler
devDependencies	Sadece geliştirmede gereken paketler

angular.json — Angular CLI Konfigürasyonu

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "my-app": {
      "projectType": "application",
      "schematics": { ... },
      "root": "",
      "sourceRoot": "src",
      "prefix": "app",
      "architect": {
        "build": { ... },
        "serve": { ... },
        "test": { ... }
      }
    }
  }
}
```

CLI'nin nasıl davranacağını söyler. Build çıktıları, test komutları, asset path'leri — hepsi burada. Modül 24'te derinlemesine işleyeceğiz.

tsconfig.json — TypeScript Yapılandırması

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "preserve",
    "strict": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true,
    "moduleResolution": "bundler",
    "esModuleInterop": true,
    "lib": ["ES2022", "dom"]
  }
}
```


Ayar	Anlamı
target	Hangi ECMAScript sürümüne derlenecek
strict	Strict mode (any kullanımı yasak gibi)
experimentalDecorators	@Component vs için (Modül 2.3)
lib	Hangi tip kütüphanelerine erişim (DOM, ES features)

5 public/ ve Diğer Klasörler

public/ — Static Assets

Doğrudan kopyalanan asset'ler. Build sonrası / root'tan erişilir.

```
public/  
├─ favicon.ico  
├─ images/  
│   ├─ logo.png  
│   └─ hero-bg.jpg  
└─ fonts/  
    └─ custom-font.woff2
```

Kullanım:

```
  
<!-- Build sonrası: my-app.com/images/logo.png -->
```

node_modules/ — Bağımlılıklar

npm install ile yüklenen tüm paketler. ASLA git'e commit edilmez! .gitignore'da var. Boyutu sıkça GB'leri bulur.

.angular/ — Build Cache

Angular CLI'nin build cache'i. Sonraki build'leri hızlandırır. .gitignore'da. Sorun çıkarsa sil, otomatik yeniden oluşur.

.vscode/ — VS Code Settings

```
.vscode/  
├─ extensions.json    ← Önerilen extension'lar  
├─ settings.json     ← Workspace ayarları  
└─ launch.json       ← Debug konfigürasyonu
```

VS Code kullanıyorsan otomatik gelir. Takım çalışmasında bu klasör commit edilebilir (öneriler tutarlı olsun diye).

6

Hangi Dosyaya Ne Sıklıkta Dokunmalıyım?

Dosya	Sıklık	Ne Zaman?
src/app/* (component'ler)	Çok sık	Her gün — kod yazıyorsun
src/styles.scss	Az	Global theme/typography değişiklikleri
src/main.ts	Çok az	Bootstrap akışı değişikliği
src/index.html	Çok az	Meta tags, favicon değişikliği
app.config.ts	Orta	Yeni provider eklerken (HttpClient gibi)
app.routes.ts	Orta	Yeni sayfa eklerken
package.json	Orta	Paket eklerken/silerken
angular.json	Az	Build config, asset path değişikliği
tsconfig.json	Çok az	TypeScript ayar değişikliği
public/	Orta	Yeni asset (resim, favicon)
node_modules/	Asla	Hiçbir zaman manuel dokunma

□ Mini Quiz

- **src/main.ts dosyasının görevi nedir?**

Cevap: Uygulamanın başlangıç noktası (entry point). bootstrapApplication ile App component'ini başlatır.

- **src/index.html ile src/app/app.html arasındaki fark nedir?**

Cevap: index.html SPA'nın tek HTML'i, sayfa kabuğu. app.html App component'inin template'i, içerik bunun içinde render edilir.

- **app.config.ts ne işe yarar?**

Cevap: Application-wide provider'ları tanımlar. Eski NgModule'ün yerini alır. Routing, HTTP client, zoneless gibi global ayarlar burada.

- **public/ klasörü ile src/assets/ arasındaki fark nedir?**

Cevap: public/ build sonrası doğrudan kopyalanır, root'tan erişilir (/images/...). Eski projelerde src/assets vardı, modern projelerde public/.

- **node_modules klasörü neden git'e commit edilmez?**

Cevap: Çok büyük (GB'ler), package.json zaten paketleri listeliyor, npm install ile yeniden oluşturulabilir, gereksiz repo şişmesi.

- **Component-scoped CSS otomatik mi çalışır?**

Cevap: Evet. Angular view encapsulation ile her component'in CSS'ine unique attribute ekler. styleUrls: './app.scss' ile bağlanan CSS sadece o component'i etkiler.

⚠ Anti-Patterns

- ✗ Component CSS'lerini global styles.scss'e yazmak — Component'in styleUrls'ünü kullan
- ✗ node_modules'ü git'e commit etmek — .gitignore'da olmalı
- ✗ angular.json'ı şişirmek — Olabildiğince default ayarları kullan
- ✗ main.ts'i karmaşıktırmak — Sadece bootstrap olmalı, business logic değil
- ✗ public/ yerine assets/ aramak — Modern Angular'da public/ kullanılır
- ✗ app.config.ts'i atlamak — Provider'lar burada toplanır, dağıtma

□ Bu Dersten Çıkarman Gerekenler

- ✓ Yeni bir Angular projesinin tüm dosya/klasörlerini biliyorum
- ✓ Her dosyanın amacını ve içeriğini anlıyorum
- ✓ src/app/ içinde nereye ne yazacağımı biliyorum
- ✓ angular.json, package.json, tsconfig.json arasındaki farkı kavradım
- ✓ public/ klasörünün modern Angular'daki rolünü biliyorum
- ✓ Hangi dosyaya ne sıklıkta dokunmam gerektiğini görüyorum

MODÜL 3 — DERS 3.3

Angular CLI Komutları

Angular CLI senin en yakın iş arkadaşın. `ng generate`, `ng serve`, `ng build`... Bu komutları derinlemesine öğreneceksin. Doğru kullanırsan saatlerce zaman kazanırsın.



Hedef

Modül 1.5'te temel komutları gördük. Şimdi her komutun tüm seçeneklerini, pratik kullanımlarını ve kısayolları öğreneceğiz.

1

CLI Felsefesi — Neden Önemli?

Angular CLI sadece "kod üreten araç" değil. Best practice'leri zorlayan, tutarlılık sağlayan, migration'ları otomatikleştiren bir asistan.

CLI ile vs Manuel

Konu	Manuel	CLI ile
Component oluşturma	5-6 dosya, dakikalar	1 komut, saniye
Naming convention	Tutarsız	Otomatik tutarlı
Test dosyası	Çoğunlukla atlanır	Otomatik üretilir
Routing güncelleme	Unutulur	Otomatik eklenir
Migration	Manuel, hatalı	Tamamen otomatik



Best Practice

Component, service, directive gibi şeyleri her zaman CLI ile oluşturun. Manuel yazmak hata kaynağıdır.

2

ng serve — Geliştirme Sunucusu

Temel kullanım:

```
ng serve
```

Default port: 4200. Hot reload aktif. Kod değişikliklerin tarayıcıya anında yansır.

Önemli Flag'ler

```
# Farklı port
ng serve --port 4300

# Tarayıcıyı otomatik aç
ng serve --open
ng serve -o          # Kısa hali

# Network'te diğer cihazlardan erişim
ng serve --host 0.0.0.0

# Production konfigürasyonu ile (test amaçlı)
ng serve --configuration production

# SSL/HTTPS
ng serve --ssl

# Live reload kapatma (debug için)
ng serve --live-reload false
```

Pratik Senaryolar

1. Mobil cihazdan test:

```
ng serve --host 0.0.0.0 --port 4200
# Sonra mobile cihazdan: http://[bilgisayar-ip]:4200
```

2. Birden fazla proje aynı anda:

```
# Terminal 1
cd backend-app && ng serve --port 4200

# Terminal 2
cd frontend-app && ng serve --port 4201
```

3 ng generate — Kod Üretici

CLI'nin en güçlü komutu. Component, service, directive, pipe — hepsini otomatik üretir.

Component Üretimi

```
ng generate component user-card
# Kısa hali:
ng g c user-card
```

Üretilen dosyalar:

```
CREATE src/app/user-card/user-card.html
CREATE src/app/user-card/user-card.spec.ts
CREATE src/app/user-card/user-card.ts
CREATE src/app/user-card/user-card.scss
```

Component Generate Flag'leri

```
# Inline template (HTML'i .ts içinde)
ng g c user-card --inline-template
ng g c user-card -t

# Inline style (CSS'i .ts içinde)
ng g c user-card --inline-style
ng g c user-card -s

# Test dosyası oluşturma
ng g c user-card --skip-tests

# CSS yerine başka format
ng g c user-card --style=css

# Özel klasöre koy
ng g c features/auth/login

# Standalone (default v17+)
ng g c user-card --standalone

# Selector prefix'i değiştir
ng g c user-card --selector my-card
```

Diğer Generator'lar

Komut	Üretir	Kısa Hali
ng g component	Component	ng g c
ng g service	Service	ng g s
ng g directive	Directive	ng g d
ng g pipe	Pipe	ng g p
ng g guard	Route Guard	ng g g
ng g interceptor	HTTP Interceptor	ng g it
ng g resolver	Route Resolver	ng g r
ng g class	Plain TS Class	ng g cl
ng g interface	TS Interface	ng g i
ng g enum	TS Enum	ng g e

Pratik Örnekler

```
# Auth feature klasörü altına service
ng g s features/auth/auth

# Functional guard (modern)
ng g g auth --functional

# Sadece interface (logic yok)
ng g i models/user

# Feature klasörü altında pipe
ng g p shared/pipes/capitalize
```


4 ng build — Production Build

Temel kullanım:

```
ng build
```

Default: production konfigürasyonu (Angular 14+). Optimize edilmiş, minified kod üretir.

Çıktı:

```
dist/my-app/  
├─ browser/  
│   ├─ index.html  
│   ├─ main-XXXXX.js      ← Hash'li dosya isimleri (cache için)  
│   ├─ polyfills-XXXXX.js  
│   ├─ styles-XXXXX.css  
│   └─ chunk-XXXXX.js     ← Lazy-loaded chunks  
└─ stats.json             ← Bundle analizi
```

Önemli Flag'ler

```
# Development build (faster, larger)  
ng build --configuration development  
  
# Bundle stat'leri ve analiz  
ng build --stats-json  
# Sonra:  
npx webpack-bundle-analyzer dist/my-app/stats.json  
  
# Watch mode (otomatik rebuild)  
ng build --watch  
  
# Source maps üret (debug için)  
ng build --source-map  
  
# Output path değiştir  
ng build --output-path dist/custom-folder  
  
# Base href (alt dizinde deploy için)  
ng build --base-href /my-app/  
  
# Specific configuration  
ng build --configuration staging
```

5 ng test, ng update, ng add

ng test — Test Çalıştırma

```
# Tüm testleri çalıştır (watch mode)
ng test

# Tek seferlik (CI için)
ng test --watch=false

# Coverage raporu ile
ng test --code-coverage

# Headless (browser açma)
ng test --browsers=ChromeHeadless

# Specific test dosyası
ng test --include="**/user-card.spec.ts"
```

ng update — Angular Versiyonu Güncelleme

```
# Hangileri güncellenebilir?
ng update

# Angular core ve CLI'yi güncelle
ng update @angular/core @angular/cli

# Belirli bir versiyona
ng update @angular/core@21 @angular/cli@21

# Migration'ları çalıştır
ng generate @angular/core:standalone

# Force (uyarıları görmezden gel)
ng update @angular/core --force
```

Migration Süper Gücü

Angular ekibi her major versiyon için migration script'leri yazar. ng update bunları otomatik çalıştırır. Eski projeler için inanılmaz değerli.

ng add — Paket Ekleme + Konfigürasyon

npm install'den farkı: paketi kurar + Angular projeye otomatik entegre eder (config dosyalarını günceller).

```
# Angular Material ekle
ng add @angular/material
# - Material yükler
# - app.config.ts'e provideAnimations() ekler
# - Theme'i seçtirir
# - styles.scss'e Material import ekler

# PWA desteği ekle
ng add @angular/pwa

# NgRx ekle
ng add @ngrx/signals

# Tailwind ekle
ng add @ngneat/tailwind
```

6 --dry-run ve --help — Güvenli Keşif

--dry-run: "Bu komut ne yapardı?"

```
# Komutu çalıştırmadan ne yapacağını gör
ng g c user-card --dry-run

# Çıktı:
# CREATE src/app/user-card/user-card.html (xxx bytes)
# CREATE src/app/user-card/user-card.spec.ts (xxx bytes)
# CREATE src/app/user-card/user-card.ts (xxx bytes)
# CREATE src/app/user-card/user-card.scss (xxx bytes)
# NOTE: The "dryRun" flag means no changes were made.
```

✓ Güvenli Yaklaşım

Şüphede kaldığında --dry-run ekle. Hiçbir dosya oluşturulmaz, sadece ne yapacağı gösterilir.

--help: Tüm Seçenekleri Gör

```
# Genel yardım
ng help
ng --help

# Komut bazında yardım
ng generate --help
ng g c --help
ng build --help
```

7

Cheat Sheet — Sık Kullanılan Komutlar

```
# Proje yönetimi
ng new my-app          # Yeni proje
ng serve --open        # Dev server + tarayıcı aç
ng build               # Production build
ng test                # Test runner
ng e2e                 # End-to-end test

# Code generation (en sık)
ng g c <name>          # Component
ng g s <name>          # Service
ng g d <name>          # Directive
ng g p <name>          # Pipe
ng g g <name>          # Guard
ng g it <name>         # Interceptor
ng g i <name>          # Interface

# Bağımlılık & güncelleme
ng add @angular/material # Paket ekle + entegre et
ng update              # Hangileri güncellenebilir
ng update @angular/core # Belirli paketi güncelle

# Yardım & keşif
ng version             # Versiyon bilgisi
ng help               # Tüm komutlar
ng <komut> --help     # Komut detayı
ng g c <name> --dry-run # Güvenli ön izleme

# Servis kontrolü
ng serve --port 4300   # Farklı port
ng serve --host 0.0.0.0 # Network erişimi
ng serve --ssl         # HTTPS

# Build varyasyonları
ng build --watch        # Otomatik rebuild
ng build --stats-json   # Bundle analizi
ng build --base-href /app/ # Subdirectory deploy
```

□ Mini Quiz

- **ng generate component yerine ne yazabilirim (kısa hali)?**

Cevap: ng g c

- **--dry-run flag'i ne işe yarar?**

Cevap: Komutu gerçekte çalıştırmadan ne yapacağını gösterir. Şüphede kaldığında güvenli keşif için ideal.

- **ng add ile npm install arasındaki fark nedir?**

Cevap: npm install sadece paketi yükler. ng add hem yükler hem de Angular projesine otomatik entegre eder (config dosyalarını günceller).

- **ng update neden manuel migration'dan daha iyidir?**

Cevap: Angular ekibi her major versiyon için migration script'leri yazar. ng update bunları otomatik çalıştırarak kodumu modernize eder.

- **ng serve --host 0.0.0.0 ne işe yarar?**

Cevap: Dev server'ı network'teki diğer cihazlara açar. Mobil cihazdan test etmek için ideal.

- **ng build çıktısı nereye yazılır?**

Cevap: dist//browser/ klasörüne. Hash'li dosya isimleri ile minified kod üretilir.

⚠ Anti-Patterns

- ✗ Component'leri manuel oluşturmak — CLI'nin tutarlılığını kaybedersin
- ✗ Test dosyalarını skip-tests ile atlamak — Sonra test yazmak zor
- ✗ ng update yerine manuel migration — Hata kaynağı, çok zaman kaybı
- ✗ npm install kullanıp ng add'i unutmak — Paket entegrasyonu eksik kalır
- ✗ --dry-run'sız riskli komutlar — "Acaba ne yapacak?" diye merak ederken
- ✗ --help'i görmezden gelmek — Çoğu sorunun cevabı orada

□ Bu Dersten Çıkarman Gerekenler

- ✓ CLI'nin neden kritik olduğunu anlıyorum
- ✓ ng generate ile component, service, directive, pipe üretebiliyorum
- ✓ ng serve'in tüm flag'lerini biliyorum (port, host, ssl, configuration)
- ✓ ng build'i production'a deploy için kullanabiliyorum
- ✓ ng update ile Angular versiyonu güncelleyebiliyorum
- ✓ ng add ile paket eklemenin avantajını biliyorum
- ✓ --dry-run ve --help kullanarak güvenli keşif yapabiliyorum

MODÜL 3 — DERS 3.4

app.config.ts & Bootstrapping

Modern Angular'ın "kalbi": *app.config.ts*. Eski NgModule'ün yerini alan bu dosya, uygulamamızın global yapılandırmasını içerir.



Hedef

Bu dersin sonunda provideX() fonksiyonlarının ne işe yaradığını, application-wide servisleri nasıl yapılandıracağını ve modern Angular'ın bootstrap akışını tamamen anlayacaksın.

1

Bootstrap Akışı — Uygulama Nasıl Başlar?

Bir Angular uygulaması başladığında ne olur? Tarayıcı index.html'i yükledikten sonra başlayan akış şu:

1. Tarayıcı index.html'i yükler
2. `<script type="module" src="main.ts">` yüklenir
3. `main.ts` → `bootstrapApplication(App, appConfig)` çağrılır
4. Angular runtime başlatılır
5. `appConfig`'deki tüm provider'lar kaydedilir
6. App component create edilir
7. App component, `<app-root>` içine render edilir
8. Routing aktif olur, ilk route yüklenir

main.ts — Entry Point

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

Çok basit. "App component'ini, appConfig ile bootstrap et" diyoruz. Detaylar appConfig'de.

2

Eski NgModule vs Modern Standalone

Eski Yaklaşım (Angular 14 öncesi)

```
// app.module.ts (artık YOK)
@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    UserListComponent,
    // ... her component burada listelenmeli
  ],
  imports: [
    BrowserModule,
    HttpClientModule,
    FormsModule,
    AppRoutingModule
  ],
  providers: [
    UserService,
    { provide: API_URL, useValue: '/api' }
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}

// main.ts
platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Sorunlar: Çok boilerplate, NgModule'lerin neyi neye bağladığını anlamak zor, lazy loading karmaşık, tree-shaking optimum değil.

Modern Yaklaşım (v14+, default v17+)

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(),
    provideRouter(routes),
    provideHttpClient(),
  ]
};

// main.ts
bootstrapApplication(App, appConfig);
```


**Avantajları**

- Boilerplate %80 az — NgModule yok
- Component'ler self-contained — Her component kendi imports'unu belirler
- Tree-shaking optimal — Kullanılmayan kod build'e girmez
- Lazy loading basit — Direkt async import
- provideX() fonksiyonları — Type-safe, açık

3

ApplicationConfig Anatomisi

Tipini görelim:

```
interface ApplicationConfig {  
  providers: Array<Provider | EnvironmentProviders>;  
}
```

Tek bir property: providers. İçinde provideX() fonksiyonları çağrılır.

Tipik bir app.config.ts

```
import { ApplicationConfig, provideZonelessChangeDetection } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { provideAnimations } from '@angular/platform-browser/animations';

import { routes } from './app.routes';
import { authInterceptor } from './core/interceptors/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    // Reactivity
    provideZonelessChangeDetection(),

    // Routing
    provideRouter(
      routes,
      withComponentInputBinding() // Route params direkt input olarak gelsin
    ),

    // HTTP
    provideHttpClient(
      withInterceptors([authInterceptor])
    ),

    // Animations
    provideAnimations(),

    // Custom providers
    { provide: API_URL, useValue: 'https://api.example.com' },
  ]
};
```

4 Yaygın provideX() Fonksiyonları

provideZonelessChangeDetection()

```
provideZonelessChangeDetection()
```

Modül 1.4'te öğrendik: Zone.js'siz çalışan modern reactivity sistemi. v21'de default.

provideRouter()

```
provideRouter(  
  routes,  
  withComponentInputBinding(),    // Route params → component input  
  withViewTransitions(),         // View Transitions API  
  withInMemoryScrolling({        // Scroll davranışı  
    scrollPositionRestoration: 'top',  
    anchorScrolling: 'enabled'  
  }),  
  withRouterConfig({  
    paramsInheritanceStrategy: 'always'  
  })  
)
```

Routing aktif eder. Modül 9'da derinlemesine işleyeceğiz.

provideHttpClient()

```
provideHttpClient(  
  withInterceptors([authInterceptor, errorInterceptor]),  
  withFetch(),                // Fetch API kullan (XHR yerine)  
  withRequestsMadeViaParent()  
)
```

HttpClient'ı kullanılabilir kılar. Modül 11'de detaylı işleyeceğiz.

provideAnimations() / provideAnimationsAsync()

```
// Senkron - daha basit ama bundle'a tüm animations dahil  
provideAnimations()  
  
// Async - lazy load, bundle'da yer kazanır  
provideAnimationsAsync()
```

Angular Animations sistemi. Material kullanırken zorunlu. Modül 15'te işleyeceğiz.

Diğer Yaygın provider'lar

```
// Forms
import { FormsModule, ReactiveFormsModule } from '@angular/forms';
// Forms modülleri olarak hala import edilir, provider değil

// Service Worker (PWA)
import { provideServiceWorker } from '@angular/service-worker';
provideServiceWorker('ngsw-worker.js', {
  enabled: !isDevMode(),
  registrationStrategy: 'registerWhenStable:30000'
}),

// Client Hydration (SSR)
import { provideClientHydration } from '@angular/platform-browser';
provideClientHydration(),

// Date adapter (Material)
import { provideNativeDateAdapter } from '@angular/material/core';
provideNativeDateAdapter(),
```

5 Custom Provider'lar — InjectionToken & DI

Sadece Angular'ın provideX'lerini değil, kendi provider'larını da yazabilirsin. Bu DI sisteminin temeli.

1. Value Provider

```
// app.config.ts
import { InjectionToken } from '@angular/core';

export const API_URL = new InjectionToken<string>('API_URL');
export const APP_VERSION = new InjectionToken<string>('APP_VERSION');

export const appConfig: ApplicationConfig = {
  providers: [
    { provide: API_URL, useValue: 'https://api.example.com' },
    { provide: APP_VERSION, useValue: '1.0.0' },
  ]
};

// Component'te kullanım
import { inject } from '@angular/core';

@Component({...})
export class MyComponent {
  private apiUrl = inject(API_URL);    // 'https://api.example.com'
  private version = inject(APP_VERSION); // '1.0.0'
}
```

2. Factory Provider

```
export const appConfig: ApplicationConfig = {
  providers: [
    {
      provide: API_URL,
      useFactory: () => {
        return window.location.hostname === 'localhost'
          ? 'http://localhost:3000/api'
          : 'https://api.production.com';
      },
    },
  ],
};
```

3. Class Provider (Alternative)

```
abstract class Logger {
  abstract log(message: string): void;
}

class ConsoleLogger extends Logger {
  log(message: string) { console.log(message); }
}

class SentryLogger extends Logger {
  log(message: string) { /* Sentry'a gönder */ }
}

export const appConfig: ApplicationConfig = {
  providers: [
    // Production'da SentryLogger, dev'de ConsoleLogger
    {
      provide: Logger,
      useClass: environment.production ? SentryLogger : ConsoleLogger
    }
  ]
};
```



Detay

DI sisteminin tüm gücünü Modül 7 (Services & DI)'da işleyeceğiz. Şimdilik provideX'lerin dışında custom provider yazabildiğini bilmen yeterli.

6 Environment-Specific Config

Development ve production için farklı ayarlar:

```
// src/environments/environment.ts (development)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api',
  enableDebugTools: true
};

// src/environments/environment.production.ts (production)
export const environment = {
  production: true,
  apiUrl: 'https://api.example.com',
  enableDebugTools: false
};
```

app.config.ts'de kullanım:

```
import { environment } from '../environments/environment';

export const appConfig: ApplicationConfig = {
  providers: [
    { provide: API_URL, useValue: environment.apiUrl },
    { provide: PROD_MODE, useValue: environment.production },
  ]
};
```

Build sırasında Angular CLI doğru environment dosyasını kullanır. Modül 24'te detaylı işleyeceğiz.

□ Mini Quiz

- **app.config.ts dosyasının modern Angular'daki rolü nedir?**

Cevap: Eski NgModule'ün yerini alır. Application-wide provider'ları (router, HTTP, animations vb.) tanımlar.

- **bootstrapApplication() ne yapar?**

Cevap: Angular uygulamasını başlatır. Verilen App component'i ve config ile runtime'ı kurar, app component'i index.html'deki içine render eder.

- **provideRouter() ile provideHttpClient() arasındaki fark nedir?**

Cevap: provideRouter routing sistemini, provideHttpClient HTTP isteklerini aktif eder. İkisi de farklı subsistemler için provider sağlar.

- **InjectionToken neden gereklidir?**

Cevap: Primitive değerleri (string, number, boolean) DI sistemine vermek için. inject(string) çalışmaz, inject(API_URL) çalışır.

- **Modern Angular'da NgModule kullanılır mı?**

Cevap: Yeni projelerde hayır. Eski projelerde geriye uyumluluk için var. Standalone components ile NgModule'e ihtiyaç kalmadı.

- **Environment-specific config nasıl çalışır?**

Cevap: src/environments/ altında environment.ts (dev) ve environment.production.ts (prod) dosyaları. Build sırasında Angular CLI doğru olanı seçer.

⚠ Anti-Patterns

- ✗ Yeni projede NgModule kullanmak — Standalone tercih et
- ✗ Provider'ları her component'te tekrar tanımlamak — App-wide ise app.config.ts'e koy
- ✗ InjectionToken'sız primitive inject etmeye çalışmak — string, number için token gerekli
- ✗ main.ts'i karmaşıktırmak — Sadece bootstrapApplication olmalı
- ✗ Environment'a hardcoded değerler — apiUrl gibi şeyler environment.ts'de olmalı
- ✗ provideAnimations'ı eklemeyen Material kullanmak — Material animasyonu gerektirir

□ Bu Dersten Çıkarman Gerekenler

- ✓ Angular bootstrap akışını adım adım biliyorum
- ✓ Eski NgModule ile modern Standalone yaklaşımının farkını kavradım
- ✓ app.config.ts içeriğini ve amacını anlıyorum
- ✓ Yaygın provideX() fonksiyonlarını biliyorum (router, http, animations)

- ✓ Custom provider yazabiliyorum (value, factory, class)
- ✓ InjectionToken'ın neden gerekli olduğunu anlıyorum
- ✓ Environment-specific config kullanabiliyorum

MODÜL 3 — DERS 3.5

Angular DevTools

Bir Angular geliştiricisinin en güçlü silahlarından biri: Angular DevTools. Component tree görmek, change detection profiling, signal graph — hepsi bu extension ile.



Hedef

DevTools'un her özelliğini öğreneceğiz: Component Explorer, Profiler, Router tree, Injector tree. Debug etmek artık inanılmaz kolay olacak.

1

Kurulum ve Aktivasyon

Chrome / Edge için

- Chrome Web Store: <https://chromewebstore.google.com>
- "Angular DevTools" ara
- "Add to Chrome" tıkla

Firefox için

- Firefox Add-ons: <https://addons.mozilla.org>
- "Angular DevTools" ara
- "Add to Firefox" tıkla

Aktivasyon

- Bir Angular sitesi aç (örn: localhost:4200)
- F12 ile DevTools aç
- Üst sekme barında "Angular" sekmesi görünmeli



Production Site'larda Çalışmaz

DevTools sadece development build'lerde aktif. Production build'lerde Angular debug bilgilerini sızdırmamak için DevTools devre dışıdır. Bu güvenlik için iyi.

2

Component Explorer — Component Tree

En sık kullanacağın özellik. Solda component tree, sağda seçili component'in detayları.

Sol Panel — Component Tree

```
App
├── HeaderComponent
├── SidebarComponent
│   └── NavigationComponent
└── RouterOutlet
    └── DashboardComponent
        ├── StatsCardComponent (×3)
        └── RecentActivityComponent
```

DOM'un değil, Angular component'lerin hiyerarşisi. Component'lere tıklayarak detaylarını görebilirsin.

Sağ Panel — Component Detayları

- Properties — Component'in property'leri (state, signals)
- Inputs — Component'e gelen input değerleri
- Outputs — Component'in yaydığı event'ler
- Listeners — Bağlı event listener'lar
- Dependencies — Inject edilmiş servisler

Canlı Edit

Property değerlerini DevTools'tan değiştirebilirsin! Örneğin `count = signal(0)` property'sini 99 yap, UI anında güncellenir.



Pratik İpucu

Component'in source kodunu görmek için: component'i seç → sağ üstteki "<>" simgesine tıkla → VS Code'da/IDE'de açılır.

3

Profiler — Performance Analizi

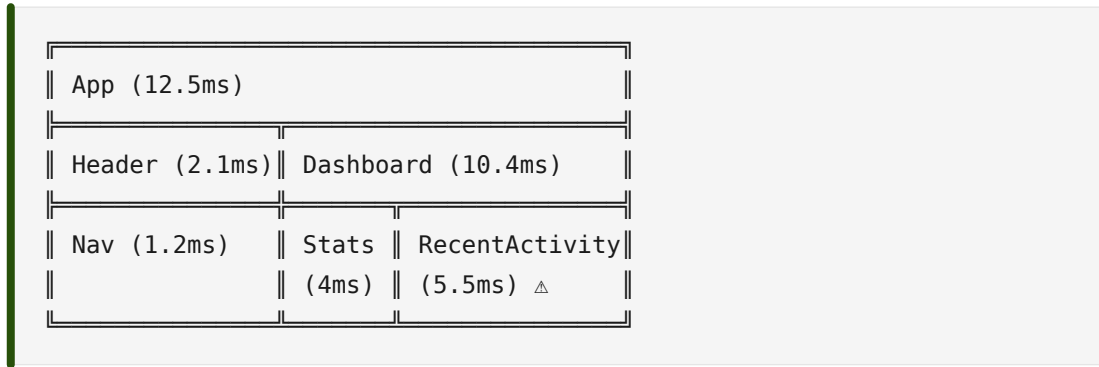
Performance bottleneck'leri bulmak için altın değerinde araç. Angular'ın change detection cycle'larını kaydeder.

Profil Etmek

- "Profiler" sekmesine geç
- "Start recording" butonuna bas
- Test etmek istediğin işlemi yap (buton tıkla, sayfa geç, vb.)
- "Stop recording" butonuna bas

Sonuç: Flame Graph

Şöyle bir görüntü gelir:



Her bar bir component'in render süresini gösterir. Geniş bar = yavaş render. Bottleneck'leri buradan bulursun.

Pratik Senaryolar

1. "Buton tıklayınca neden 1 saniye bekliyor?"

Profiler'ı başlat → Butona tıkla → Stop
 → Hangi component'in render uzun sürdüğünü gör
 → Optimize et (OnPush, computed, memoize)

2. "Sayfa scroll edilince yavaşlıyor"

Profiler'ı başlat → Hızlı scroll → Stop
 → Çok fazla component re-render mi oluyor?
 → Track by, virtual scrolling kullan



Hedef Süre

Bir component render'ı 16ms'den az olmalı (60 FPS için). Bu üstü artarsa kullanıcı "lag" hissetmeye başlar.

4 Router Tree — Routing Görünümü

Tanımladığın tüm route'ları hierarşik gösterir. "Aktif route hangisi?", "Lazy chunk'lar yüklendi mi?" gibi soruları cevaplar.

```

/                                ← root
├─ /home (HomeComponent)
├─ /products
│   ├─ /products/:id           ← param
│   └─ /products/new
└─ /admin (lazy)               ← Lazy loaded chunk
    ├─ /admin/users
    └─ /admin/settings
  
```

Pratik kullanım:

- Hangi route aktif?
- Hangi route lazy loaded?
- Route param'ları nasıl gelmiş?
- Guard'lar tetiklenmiş mi?

5 Injector Tree — DI Hiyerarşisi

Dependency Injection sisteminin görünümü. Hangi servis nereye inject edilmiş, hangi seviyede sağlanmış?

```

Root Injector
├─ HttpClient
├─ Router
├─ ApplicationRef
└─ Component Injectors
    ├─ App
    │   ├─ UserService
    │   └─ ThemeService
    └─ ProductsComponent
        └─ ProductService (component-scoped!)
  
```



Özellikle Yararlı

"NullInjectorError: No provider for X" hatası alıyorsun? Injector tree'de provider'ın nereye konduğuna bak. Component scope'unda mı, root'ta mı?

6 Pratik Workflow İpuçları

İpucu 1: Hızlı Component Bulma

Sayfada bir element var, hangi component'in olduğunu öğrenmek istiyorsun:

- DevTools'ta "Inspect" simgesine tıkla
- Sayfada o element'e tıkla
- Angular DevTools otomatik o component'e geçer

İpucu 2: \$ng Magic Variable

Bir component seçip console'a geçtiğinde, o component'i \$ng0 ile referans alabilirsin:

```
// Console'da
$ng0          // Seçili component instance
$ng0.title()   // Signal değerini al
$ng0.count.set(42) // Signal'i değiştir
```

İpucu 3: ng.applicationRef

```
// Tüm Angular API'sine console'dan erişim
ng.applicationRef.tick()    // Manual change detection
ng.getDirectives($0)        // Element'teki directive'leri al
ng.getOwningComponent($0)   // Element'in component'ini al
```

İpucu 4: Signal Graph

v17+ ile bir signal'in dependency graph'ını görebilirsin. Component Explorer'da signal'e sağ tık → "Show Graph".

```
count (signal)
  ↓
doubled (computed) ← count'a bağlı
  ↓
displayText (computed) ← doubled ve threshold'a bağlı
```

7 Yaygın Sorunlar ve Çözümleri

Sorun 1: "Angular sekmesi görünmüyor"

- Production build'inde misin? Development'a geç (ng serve)

- Sayfayı yenile (F5)
- Extension'ı disable/enable et

Sorun 2: "Component görünüyor ama state yok"

- Build cache problemi. .angular/ sil, ng serve tekrar başlat
- Source maps'i kontrol et: ng serve --source-map

Sorun 3: "Profile alırken çok yavaşlıyor"

- Profil normal çalışmadan yavaş — bu beklenir
- Kısa süre profil al (3-5 saniye yeterli)
- Tüm tablaları kapat, sadece test ettiğin sayfa kalsın

📝 Mini Quiz

- **Angular DevTools production build'lerde neden çalışmaz?**

Cevap: Güvenlik nedeniyle. Production'da Angular debug bilgilerini (component yapısı, state) sızdırmamak için DevTools devre dışıdır.

- **Component Explorer'da component property'lerini değiştirebilir miyim?**

Cevap: Evet. State'i canlı değiştirebilirsin, UI anında güncellenir. Test ve debug için ideal.

- **Profiler'da geniş bar ne anlama gelir?**

Cevap: O component'in render süresi uzun. Performance bottleneck. Optimize edilmesi gereken bir yer.

- **60 FPS için bir frame'in maksimum ne kadar sürmesi gerekir?**

Cevap: 16ms ($1000ms / 60 = 16.67ms$). Bu süreyi geçen render'lar lag'a neden olur.

- **\$ng0 magic variable ne işe yarar?**

Cevap: DevTools'ta seçili component'i console'dan referans alma. \$ng0.title() ile property'lerine erişebilirsin.

- **"NullInjectorError" hatası aldığında nereye bakmalısın?**

Cevap: Injector tree'ye. Provider'ın hangi seviyede sağlandığına bak. Component scope'unda mı, root'ta mı, hiç sağlanmamış mı?

⚠️ Anti-Patterns

- ✗ console.log ile debug etmek — DevTools her şeyi gösteriyor, console'a gerek yok
- ✗ DevTools'u görmezden gelmek — Profiler bottleneck'leri inanılmaz hızlı bulur
- ✗ Production'da debug etmek — DevTools production'da çalışmaz, dev'de test et
- ✗ 16ms üstü render'ları normal saymak — Kullanıcı lag hisseder, optimize et
- ✗ Injector tree'yi atlayıp provider hatası ile uğraşmak — Önce ağaca bak
- ✗ ng global'ini bilmemek — Console'dan Angular API'sine erişim çok güçlü

📝 Bu Dersten Çıkarman Gerekenler

- ✓ Angular DevTools'u tarayıcıma kurdum
- ✓ Component Explorer ile component tree'yi keşfedebiliyorum
- ✓ Component property'lerini canlı değiştirebiliyorum
- ✓ Profiler ile performance ölçebiliyorum
- ✓ Router Tree ve Injector Tree'yi okuyabiliyorum
- ✓ \$ng0 magic variable ve ng global'ini biliyorum
- ✓ Bottleneck bulma workflow'umu oluşturdum

MODÜL 3 — DERS 3.6

MCP Server — AI ile Modern Angular

AI çağında programcılar yalnız değil. Angular MCP Server, Cursor/Claude Code/VS Code Copilot gibi AI tool'larına Angular'ın güncel best practice'lerini öğretir.



Hedef

MCP Server'in ne olduğunu, neden devrim olduğunu, nasıl kuracağını ve günlük geliştirme akışına nasıl entegre edeceğini öğreneceksin.

1

Problem: AI'ların Eski Bilgisi

Cursor, Copilot, ChatGPT gibi AI araçları training data'larından çalışır. Bu data aylar/yıllar öncesinden.

Eski Dünya (MCP'siz)

Sen Cursor'a yazıyorsun:

"Angular'da bir reactive form component'i yap"

Cursor 2 yıl eski training data'dan kod üretiyor:

- @NgModule kullanıyor (modern Angular standalone)
- *ngIf, *ngFor (modern @if, @for var)
- @Input() decorator (modern input() function var)
- HttpClientModule import ediyor (provideHttpClient var)
- FormsModule'a NgModule'den import ediyor
- Template-driven form yazıyor (Reactive Forms tercih edilir)

Sonuç: Sen düzeltmek için 30 dakika harcıyorsun. AI'nın yardımı negatif değer üretiyor.

Yeni Dünya (MCP ile)

Sen Cursor'a yazıyorsun:

"Angular'da bir reactive form component'i yap"

Cursor MCP üzerinden Angular CLI'ye bağlanıyor:

1. find_examples ile en güncel form örneklerini alır
2. get_best_practices ile aktif standartları öğrenir
3. v21 standartlarında modern kod üretir:
 - Standalone component
 - input(), output(), inject() functions
 - @if, @for control flow
 - Signal-based state
 - provideHttpClient() ile yapılandırılmış



Sonuç

AI seninle aynı sayfada. Modern, doğru, optimize kod üretiyor.

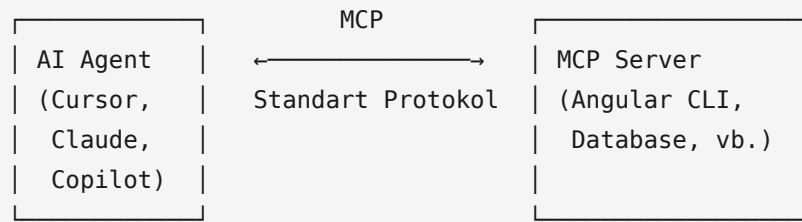
2

MCP (Model Context Protocol) Nedir?

MCP (Model Context Protocol): 2024'te Anthropic'in çıkardığı bir açık standart. AI agent'lar ile araçlar/servisler arasındaki iletişimi standartlaştırır.

Mental Model

USB gibi düşün. Eskiden her cihaz farklı kablo isterdi. USB ile her cihaz aynı port'u kullanıyor. MCP de AI ile araç/servisler arasındaki "USB".



Angular ekibi: "Angular CLI'yi MCP Server yapalım. AI'lar Angular'a doğrudan bağlanabilsin."

3 Angular MCP Server'ı Çalıştırma

En basit yol:

```
ng mcp
```

Bu komut bir MCP server başlatır. AI tool'lar bu server'a bağlanıp Angular'ın güncel API'sine erişir.

Sürekli Çalıştırma

ng mcp'yi her sefer manuel başlatmak istemezsin. AI tool'una nasıl başlatacağını söylersin, otomatik çalıştırır.

4 Cursor IDE'ye Entegrasyon

Cursor, popüler bir AI IDE'sidir. Angular MCP Server'ı Cursor'a entegre edelim:

1. Adım: .cursor/mcp.json oluştur

Proje root'unda .cursor/ klasörü içinde:

```
// .cursor/mcp.json
{
  "mcpServers": {
    "angular": {
      "command": "npx",
      "args": ["-y", "@angular/cli@latest", "mcp"]
    }
  }
}
```

2. Adım: Cursor'u yeniden başlat

Cursor MCP config'ini okuyup Angular MCP Server'a bağlanır.

3. Adım: Test

Cursor chat'inde:

```
@angular Modern bir Angular signal-based form component'i yap.
Validation ve error handling ekle.
```

Cursor MCP üzerinden Angular CLI'ye sorar, modern v21 kodu döner.

5

Claude Code (Terminal) Entegrasyonu

Claude Code, Anthropic'in komut satırı AI aracı. Terminal'den çalışırken AI yardımı alabilirsin.

Kurulum (Linux/macOS/Ubuntu)

```
# Claude Code'u aç (proje klasöründe)
claude

# MCP server ekle
claude mcp add --scope user angular-cli -- npx --yes @angular/cli@latest mcp --read-only
```

i

Linux'ta Önemli

Komutu Claude Code'un içinden çalıştır. Yani önce claude ile başlat, sonra orada bu komutu yaz.

Kurulum (Windows)

```
# Normal PowerShell'den (Claude Code'un içinden değil)
claude mcp add --scope user angular-cli -- npx --yes @angular/cli@latest mcp --read-only
```

Doğrulama

```
claude mcp list
```

"angular-cli" listede görünmeli.

6

VS Code Copilot Entegrasyonu

GitHub Copilot v1.300+ MCP destekler. Setup:

```
// .vscode/mcp.json
{
  "servers": {
    "Angular CLI": {
      "type": "stdio",
      "command": "npx",
      "args": ["@angular/cli", "mcp"]
    }
  }
}
```

VS Code'u yeniden başlat. Copilot Chat artık Angular MCP'ye bağlı.

7

Angular MCP Tools — Neler Yapabilir?

Angular MCP Server şu tool'ları sunar (AI bunları otomatik kullanır):

Tool	Ne Yapar?
find_examples	Angular ekibinin kürate ettiği güncel örneklerden arama
get_best_practices	Şu anki standartları, anti-pattern'leri öğretir
search_documentation	Resmi Angular docs'tan arama
ai_tutor	Etkileşimli AI öğretmen modu
modernize	Eski Angular kodunu v21'e dönüştürme önerisi
list_projects	Workspace'deki Angular projelerini listele
ng_generate	Component, service vs üretmeye yardım

AI bu tool'ları nasıl kullanır?

Sen: "Angular signals nedir?"

AI (arka planda):

1. search_documentation tool'unu çağırır → "signals" arar
2. find_examples ile signal kullanım örneklerini alır
3. get_best_practices ile modern kullanım önerilerini alır
4. Tüm bilgiyi sentezleyip cevap üretir

Sen sadece doğru, güncel, örnekli cevap görürsün.

8

Pratik İpuçları

İpucu 1: --read-only Kullan

ng mcp --read-only ile MCP'nin sadece okuma yapmasını sağla. Yanlışlıkla dosya değiştirilmesi tehlikesini ortadan kaldırır.

İpucu 2: Workspace İçinde Çalıştır

MCP Server'ı bir Angular projesinin içinden çalıştır. Aksi halde projeye özel context göremez.

İpucu 3: Verify the Output

AI MCP'den güncel bilgi alsa bile, ürettiği kodu mutlaka kontrol et. Çoğu zaman doğru ama %100 değil.



Bu Eğitimde MCP Kullanma

Bu eğitim boyunca yazacağın kodda MCP'yi kullanmanı şiddetle tavsiye ediyorum. Sen Cursor/Claude Code'da Angular kodu yazarken AI sana modern v21 standartlarında yardım edecek. Bu, eğitimin etkinliğini katlayarak artırır.

□ Mini Quiz

- **MCP nedir, kim çıkardı?**

Cevap: Model Context Protocol. 2024'te Anthropic çıkardı. AI agent'lar ile araçlar arasındaki iletişimi standartlaştıran açık bir standart.

- **AI tool'larının "eski bilgi" problemi nedir?**

Cevap: Cursor, Copilot gibi AI'lar training data'larından çalışır. Bu data aylar/yıllar öncesinden, modern Angular'ı (signals, standalone, control flow) bilmez.

- **Angular MCP Server'ı nasıl başlatırım?**

Cevap: `ng mcp` komutuyla. Veya AI tool'una otomatik başlatması için config dosyasında belirtirsin.

- **Cursor'a Angular MCP'yi nasıl entegre ederim?**

Cevap: `.cursor/mcp.json` dosyasına server tanımı eklerim, Cursor'u yeniden başlatırım.

- **--read-only flag'i ne işe yarar?**

Cevap: MCP'nin sadece okuma yapmasını sağlar. AI yanlışlıkla dosya değiştiremez. Güvenlik için ideal.

- **MCP olmadan AI ile çalışmak neden problemli olabilir?**

Cevap: AI eski API'lar üretir (NgModule, *ngIf, @Input decorator). Sen düzeltmek için zaman harcarsın, AI'nın yardımı negatif değer üretebilir.

⚠ Anti-Patterns

- ✗ MCP'siz AI ile Angular kodu üretmek — Eski kod alma riski yüksek
- ✗ AI çıktısını körü körüne kabul etmek — MCP olsa bile mutlaka kontrol et
- ✗ --read-only kullanmamak — AI yanlışlıkla dosya değiştirebilir
- ✗ MCP'yi workspace dışında çalıştırmak — Projeye özel context göremez
- ✗ Angular MCP varken eski tutorial'lara körü körüne uymak — MCP güncel öneriler verir

□ Bu Dersten Çıkarman Gerekenler

- ✓ MCP'nin ne olduğunu ve neden devrim olduğunu anlıyorum
- ✓ AI tool'larının eski bilgi problemini biliyorum
- ✓ Angular MCP Server'ı `ng mcp` ile başlatabiliyorum
- ✓ Cursor IDE'ye MCP entegrasyonu yapabiliyorum
- ✓ Claude Code (terminal) ile MCP kullanabiliyorum
- ✓ VS Code Copilot'a MCP ekleyebiliyorum
- ✓ `find_examples`, `get_best_practices` gibi MCP tool'larını biliyorum

✓ --read-only ile güvenli kullanım önemini biliyorum

MODÜL 3 — DERS 3.7

Mini Proje: Counter App

Modül 3'ün son dersi: ilk gerçek Angular projemizi yapacağız! Counter app — basit ama Angular'ın temel kavramlarını birleştiren bir uygulama.



Hedef

Bu mini projede Modül 3'te öğrendiğimiz tüm kavramları kullanacağız: ng generate, component anatomi, signals, event binding, computed, app.config.ts. Hepsi basit bir Counter'da bir araya gelecek.

1

Proje Tanımı

Counter App Özellikleri:

- ✓ +1, -1, +5, -5, Reset butonları
- ✓ Sayı negatifse kırmızı, pozitifse yeşil
- ✓ Step ayarı (1, 5, 10 ile artırma)
- ✓ "Çift sayı" / "Tek sayı" gösterimi
- ✓ Maksimum/minimum değer takibi
- ✓ İşlem geçmişi (son 5 değişiklik)

Bu proje basit gözüküyor ama içinde Angular'ın temellerinin %80'i var: signals, computed, event binding, conditional rendering, list rendering.

2

Adım 1: Yeni Proje Oluştur

```
# Yeni proje
ng new counter-app --style=scss --ssr=no

# Sorulara cevaplar:
# Stylesheet: SCSS
# SSR: No
# Zoneless: Yes
# Tailwind: No

cd counter-app
ng serve --open
```

Tarayıcı açılır, default Angular sayfası görünür. Şimdi App component'ini düzenleyeceğiz.

3

Adım 2: app.ts Component'i

```
// src/app/app.ts
import { Component, signal, computed } from '@angular/core';

interface HistoryEntry {
  oldValue: number;
  newValue: number;
  delta: number;
  timestamp: Date;
}

@Component({
  selector: 'app-root',
  templateUrl: './app.html',
  styleUrls: ['./app.scss']
})
export class App {
  // State signals
  count = signal(0);
  step = signal(1);
  history = signal<HistoryEntry[]>([]);

  // Computed values (otomatik türetilir)
  isPositive = computed(() => this.count() > 0);
  isNegative = computed(() => this.count() < 0);
  isZero = computed(() => this.count() === 0);
  isEven = computed(() => this.count() % 2 === 0);

  maxValue = computed(() => {
    const all = [this.count(), ...this.history().map(h => h.newValue)];
    return Math.max(...all);
  });

  minValue = computed(() => {
    const all = [this.count(), ...this.history().map(h => h.newValue)];
    return Math.min(...all);
  });

  // Action methods
  increment() {
    this.changeBy(this.step());
  }

  decrement() {
    this.changeBy(-this.step());
  }

  reset() {
```

```
    this.changeBy(-this.count());
  }

  setStep(newStep: number) {
    this.step.set(newStep);
  }

  // Helper
  private changeBy(delta: number) {
    if (delta === 0) return;

    const oldValue = this.count();
    const newValue = oldValue + delta;

    this.count.set(newValue);

    // History'ye ekle (max 5 entry)
    this.history.update(h => [
      { oldValue, newValue, delta, timestamp: new Date() },
      ...h
    ].slice(0, 5));
  }
}
```

Burada Modül 2'de öğrendiğimiz `signal()` ve `computed()` kullanıyoruz. Modül 5'te derinlemesine işleyeceğiz.

4

Adım 3: app.html Template'i

```
<!-- src/app/app.html -->
<main class="counter-container">
  <h1>Counter App</h1>

  <!-- Counter display -->
  <div class="counter-display"
    [class.positive]="isPositive()"
    [class.negative]="isNegative()"
    [class.zero]="isZero()">
    {{ count() }}
  </div>

  <!-- Status info -->
  <div class="counter-info">
    <p>{{ isEven() ? 'Çift sayı' : 'Tek sayı' }}</p>
    <p>En yüksek: {{ maxValue() }} | En düşük: {{ minValue() }}</p>
  </div>

  <!-- Step selector -->
  <div class="step-selector">
    <span>Step:</span>
    @for (s of [1, 5, 10]; track s) {
      <button
        [class.active]="step() === s"
        (click)="setStep(s)">
        {{ s }}
      </button>
    }
  </div>

  <!-- Action buttons -->
  <div class="actions">
    <button class="btn-decrement" (click)="decrement()">
      - {{ step() }}
    </button>
    <button class="btn-reset" (click)="reset()">
      Reset
    </button>
    <button class="btn-increment" (click)="increment()">
      + {{ step() }}
    </button>
  </div>

  <!-- History -->
  @if (history().length > 0) {
    <div class="history">
      <h3>Son İşlemler</h3>
```



```
    <ul>
      @for (entry of history(); track entry.timestamp) {
        <li>
          <span [class.positive]="entry.delta > 0" [class.negative]="
entry.delta < 0">
            {{ entry.delta > 0 ? '+' : '' }}{{ entry.delta }}
          </span>
          : {{ entry.oldValue }} → {{ entry.newValue }}
        </li>
      }
    </ul>
  </div>
}
</main>
```

Burada @for, @if (modern control flow), (click) (event binding), {{ }} (interpolation), [class.x] (class binding) kullandık. Modül 4'te detaylı işleyeceğiz.

5

Adım 4: app.scss Styling

```
/* src/app/app.scss */
.counter-container {
  max-width: 500px;
  margin: 40px auto;
  padding: 30px;
  font-family: 'Segoe UI', sans-serif;
  text-align: center;

  h1 {
    color: #dd0031;
    margin-bottom: 30px;
  }
}

.counter-display {
  font-size: 96px;
  font-weight: bold;
  margin: 20px 0;
  transition: color 0.3s;

  &.positive { color: #0d7a3e; }
  &.negative { color: #c0392b; }
  &.zero     { color: #888; }
}

.counter-info {
  background: #f5f5f5;
  padding: 12px;
  border-radius: 8px;
  margin: 20px 0;

  p {
    margin: 4px 0;
    color: #555;
  }
}

.step-selector {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 8px;
  margin: 20px 0;

  span { font-weight: bold; }

  button {
```

```
padding: 6px 16px;
border: 2px solid #ddd;
background: white;
border-radius: 6px;
cursor: pointer;

&.active {
  background: #dd0031;
  color: white;
  border-color: #dd0031;
}
}
}

.actions {
  display: flex;
  gap: 12px;
  justify-content: center;
  margin: 30px 0;

  button {
    flex: 1;
    padding: 12px 20px;
    font-size: 16px;
    font-weight: bold;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    transition: transform 0.1s, box-shadow 0.2s;

    &:hover { transform: translateY(-2px); }
    &.active { transform: translateY(0); }

    &.btn-increment {
      background: #0d7a3e;
      color: white;
    }
    &.btn-decrement {
      background: #c0392b;
      color: white;
    }
    &.btn-reset {
      background: #888;
      color: white;
    }
  }
}
```

```
.history {  
  margin-top: 30px;  
  text-align: left;  
  background: #fafafa;  
  padding: 16px;  
  border-radius: 8px;  
  
  h3 { color: #555; margin-top: 0; }  
  
  ul {  
    list-style: none;  
    padding: 0;  
  
    li {  
      padding: 6px 0;  
      border-bottom: 1px solid #eee;  
  
      &:last-child { border-bottom: none; }  
  
      .positive { color: #0d7a3e; font-weight: bold; }  
      .negative { color: #c0392b; font-weight: bold; }  
    }  
  }  
}
```

6 Adım 5: Test ve DevTools

Şimdi tarayıcıda dene:

- ✓ +/– butonları çalışıyor mu?
- ✓ Step değiştirince adım değeri değişiyor mu?
- ✓ Negatif sayıda kırmızı, pozitifte yeşil oluyor mu?
- ✓ History düzgün ekleniyor mu?
- ✓ Çift/tek sayı doğru gösteriliyor mu?
- ✓ Max/min güncelleniyor mu?

DevTools ile İncele

- F12 → Angular sekmesi aç
- App component'ini seç
- count signal'ini DevTools'tan değiştir → UI anında güncellenir
- Profiler ile increment yaparken render süresini ölç

7 Pekiştirme: Ne Öğrendin?

Konsept	Nerede Kullandık
ng new (M3.1)	Projeyi oluşturduk
Proje yapısı (M3.2)	src/app/ içinde çalıştık
ng serve (M3.3)	Dev server'ı başlattık
signal() (M2.2)	count, step, history
computed() (M2.2)	isPositive, isEven, maxValue
Component decorator (M2.3)	@Component({...})
Interface (M2.1)	HistoryEntry
Generic (M2.2)	signal([])
Modern control flow	@for, @if
Event binding	(click)="..."
Class binding	[class.positive]="..."
Interpolation	{{ count() }}
Component-scope d CSS (M3.2)	app.scss sadece bu component'i etkiliyor



Tebrikler!

İlk gerçek Angular projeni yazdın. Modül 3'te öğrendiğin her şey burada bir araya geldi. Bundan sonraki modüllerde bu temellerin üstüne inşa edeceğiz.

8

Kendi Pratiğin (Egzersizler)

Bu projeyi geliştirmeyi dene:

Egzersiz 1: Limit Ekle

```
// Counter'a min/max limit ekle:
minLimit = signal(-100);
maxLimit = signal(100);

// Limit aşılsa butonu disable et
canIncrement = computed(() => this.count() + this.step() <= this.maxLimit());
canDecrement = computed(() => this.count() - this.step() >= this.minLimit());
```

Egzersiz 2: LocalStorage Persist

Sayfa yenilenince count kaybolmasın:

```
// effect() kullan - signal değişikliklerini izle
import { effect } from '@angular/core';

constructor() {
  // Page load: localStorage'dan oku
  const saved = localStorage.getItem('counter');
  if (saved) this.count.set(+saved);

  // Effect: signal değişince kaydet
  effect(() => {
    localStorage.setItem('counter', this.count().toString());
  });
}
```

Egzersiz 3: Undo/Redo

History'den faydalanarak undo butonu yap:

```
undo() {
  const lastEntry = this.history()[0];
  if (!lastEntry) return;

  this.count.set(lastEntry.oldValue);
  this.history.update(h => h.slice(1));
}
```


Egzersiz 4: Multiple Counters

Birden fazla counter'ı yan yana göster. ng generate ile yeni component oluştur:

```
ng g c counter
# Sonra App'te:
# <app-counter />
# <app-counter />
# <app-counter />
```

Egzersiz 5: Animation

Sayı değişince animasyon ekle (CSS transition):

```
.counter-display {
  transition: transform 0.2s, color 0.3s;

  &:hover {
    transform: scale(1.05);
  }
}
```

□ Mini Quiz

- **signal() ve computed() arasındaki fark nedir?**

Cevap: `signal()` değiştirilebilir state. `computed()` bu state'lerden türetilen ve otomatik güncellenen değer. `computed`'da `set/update` yok.

- **Template'te `{{ count() }}` neden parantezli?**

Cevap: `count` bir `Signal`. Değerini almak için fonksiyon gibi çağrılır: `count()`. React'taki `state.value` gibi.

- **`[class.positive]="isPositive()"` ne işe yarar?**

Cevap: `Class binding`. `isPositive()` `true` dönerse `element`'e `"positive"` class'ı ekler, `false` dönerse kaldırır.

- **`@for` ve `track` neden ayrı yazılır?**

Cevap: `track` Angular'ın hangi item'ın hangisi olduğunu bilmesi için `unique identifier` sağlar. `Performance` ve doğru `DOM update` için kritik.

- **`history.update(h => [...])` yerine `history.set([...])` yazsam ne fark eder?**

Cevap: `update` mevcut değeri alır, yeni değer döndürür. `set` direkt yeni değeri yazar. `update functional pattern`, daha temiz.

- **App component'inde `history`'yi sınırlandırırken neden `.slice(0, 5)` yaptık?**

Cevap: `History`'nin sonsuz büyümesini önlemek için. Sadece son 5 işlemi tutmak. `Memory` ve `UI performansı` için önemli.

☐ Modül 3 Tamamlandı!

Tebrikler! Modül 3'ü başarıyla tamamladın. Artık Angular ile gerçek kod yazıyorsun. Counter app'in bunun en güzel kanıtı.

Neler Öğrendin?

Ders 3.1 — ng new ile Proje Oluşturma

CLI'nin sorduğu her sorunun anlamını, SSR/Zoneless/Tailwind seçimlerini, yaygın hata çözümlerini öğrendik.

Ders 3.2 — Proje Yapısı Anatomisi

src/, angular.json, package.json, tsconfig — her dosyanın amacını ve ne zaman dokunulacağını gördük.

Ders 3.3 — Angular CLI Komutları

ng generate, ng serve, ng build, ng update, ng add — günlük workflow'umuzun temel komutlarını öğrendik.

Ders 3.4 — app.config.ts & Bootstrapping

Modern Angular'ın bootstrap akışını, provideX() fonksiyonlarını, custom provider yazmayı kavradık.

Ders 3.5 — Angular DevTools

Component Explorer, Profiler, Router Tree, Injector Tree ile debug ve performance analizi yapabiliyoruz.

Ders 3.6 — MCP Server (v21 Yeniliği)

AI tool'larının eski bilgi problemini, Angular MCP Server'ı, Cursor/Claude Code/VS Code entegrasyonunu öğrendik.

Ders 3.7 — Mini Proje: Counter App

İlk gerçek Angular projemizi yazdık. Signal, computed, event binding, control flow — hepsi bir arada.



Sıradaki Modül: Component'lar & Modern Template Syntax

Modül 4 — Component'lara derinlemesine ineceğiz! Anatomi, standalone components, data binding 4 çeşidi, modern control flow (@if/@for/@switch), @let, input()/output()/model(), view encapsulation. Modül 3'te dokunduğumuz konuların hepsini detaylı işleyeceğiz.

Modül 4'e Hoş Geldin

Bu modül Angular'ın kalbi: Component'lar. Modül 3'te yüzeysel olarak dokunduğumuz her şeyi şimdi derinlemesine işleyeceğiz. Component anatomy, standalone components, data binding'in 4 çeşidi, modern control flow (@if/@for/@switch), @let template değişkenleri ve modern input()/output()/model() API'ları. Ve sonunda Angular Todo App mini projesini yapacağız.

Bu modülün sonunda:

- ✓ Component anatomy'sini derinlemesine kavrayacaksın
- ✓ Standalone components ile NgModule'siz dünyada çalışabileceksin
- ✓ 4 data binding çeşidini ustalıkla kullanabileceksin
- ✓ Modern control flow (@if/@for/@switch) ile çalışabileceksin
- ✓ input(), output(), model() ile component'ler arası iletişim kurabileceksin
- ✓ Çok component'li gerçek bir Angular uygulaması yapmış olacaksın



Önemli Modül

Bu modül Angular'ın en kritik modülü. Buradan sonraki tüm modüller bu temellerin üstüne inşa edilecek. Bu modülü iyi anladığından emin ol — özellikle modern syntax'ı (input(), output(), @if, @for, signals).

MODÜL 4 — DERS 4.1

Component Anatomisi

Angular'ın temel yapı taşı: Component. Her sayfa, her UI parçası bir component. Bu derste component'in anatomisini derinlemesine inceleyeceğiz.



Hedef

Modül 3'te component'lere dokunduk. Şimdi "Her parçası ne işe yarıyor?", "Hangi seçenekler var?", "Best practice'ler ne?" sorularına derinlemesine cevap vereceğiz.

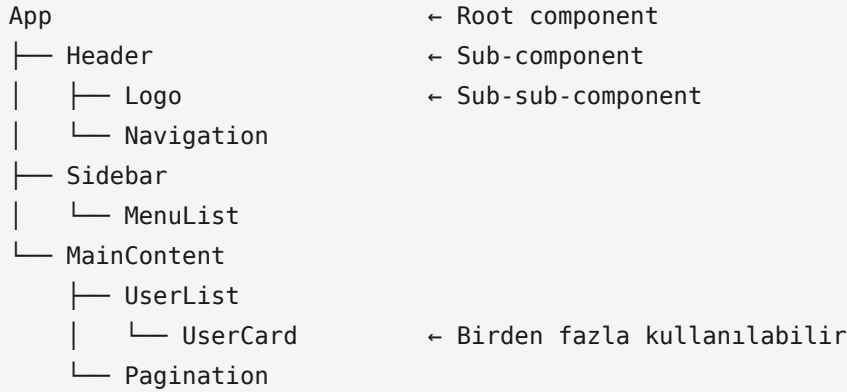
1

Component Nedir?

Component: Angular'ın temel yapı taşı. Bir UI parçasını temsil eder.

Mental Model: Component = Lego Parçası

Bir Angular uygulaması, iç içe geçmiş Lego parçaları gibidir. Her parça (component) kendi başına bir şey yapar, ama birleştirildiklerinde büyük resmi oluştururlar.



Component'in Üç Parçası

Her component üç şeyden oluşur:

Parça	Ne İçerir?	Dosya Uzantısı
Class (TypeScript)	State (signals), behavior (methods), DI	.ts
Template (HTML)	Görsel yapı, data binding, control flow	.html veya inline
Styles (CSS/SCSS)	Görünüm, layout, animation	.scss veya inline

2 Component'in Tam Yapısı

Tam donanımlı bir component örneği:

```
// user-card.ts
import {
  Component,
  ChangeDetectionStrategy,
  input,
  output,
  signal,
  computed,
} from '@angular/core';

export interface User {
  id: number;
  name: string;
  email: string;
  isActive: boolean;
}

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.html',
  styleUrls: ['./user-card.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
  host: {
    '[class.inactive]': '!user().isActive',
    '(click)': 'onCardClick()',
  }
})
export class UserCardComponent {
  // === Inputs (component'e dışarıdan gelen) ===
  user = input.required<User>();
  showEmail = input(true);

  // === Outputs (component'ten dışarıya event) ===
  selected = output<User>();

  // === Local state ===
  isExpanded = signal(false);

  // === Computed (türetilmiş değer) ===
  displayName = computed(() => {
    const u = this.user();
    return u.isActive ? u.name : `${u.name} (pasif)`;
  });

  // === Methods (davranış) ===
  toggleExpand() {
    this.isExpanded.update(v => !v);
  }
}
```



```
    }  
  
    onCardClick() {  
        this.selected.emit(this.user());  
    }  
}
```

3 @Component Decorator Anatomisi

Modül 2.3'te decorator'lara kavramsal olarak baktık. Şimdi @Component'in her ayarına derinlemesine bakalım.

selector

HTML'de bu component'i hangi tag ile kullanacağımızı belirler:

```
@Component({
  selector: 'app-user-card', // Konvansiyon: app-xxx prefix
})

// Kullanımı:
<app-user-card [user]="currentUser" />
```

Selector Türü	Örnek	Açıklama
Element (default)	app-user-card	HTML tag olarak
Attribute	[appHighlight]	Attribute olarak
Class	.app-card	CSS class olarak
Çoklu	app-card, [appCard]	Birden fazla şekilde

template / templateUrl

```
// Inline template (küçük component'ler için)
@Component({
  template: '<h1>{{ title() }}</h1>'
})

// Template literal (multiline, syntax highlighting destekli)
@Component({
  template: `
    <h1>{{ title() }}</h1>
    <p>Welcome!</p>
  `
})

// Ayrı dosya (büyük component'ler için – önerilen)
@Component({
  templateUrl: './user-card.html'
})
```



Best Practice

Template 10 satırdan kısa ise inline, uzunsa templateUrl kullan. Karışıklık olmasın diye yarı yarıya kullanım yapma.

styles / styleUrls / styleUrls

```
// Tek dosya (modern – önerilen)
@Component({
  styleUrls: ['./user-card.scss']
})

// Birden fazla dosya
@Component({
  styleUrls: ['./user-card.scss', './shared.scss']
})

// Inline (küçük style'lar için)
@Component({
  styles: [`
    h1 { color: red; }
    p { font-size: 14px; }
  `]
})
```

changeDetection

```
@Component({  
  changeDetection: ChangeDetectionStrategy.OnPush,  
})
```

OnPush: Change detection'ı optimize eder. Sadece input değişimi, event veya signal değişiminde çalışır. Performans için kritik.



Modern Angular

Zoneless modunda (Modül 1.4) OnPush default davranış. Yeni component'lerde her zaman OnPush kullanın.

host

Component'in kendi root element'ine attribute/class/event ekler:

```
@Component({  
  selector: 'app-user-card',  
  host: {  
    '[class.active]': 'isActive()', // class binding  
    '[class.inactive]': '!isActive()',  
    '[attr.role]': '"button"', // attribute  
    '(click)': 'onClick()', // event listener  
    '(mouseenter)': 'onHover()',  
    'class': 'card', // static class  
  }  
})  
  
// HTML'de:  
<app-user-card class="active card" role="button"></app-user-card>
```

Modern Angular: @HostBinding ve @HostListener decorator'ları yerine bu host object kullanılır.

imports

```
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ButtonComponent } from '../button/button.component';

@Component({
  selector: 'app-user-card',
  imports: [
    CommonModule,          // ngIf, ngFor (eski - artık @if/@for var)
    FormsModule,           // ngModel
    ButtonComponent,       // Custom component
  ],
})
```

Standalone components'ın kalbi. Bu component'in hangi başka şeyleri kullandığını burada belirtiyorsun. Modül 4.2'de detaylı işleyeceğiz.

4 Naming Conventions — İsimlendirme Kuralları

Angular topluluğunun standartları:

Eleman	Konvansiyon	Örnek
Component dosya	kebab-case	user-card.ts
Class name	PascalCase + suffix	UserCardComponent
Selector	kebab-case + prefix	app-user-card
Method	camelCase	onUserClick()
Property	camelCase	isLoading
Constant	UPPER_SNAKE_CASE	MAX_ITEMS
Interface	PascalCase	User, ButtonProps
Type alias	PascalCase	Status, FilterMode

Suffix Konvansiyonu (Önemli)

Tip	Suffix	Örnek
Component	Component	UserCardComponent
Service	Service	UserService
Directive	Directive	HighlightDirective
Pipe	Pipe	CapitalizePipe
Guard	Guard	AuthGuard
Interceptor	Interceptor	AuthInterceptor
Resolver	Resolver	UserResolver

i Modern Angular Trendi

Angular v20+'da CLI varsayılan olarak suffix'siz isimler üretiyor (örn. UserCard, UserService yerine sadece Users). Ama Component, Service suffix'leri hala yaygın ve tavsiye ediliyor — okunabilirlik için. Bu eğitimde suffix kullanacağız.

5

Lifecycle Hooks — Component Yaşam Döngüsü

Bir component'in yaşam döngüsü vardır: yaratılır, çalışır, yok edilir. Belirli noktalarda kod çalıştırmak için hook'lar kullanılır.

Hook	Ne Zaman?	Sık Kullanım
ngOnInit	Component yaratıldıktan sonra	İlk veri yükleme
ngOnDestroy	Component yok edilmeden önce	Cleanup, unsubscribe
ngAfterViewInit	View render edildikten sonra	DOM erişimi
ngOnChanges	Input değiştiğinde (eski)	Modern: input + effect

Modern Yaklaşım: constructor + inject + effect

```
export class UserCardComponent {
  private userService = inject(UserService);

  user = input.required<User>();

  // ngOnInit yerine constructor + effect
  constructor() {
    // input değişimini izle (ngOnChanges yerine)
    effect(() => {
      const u = this.user();
      console.log('User changed:', u);
    });
  }

  // Hala bazı durumlarda gerekli:
  ngOnDestroy() {
    // Cleanup
  }
}
```



Modern Trend

Modern Angular'da çoğu hook gerekli değil. Signal'ler ve effect() ile aynı işi daha temiz yapıyoruz. Modül 5'te detaylı işleyeceğiz.

□ Mini Quiz

- **Bir component'in üç temel parçası nedir?**

Cevap: Class (TypeScript - state ve behavior), Template (HTML - görsel yapı), Styles (CSS/SCSS - görünüm). Bu üçü birleşip component'i oluşturur.

- **selector neden kebab-case ve prefix'li (app-xxx) olur?**

Cevap: kebab-case: HTML tag'leri büyük-küçük harfe duyarsız. Prefix (app-): kendi component'lerimizi başka library'lerinkilerden ayırmak için (çakışmayı önler).

- **templateUrl ve template arasındaki fark nedir?**

Cevap: templateUrl ayrı bir HTML dosyasını işaret eder, template inline string. Küçük component'ler için inline, büyük olanlar için templateUrl tercih edilir.

- **changeDetection: OnPush ne işe yarar?**

Cevap: Change detection'ı optimize eder. Sadece input/event/signal değişiminde çalışır, sürekli kontrol etmez. Performans için kritik. Zoneless'da default.

- **host: {} ne için kullanılır?**

Cevap: Component'in root element'ine class, attribute, event listener eklemek için. @HostBinding ve @HostListener decorator'larının modern alternatifi.

- **imports array'i ne işe yarar?**

Cevap: Standalone component'lerde, bu component'in hangi başka component, directive, pipe veya module'lere ihtiyaç duyduğunu belirtir. NgModule'ün modern alternatifi.

⚠ Anti-Patterns

- ✗ changeDetection belirtmemek — Modern projede her zaman OnPush kullan
- ✗ İsimlendirme tutarsız — UserCard.ts/userCard.ts karışıklığı, hep kebab-case
- ✗ Prefix kullanmamak — selector çakışması olabilir, hep app-xxx
- ✗ 500+ satırlık component — Parçala, daha küçük component'lere böl
- ✗ @HostBinding kullanmak — host: {} kullan, modern syntax
- ✗ Constructor'da iş yapmak — Lifecycle hook'ları kullan veya inject() ile

□ Bu Dersten Çıkarman Gerekenler

- ✓ Component'in üç parçasını (class, template, styles) biliyorum
- ✓ @Component decorator'ının tüm ayarlarını anlıyorum
- ✓ selector, template, styles seçeneklerini doğru kullanabiliyorum
- ✓ changeDetection: OnPush'ı her zaman kullanmam gerektiğini biliyorum
- ✓ host: {} ile root element'e bağlantı yapabiliyorum
- ✓ Naming convention'larını (kebab-case dosya, PascalCase class) içselleştirdim

- ✓ Lifecycle hook'larının modern alternatiflerini biliyorum

MODÜL 4 — DERS 4.2

Standalone Components

*Eski NgModule dünyasından modern standalone dünyasına.
Component'ler artık self-contained — kendi bağımlılıklarını kendileri belirtiyor.*



Hedef

Standalone component'lerin neden var olduğunu, NgModule'den farkını, nasıl kullanıldığını ve nasıl kompoze edildiğini öğreneceğiz.

1

Hatırlatma: NgModule Problemi

Modül 3.4'te app.config.ts'i öğrenirken NgModule'den bahsetmiştik. Hatırla:

```
// ESKİ DÜNYA (Angular 14 öncesi)
@NgModule({
  declarations: [
    AppComponent,
    UserCardComponent,
    UserListComponent,
    // ... her component burada listelenmek ZORUNDA
  ],
  imports: [
    BrowserModule,
    CommonModule,
    FormsModule,
    HttpClientModule,
  ],
  providers: [UserService],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Problemler:

- ✗ Her yeni component'i NgModule'e manuel ekleme zorunluluğu
- ✗ Lazy loading karmaşık — her route için ayrı module
- ✗ Tree-shaking suboptimal — kullanılmayan kod build'e girebilir
- ✗ Yeni başlayanlar için karmaşık — "Neden bu kadar boilerplate?"

2

Standalone Components Çözümü

Fikir: Her component kendi başına yeterli, kendi bağımlılıklarını kendisi belirtir. NgModule'e ihtiyaç yok.

```
@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.html',
  imports: [
    // Bu component ne kullanıyor?
    CommonModule,
    UserAvatarComponent,
    DateFormatPipe,
  ],
})
export class UserCardComponent { }
```

Component artık "self-contained". Başka bir yerde register etmeye gerek yok.

Tarihsel Bağlam

Versiyon	Durum
Angular 14	Standalone öne sürüldü (opsiyonel)
Angular 15	Standalone stabil hale geldi
Angular 16	Çoğu yeni proje standalone
Angular 17	"Standalone by default" — yeni CLI standalone üretir
Angular 19+	standalone: true gereksiz (default)
Angular 21	NgModule artık sadece migration için

3 NgModule vs Standalone — Yan Yana

Eski Yol (NgModule)

```
// 1. component
@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.html',
})
export class UserCardComponent { }

// 2. AYRI bir module
@NgModule({
  declarations: [UserCardComponent],
  imports: [CommonModule, FormsModule],
  exports: [UserCardComponent], // Dışarıdan kullanım için
})
export class UserModule { }

// 3. AppModule'a ekle
@NgModule({
  imports: [UserModule, ...],
})
export class AppModule { }
```

3 yerde değişiklik. Yorucu, hata kaynağı.

Modern Yol (Standalone)

```
// SADECE component dosyası
@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.html',
  imports: [CommonModule, FormsModule], // ← Buraya bağımlılıklar
})
export class UserCardComponent { }

// Başka yerde register etmeye GEREK YOK
// Kullanan component'ler imports'una ekler
```

Avantajlar

- ✓ Daha az boilerplate — NgModule yazmak yok
- ✓ Self-contained — Component'i kopyala/taşı, çalışır
- ✓ Better tree-shaking — Kullanılmayan kod build'e girmez

- ✓ Daha kolay lazy loading — Sadece component'i import et
- ✓ IDE autocomplete daha iyi

4 Component Composition — Component'leri Birleştirme

Standalone component'ler birbirlerini imports array'i ile kullanır.

Senaryo: **UserList**, **UserCard**'ları gösteriyor

```
// user-card.ts
@Component({
  selector: 'app-user-card',
  template: `<div>{{ user().name }}</div>`,
})
export class UserCardComponent {
  user = input.required<User>();
}

// user-list.ts
import { UserCardComponent } from './user-card';

@Component({
  selector: 'app-user-list',
  imports: [UserCardComponent], // ← Burada import et
  template: `
    @for (user of users(); track user.id) {
      <app-user-card [user]="user" />
    }
  `,
})
export class UserListComponent {
  users = input.required<User[]>();
}
```

UserList, UserCard'ı kullanabilmek için imports'una ekliyor. Bu kadar basit.

5 imports Array'ine Ne Konabilir?

Tip	Örnek	Ne Sağlar?
Standalone component	UserCardComponent	
Standalone directive	HighlightDirective	[appHighlight]
Standalone pipe	CapitalizePipe	{{ x capitalize }}
NgModule (legacy)	CommonModule	@if, @for (eski)
NgModule (legacy)	FormsModule	ngModel, formGroup
NgModule (legacy)	RouterModule	routerLink, router-outlet

Önemli Bir Detay: CommonModule Artık Gerekli Değil

```
// ESKİ
@Component({
  imports: [CommonModule], // *ngIf, *ngFor için
  template: `
    <div *ngIf="user">{{ user.name }}</div>
    <div *ngFor="let u of users">{{ u.name }}</div>
  `,
})

// MODERN (v17+)
@Component({
  imports: [], // Boş! Modern control flow built-in
  template: `
    @if (user()); as u { <div>{{ u.name }}</div> }
    @for (u of users(); track u.id) { <div>{{ u.name }}</div> }
  `,
})
```



Modern Angular

Modern control flow (@if, @for, @switch) built-in geldiği için CommonModule import etmene gerek yok. Modül 4.4'te detaylı işleyeceğiz.

6 Standalone ile Bootstrap

Modül 3.4'te gördük ama tekrar:

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { App } from './app/app';
import { appConfig } from './app/app.config';

bootstrapApplication(App, appConfig);

// App, standalone bir component. Direkt bootstrap edilebilir.
// AppModule'a gerek yok.
```

Lazy Loading ile Routes

```
// app.routes.ts
export const routes: Routes = [
  {
    path: '',
    loadChildren: () => import('./home/home').then(m => m.HomeComponent),
  },
  {
    path: 'users',
    loadChildren: () => import('./users/user-list').then(m => m.UserListComponent),
  },
];
```

loadComponent: Sadece o route'a gidildiğinde component yüklenir. Bundle'ı küçük tutar.

7

Eski Projeyi Migrate Etmek

NgModule kullanan eski bir projen varsa, Angular CLI bunu otomatik dönüştürür:

```
# Tüm component'leri standalone'a çevir
ng generate @angular/core:standalone

# Hangisi?
? Choose the type of migration:
> Convert all components, directives and pipes to standalone
  Remove unnecessary NgModule classes
  Bootstrap the application using standalone APIs
```

Bu komut binlerce satır kodu otomatik dönüştürür. Manuel yapmak çılgınlık olur.

□ Mini Quiz

- **Standalone component nedir?**

Cevap: NgModule'e ihtiyaç duymayan, kendi bağımlılıklarını imports array'i ile kendisi belirten component. Self-contained ve daha az boilerplate.

- **Bir standalone component başka bir standalone component'i nasıl kullanır?**

Cevap: imports array'ine ekleyerek. Örn: imports: [UserCardComponent]. Sonra template'te kullanılabilir.

- **Modern Angular'da CommonModule gerekli mi?**

*Cevap: Hayır. Modern control flow (@if, @for, @switch) built-in. CommonModule sadece eski *ngIf, *ngFor, ngClass için gerekiyordu.*

- **Lazy loading bir standalone component için nasıl yapılır?**

Cevap: Route'da loadComponent: () => import(...).then(m => m.SomeComponent) ile. NgModule'lü lazy loading'den çok daha basit.

- **Eski NgModule projesi nasıl standalone'a dönüştürülür?**

Cevap: ng generate @angular/core:standalone komutu ile. Angular CLI otomatik dönüştürür.

- **v19+'da standalone: true yazmak gerekli mi?**

Cevap: Hayır. v19+'da default davranış standalone. standalone: true yazılması gerekmiyor (geriye uyumluluk için hala çalışır).

⚠ Anti-Patterns

- ✗ Yeni projede NgModule kullanmak — Standalone tercih et
- ✗ imports'a CommonModule eklemek — Modern syntax için gereksiz
- ✗ Her component için ayrı module yazmak — Standalone yeterli
- ✗ Component'i kullanmadan önce imports'a eklemeyi unutmak — IDE uyarır ama dikkat
- ✗ Eski projeyi manuel migrate etmek — ng generate @angular/core:standalone kullan

□ Bu Dersten Çıkarman Gerekenler

- ✓ Standalone component'in ne olduğunu ve neden var olduğunu anlıyorum
- ✓ NgModule ile farkı net biliyorum
- ✓ imports array'ini doğru kullanabiliyorum
- ✓ Component composition yapabiliyorum
- ✓ Standalone ile lazy loading yazabiliyorum
- ✓ CommonModule'ün modern Angular'da gerekli olmadığını biliyorum

- ✓ Eski projeyi migrate etme komutunu biliyorum

MODÜL 4 — DERS 4.3

Data Binding 4 Çeşidi

Angular'ın "büyüsünün" geldiği yer: data binding. Component ile template arasında veri akışı. 4 çeşidini detaylı öğreneceğiz.



Hedef

Component'teki veri template'e nasıl gider? Template'teki event component'e nasıl ulaşır? Bu derste data flow'un her yönünü öğreneceğiz.

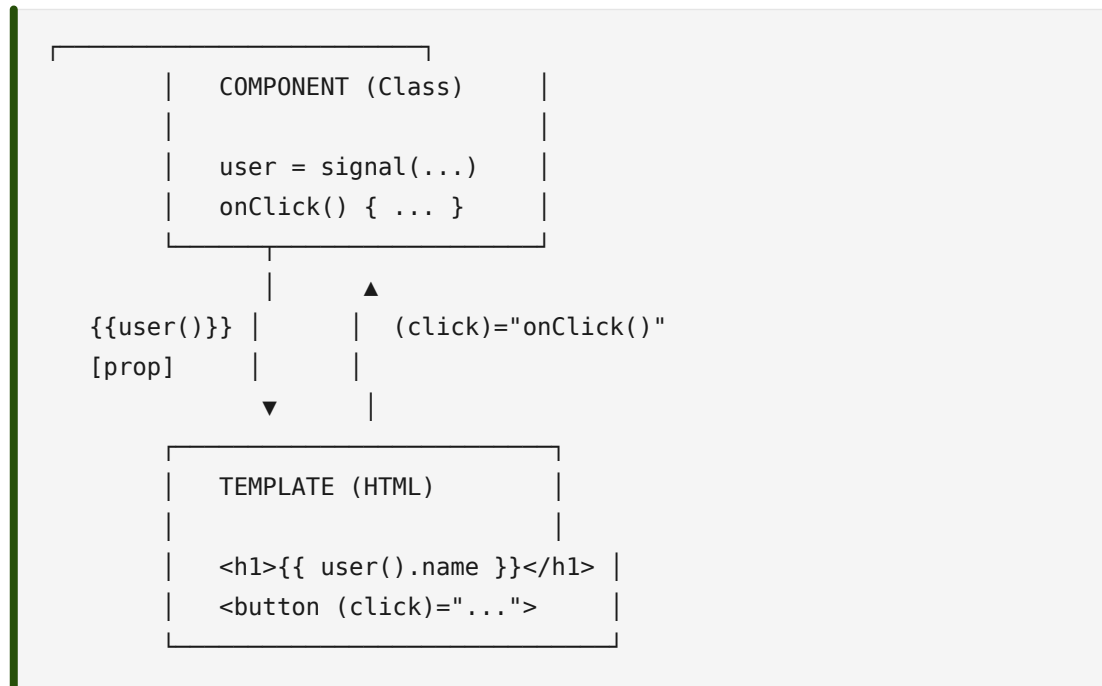
1

Data Binding Genel Bakış

Angular'da 4 çeşit data binding var:

Çeşit	Yön	Syntax	Kullanım
Interpolation	Class → Template	{{ value }}	Text gösterme
Property Binding	Class → Template	[prop]="value"	Element attribute'u
Event Binding	Template → Class	(event)="handler()"	Kullanıcı etkileşimi
Two-way Binding	Her iki yön	[(ngModel)]="value"	Form input'ları

Mental Model — Veri Akış Yönleri



2 1. Interpolation — {{ value }}

En basit ve en sık kullanılan binding. Component'teki değeri template'e text olarak bastırır.

```

// Component
export class UserCardComponent {
  name = signal('John');
  age = signal(30);
  email = signal('john@example.com');
}

// Template
<h1>{{ name() }}</h1>
<p>Age: {{ age() }}</p>
<p>Email: {{ email() }}</p>

// Render edilmiş HTML
<h1>John</h1>
<p>Age: 30</p>
<p>Email: john@example.com</p>
  
```

Önemli: Signal'i fonksiyon gibi çağırıyoruz: `name()`. Parantezsiz olursa `signal` object'ini gösterir.

Expression'lar Çalıştırılabilir

```

<p>{{ name().toUpperCase() }}</p>           → JOHN
<p>{{ age() + 5 }}</p>                       → 35
<p>{{ 'Hello, ' + name() }}</p>             → Hello, John
<p>{{ age() > 18 ? 'Adult' : 'Minor' }}</p> → Adult
<p>{{ items().length }}</p>                 → 3

```



Dikkat

Template'te side effect yapan kod yazma! Çağrılan her seferde işlem yapacak: `{{ saveToDb() }}` ÇOK kötü.

Computed Tercihinde Bulun

```

// ❌ Template'te kompleks işlem
<p>{{ users().filter(u => u.active).map(u => u.name).join(', ') }}</p>

// ✓ Computed kullan
export class UserListComponent {
  users = signal<User[]>([...]);
  activeUserNames = computed(() =>
    this.users()
      .filter(u => u.active)
      .map(u => u.name)
      .join(', ')
  );
}

<p>{{ activeUserNames() }}</p>

```

3 2. Property Binding — [prop]="value"

HTML element'inin property'sini component'teki değere bağlar. Köşeli parantez içinde.

```
// Component
export class UserCardComponent {
  avatarUrl = signal('/images/john.jpg');
  isDisabled = signal(false);
  user = signal({ name: 'John', age: 30 });
}

// Template
<img [src]="avatarUrl()" />           ← src attribute
<button [disabled]="isDisabled()">Click</button>
<app-detail [user]="user()" />       ← Custom component'e veri
```

Property Binding vs Interpolation

```
<!-- Bu ikisi YAKLAŞIK aynı -->
    ← String concatenation
<img [src]="avatarUrl()" />       ← Direct property binding

<!-- AMA farkları var: -->
<!-- Interpolation HER ZAMAN string -->
<button disabled="{{ isDisabled() }}"> <!-- "true" veya "false" string
! -->

<!-- Property binding GERÇEK type -->
<button [disabled]="isDisabled()">    <!-- gerçek boolean -->
```

Best Practice

Boolean veya object property için her zaman [prop] kullan. String için ikisi de OK ama [prop] daha açık.

Özel Property Binding'ler

```
<!-- Class binding -->
<div [class.active]="isActive()">                ← Tek class
<div [class]="'card highlighted'">              ← Multiple classes (string)
<div [class]="{ active: isActive(), big: isBig() }"> ← Object form

<!-- Style binding -->
<div [style.color]="textColor()">                ← Tek property
<div [style.fontSize.px]="fontSize()">           ← Birim ile
<div [style]="{ color: 'red', fontSize: '16px' }">

<!-- Attribute binding -->
<button [attr.aria-label]="label()">            ← HTML attribute
<input [attr.data-id]="userId()">
```

Önemli ayrım: `[disabled]` property binding (DOM property), `[attr.disabled]` attribute binding (HTML attribute). Çoğunlukla property kullanırsın.

4

3. Event Binding — (event)="handler()"

Kullanıcı etkileşimlerini yakalama. Click, input, submit, vs. Parantez içinde.

```
// Component
export class CounterComponent {
  count = signal(0);

  increment() {
    this.count.update(c => c + 1);
  }

  reset() {
    this.count.set(0);
  }
}

// Template
<button (click)="increment()">+1</button>
<button (click)="reset()">Reset</button>
<p>Count: {{ count() }}</p>
```

\$event Parametresi

```
// Component
onInput(event: Event) {
  const target = event.target as HTMLInputElement;
  console.log('Value:', target.value);
}

onKey(event: KeyboardEvent) {
  if (event.key === 'Enter') {
    this.submit();
  }
}

// Template
<input (input)="onInput($event)">
<input (keydown)="onKey($event)">
```

Yaygın Event'ler

Event	Ne Zaman?
(click)	Element'e tıklandığında
(dblclick)	Çift tıklandığında
(input)	Input değiştiğinde (her karakter)
(change)	Input "commit" edildiğinde (blur)
(submit)	Form submit edildiğinde
(keydown)	Klavye basıldığında
(keyup)	Klavye bırakıldığında
(focus)	Focus alındığında
(blur)	Focus kaybedildiğinde
(mouseenter)	Mouse element'in üzerine geldiğinde
(mouseleave)	Mouse element'i bıraktığında

Key Modifier'lar

```
<!-- Spesifik tuş -->
<input (keydown.enter)="submit()">
<input (keydown.escape)="cancel()">
<input (keydown.tab)="onTab()">

<!-- Kombinasyon -->
<input (keydown.control.s)="save($event)">
<input (keydown.shift.enter)="newLine()">
```


5 4. Two-way Binding — [(value)]="..."

Banana in a box [()] — property + event bir arada. Hem oku hem yaz.

Klasik Kullanım: ngModel

```
import { FormsModule } from '@angular/forms';

@Component({
  imports: [FormsModule], // ← ngModel için gerekli
  template: `
    <input [(ngModel)]="name" />
    <p>Hello, {{ name }}!</p>
  `,
})
export class GreetingComponent {
  name = '';
}

// Kullanıcı yazdıkça <p> otomatik güncellenir
// İki yönlü senkronizasyon
```

Aslında Ne Yapıyor?

```
<!-- Bu... -->
<input [(ngModel)]="name" />

<!-- ...aslında bu demek: -->
<input
  [ngModel]="name"
  (ngModelChange)="name = $event"
/>

<!-- Yani [property] + (event) birleşimi -->
```

Modern Angular: model() Signal

```
// Custom component'te two-way için
@Component({
  selector: 'app-counter-input',
  template: `
    <button (click)="decrement()">-</button>
    <input [value]="value()" (input)="onInput($event)" />
    <button (click)="increment()">+</button>
  `,
})
export class CounterInputComponent {
  // model() ile two-way input
  value = model.required<number>();

  increment() {
    this.value.update(v => v + 1);
  }

  decrement() {
    this.value.update(v => v - 1);
  }

  onInput(event: Event) {
    const target = event.target as HTMLInputElement;
    this.value.set(+target.value);
  }
}

// Kullanım
<app-counter-input [(value)]="count" />
```



Modül 4.6'da

`model()`'i derinlemesine işleyeceğiz. Modern Angular'da custom two-way binding bu şekilde yapılır.

6 Pratik Örnek — Hepsi Bir Arada

Bir form component'inde tüm binding türleri:

```

@Component({
  selector: 'app-user-form',
  imports: [FormsModule],
  template: `
    <div class="form" [class.invalid]="!isValid()">

      <!-- 1. Interpolation -->
      <h2>{{ formTitle() }}</h2>

      <!-- 2. Property Binding -->
      <input
        type="text"
        [value]="name()"
        [disabled]="isLoading()"
        [class.error]="hasNameError()"
      />

      <!-- 3. Event Binding -->
      <input
        (input)="updateName($event)"
        (focus)="onFocus()"
        (blur)="validateName()"
      />

      <!-- 4. Two-way Binding -->
      <input [(ngModel)]="email" />

      <!-- Hepsini birleştirme -->
      <button
        [disabled]="!isValid()"
        [class.primary]="isValid()"
        (click)="submit()"
      >
        {{ isLoading() ? 'Saving...' : 'Save' }}
      </button>
    </div>
  `,
})
export class UserFormComponent {
  formTitle = signal('Edit Profile');
  name = signal('');
  email = ''; // Plain (two-way için)
  isLoading = signal(false);

  hasNameError = computed(() => this.name().length < 3);
  isValid = computed(() => !this.hasNameError() && this.email.length > 0);
}

```

```
updateName(event: Event) {  
  const target = event.target as HTMLInputElement;  
  this.name.set(target.value);  
}  
  
onFocus() { /* ... */ }  
validateName() { /* ... */ }  
  
submit() {  
  this.isLoading.set(true);  
  // ...  
}  
}
```

□ Mini Quiz

- **Interpolation ile property binding arasındaki fark nedir?**

Cevap: Interpolation (`{{ }}`) her zaman string sonuç verir. Property binding (`[prop]`) gerçek tipi korur. Boolean, object için `[prop]` kullan.

- **(click)="onClick()" syntax'ında parantez ne anlama gelir?**

Cevap: Event binding — template'ten component'e doğru veri akışı. Kullanıcı etkileşimini yakalamak için.

- **[(ngModel)] neden "banana in a box" diye anılır?**

Cevap: [()] görüntüsü bir kutu içindeki muzaya benziyor. Anlamı: [property] + (event) iki yönlü binding.

- **Template'te `{{ user().name }}` yazarken neden `user()` (parantezli)?**

Cevap: user bir signal. Signal'in değerini almak için fonksiyon gibi çağrılır. user (parantezsiz) signal object'ini gösterir.

- **[disabled]="isDisabled()" ile disabled="{{ isDisabled() }}" aynı şey mi?**

Cevap: Hayır. İlki gerçek boolean, ikincisi "true"/"false" string. HTML attribute'lar string olduğu için ikincisi her zaman truthy davranır.

- **Event handler'da \$event nedir?**

Cevap: Event'in kendisi (KeyboardEvent, MouseEvent, vb.). target.value gibi bilgilere erişmek için kullanılır.

⚠ Anti-Patterns

- ✗ Template'te kompleks expression — computed signal kullan
- ✗ Template'te function call (parantezli, side effect) — her render'da çalışır
- ✗ Boolean için interpolation — `[prop]` kullan, string'e çevrilmesin
- ✗ \$event'i any tipinde kullanmak — Event, KeyboardEvent gibi cast et
- ✗ Two-way binding'i her yere koymak — Form'larda OK, başka yerde dikkatli
- ✗ (click) ile (submit) karıştırmak — Button submit için (submit) form'da olur

□ Bu Dersten Çıkarman Gerekenler

- ✓ 4 binding çeşidini biliyorum: interpolation, property, event, two-way
- ✓ Her birinin syntax'ını ve yönünü anlıyorum
- ✓ `{{ value }}` ile `[prop]="value"` farkını biliyorum
- ✓ (click), (input), (submit) gibi event'leri kullanabiliyorum
- ✓ \$event ile event objesinden veri alabiliyorum
- ✓ [class.x], [style.x], [attr.x] özel binding'lerini biliyorum
- ✓ [(ngModel)] ile two-way binding yapabiliyorum

MODÜL 4 — DERS 4.4

Modern Control Flow (@if, @for, @switch)

Angular v17'nin en güzel yeniliklerinden: yeni control flow syntax'ı. Eski *ngIf, *ngFor'a göre %90 daha hızlı, daha temiz, daha güvenli.



Hedef

Modern @if, @for, @switch syntax'ını öğrenecek; eski directive yapısından farklarını ve neden migrate etmemiz gerektiğini anlayacaksınız.

1

Neden Yeni Syntax?

Eski yaklaşım (*ngIf, *ngFor) — Problemler:

- ✗ Karmaşık syntax — yıldız (*) ve directive birleşimi
- ✗ Performance issues — virtual DOM overhead
- ✗ CommonModule import zorunluluğu
- ✗ Type narrowing zayıf — TypeScript template'i tam anlamıyor
- ✗ Nested template'ler okunmuyor — ng-template, ngIfElse karmaşası

Modern yaklaşım (@if, @for, @switch) — Avantajlar:

- ✓ Built-in — CommonModule gerekmiyor
- ✓ %90 hızlı — Direct DOM manipulation
- ✓ Daha okunabilir — JavaScript benzeri syntax
- ✓ Type narrowing güçlü — TS template'te de çalışıyor
- ✓ track zorunlu — Performance pitfall'lar engelleniyor

2 @if — Koşullu Render

Basit Kullanım

```
@if (isLoggedIn()) {  
  <p>Welcome, {{ userName() }}!</p>  
}
```

@else ile Alternatif

```
@if (isLoading()) {  
  <app-spinner />  
} @else {  
  <app-content />  
}
```

@else if Chain

```
@if (status() === 'loading') {  
  <app-spinner />  
} @else if (status() === 'error') {  
  <app-error [message]="errorMsg()" />  
} @else if (status() === 'success') {  
  <app-data [items]="items()" />  
} @else {  
  <p>Hadi başlayalım!</p>  
}
```

@if as — Type Narrowing (↩ Modül 2.5)

```
// Modül 2.5'te narrowing'i öğrenmiştik  
// @if as ile template'te narrow ediyoruz  
  
@if (user(); as currentUser) {  
  <!-- currentUser: User (User | undefined değil!) -->  
  <h1>{{ currentUser.name }}</h1>  
  <p>{{ currentUser.email }}</p>  
  <p>{{ currentUser.age }}</p>  
}
```


⚡ Tekrar İşlenen Konu — Type Narrowing

Modül 2.5'te öğrendiğimiz type narrowing burada aktif. `user()` dönüş tipi `User | undefined` olabilir. `@if (user(); as currentUser)` ile TypeScript "user truthy ise" anlar, `currentUser` sadece `User` tipinde olur.

Eski vs Yeni

```
<!-- ESKİ (*ngIf) -->
<div *ngIf="isLoggedIn; else loginPrompt">
  Welcome!
</div>
<ng-template #loginPrompt>
  Please login.
</ng-template>

<!-- YENİ (@if) -->
@if (isLoggedIn()) {
  <div>Welcome!</div>
} @else {
  <div>Please login.</div>
}
```

Yeni syntax açıkça daha kısa ve okunabilir.

3 @for — Liste Render

Temel Kullanım

```
@for (item of items(); track item.id) {  
  <li>{{ item.name }}</li>  
}
```



track Zorunlu!

Eski *ngFor'da trackBy opsiyoneldi. Modern @for'da zorunlu. Bu, performans pitfall'larını engeller. Track yoksa compile error alırsın.

track Ne İşe Yarar?

Angular'a "her item'i nasıl tanıyacağını" söyler. Liste güncellendiğinde hangi item'in eklendiği, silindiği, sıralandığı bilinsin diye.

```
// ✓ İyi - ID kullan (en yaygın)  
@for (user of users(); track user.id) { ... }  
  
// ✓ İyi - Eşsiz string kullan  
@for (item of items(); track item.code) { ... }  
  
// ⚠ Sadece primitive listede - index kullan  
@for (name of stringList(); track $index) { ... }  
  
// ❌ Anti-pattern - identity track (default JS comparison)  
@for (item of items(); track item) { ... }  
// Object reference değiştiğinde her şey re-render olur
```

Built-in Variables

```
@for (user of users(); track user.id; let i = $index) {  
  <div>{{ i + 1 }}. {{ user.name }}</div>  
}
```

Variable	Ne?
\$index	Index (0'dan başlar)
\$first	İlk eleman mı? (boolean)
\$last	Son eleman mı? (boolean)
\$even	Çift index mi? (zebra striping için)
\$odd	Tek index mi?
\$count	Toplam eleman sayısı

@empty — Boş Liste Durumu

```
@for (todo of todos(); track todo.id) {  
  <app-todo-item [todo]="todo" />  
} @empty {  
  <p class="empty-state">Henüz todo yok. Yeni bir tane ekle!</p>  
}
```

Liste boşsa @empty bloğu render edilir. Eski yaklaşımda ayrı *ngIf="todos.length === 0" gerekiyordu.

4

@switch — Çoklu Koşul

Birçok @if/else if yerine:

```
@switch (status()) {  
  @case ('loading') {  
    <app-spinner />  
  }  
  @case ('success') {  
    <app-data [items]="data()" />  
  }  
  @case ('error') {  
    <app-error [message]="errorMsg()" />  
  }  
  @default {  
    <p>Idle</p>  
  }  
}
```

JavaScript'in switch'ine benziyor. @default opsiyonel.

Discriminated Union ile Birlikte (↗ Modül 2.1, 2.5)

```
// Modül 2.1 ve 2.5'te öğrendik  
type AsyncState<T> =  
  | { status: 'idle' }  
  | { status: 'loading' }  
  | { status: 'success'; data: T }  
  | { status: 'error'; error: string };  
  
export class DataComponent {  
  state = signal<AsyncState<User[]>>({ status: 'idle' });  
}
```

```
@switch (state().status) {
  @case ('idle') {
    <p>Click to load</p>
  }
  @case ('loading') {
    <app-spinner />
  }
  @case ('success') {
    <!-- state(): success state - data property var! -->
    <app-list [items]="state().data" />
  }
  @case ('error') {
    <!-- state(): error state - error property var! -->
    <p>Error: {{ state().error }}</p>
  }
}
```

⚡ Tekrar İşlenen Konu — Discriminated Union

Modül 2.1'de discriminated union'ları öğrenmiştik. Modül 2.5'te narrowing pattern'ini gördük. Bu tam o pattern'in template'te kullanımı. Status'a göre TypeScript hangi property'lerin olduğunu biliyor.

5

Pratik Örnek — Hepsi Bir Arada

```

@Component({
  selector: 'app-user-list',
  template: `
    <div class="user-list">

      <!-- @switch ile farklı durumlar -->
      @switch (state()) {
        @case ('loading') {
          <app-spinner />
        }
        @case ('error') {
          <app-error />
        }
        @case ('success') {

          <!-- @for ile listele -->
          @for (user of users(); track user.id; let i = $index, isFirst
= $first) {
            <div
              class="user-card"
              [class.first]="isFirst"
              [class.even]="$even"
            >
              <span>{{ i + 1 }}.</span>

              <!-- @if ile koşullu detay -->
              @if (user.isAdmin) {
                <span class="badge">Admin</span>
              }

              <span>{{ user.name }}</span>

              <!-- @if as ile narrowing -->
              @if (user.lastLogin; as date) {
                <small>Last login: {{ date | date }}</small>
              }
            </div>
          } @empty {
            <p>Henüz kullanıcı yok.</p>
          }
        }
      }

    </div>
  `,
})
export class UserListComponent {

```

```
users = input.required<User[]>();  
state = input.required<'loading' | 'error' | 'success'>();  
}
```

6

Eski Syntax'tan Migrate Etme

Eski projeyi yeni syntax'a otomatik dönüştür:

```
ng generate @angular/core:control-flow  
  
# Ne yapar:  
# *ngIf → @if  
# *ngFor → @for (track ekler!)  
# *ngSwitchCase → @switch / @case  
# trackBy fonksiyonlarını track expression'a çevirir
```

Tüm template'leri otomatik dönüştürür. Manuel yapmak çılgınlık olur.

□ Mini Quiz

- **@if @else if @else syntax'ı eski *ngIf'e göre neden daha iyi?**

Cevap: Built-in (CommonModule yok), JavaScript benzeri syntax, type narrowing destekli, ng-template/ngIfElse boilerplate'i yok, daha okunabilir.

- **@for'da track neden zorunlu?**

Cevap: Performance için kritik. Liste değiştiğinde Angular hangi item'in hangi olduğunu bilmeli. Track yoksa her şey re-render olur (yavaş + state kaybı).

- **@if (user()); as u) syntax'ı ne yapar?**

Cevap: Type narrowing. user() User | undefined olabilir. as ile narrowed reference (u: User) alırsınız ve template'te güvenle u.name yazabilirsiniz.

- **@for ... @empty ne işe yarar?**

Cevap: Liste boşsa render edilecek alternatif blok. Eski yaklaşımda ayrı *ngIf="items.length === 0" gerekiyordu.

- **\$index, \$first, \$last gibi variable'lar nasıl kullanılır?**

Cevap: @for (item of items; track item.id; let i = \$index) syntax'ı ile özel isim verilebilir. \$index, \$first, \$last, \$even, \$odd, \$count var.

- **Eski projeyi modern syntax'a nasıl çeviririm?**

Cevap: ng generate @angular/core:control-flow komutu ile Angular CLI otomatik dönüştürür.

⚠ Anti-Patterns

- ✗ Yeni projede *ngIf, *ngFor kullanmak — Modern syntax tercih et
- ✗ track item (identity) kullanmak — id veya unique field kullan
- ✗ track \$index (object listede) — Object referansları değişebilir, id kullan
- ✗ @if ile uzun chain — 3+ alternatif varsa @switch düşün
- ✗ @empty unutmak — Boş liste için her zaman fallback ekle
- ✗ Template'te kompleks expression — computed signal tercih et

□ Bu Dersten Çıkarman Gerekenler

- ✓ @if @else if @else syntax'ını rahatça kullanabiliyorum
- ✓ @for ile track'in neden zorunlu olduğunu biliyorum
- ✓ @for @empty ile boş liste durumlarını ele alabiliyorum
- ✓ \$index, \$first, \$last gibi variable'ları biliyorum
- ✓ @switch ile çoklu durum yönetimi yapabiliyorum
- ✓ @if as ile type narrowing yapabiliyorum
- ✓ Discriminated union ile @switch kombinasyonunu kavradım

MODÜL 4 — DERS 4.5

@let ile Template Değişkenleri

Angular v18 yeniliği: @let. Template içinde local değişken tanımlama. Computed signal yazmak istemediğin küçük dönüşümler için ideal.



Hedef

@let'in ne işe yaradığını, computed'la farkını, ne zaman kullanılması gerektiğini öğreneceğiz.

1

Problem: Template'te Tekrar Eden Expression'lar

Şu kodu düşün:

```
<!-- Aynı expression 3 kez tekrar -->
<div>
  <p>Hello {{ user().firstName + ' ' + user().lastName }}!</p>
  <h1>{{ user().firstName + ' ' + user().lastName }}'s Profile</h1>
  <title>{{ user().firstName + ' ' + user().lastName }}</title>
</div>
```

Problemler:

- ✗ Tekrar var — 3 yerde aynı expression
- ✗ Performance — Her change detection'da 3 kez hesaplanır
- ✗ Okunmaz — Asıl niyet kayboluyor

2

Çözüm: @let

```
@let fullName = user().firstName + ' ' + user().lastName;

<div>
  <p>Hello {{ fullName }}!</p>
  <h1>{{ fullName }}'s Profile</h1>
  <title>{{ fullName }}</title>
</div>
```

Bu kadar! Tek satır tanımla, her yerde kullan.

@let Özellikleri

- ✓ Local scope — Sadece tanımlandığı template bloğunda geçerli
- ✓ Reactive — Signal değişince otomatik güncellenir
- ✓ Read-only — Sonradan değiştirilemez
- ✓ Type-safe — TypeScript tipini doğru çıkarır

3 Pratik Senaryolar

Senaryo 1: Async Data Unwrap

```
<!-- Eski yöntem - async pipe + ng-template -->
<div *ngIf="user$ | async as user">
  <h1>{{ user.name }}</h1>
  <p>{{ user.email }}</p>
</div>

<!-- Modern yöntem - @let -->
@let user = (user$ | async);
@if (user) {
  <h1>{{ user.name }}</h1>
  <p>{{ user.email }}</p>
}
```

Senaryo 2: Computed Expression

```
@let activeCount = users().filter(u => u.isActive).length;
@let inactiveCount = users().length - activeCount;

<div class="stats">
  <p>Aktif: {{ activeCount }}</p>
  <p>Pasif: {{ inactiveCount }}</p>
  @if (activeCount > inactiveCount) {
    <p>☐ Çoğunluk aktif!</p>
  }
</div>
```

Senaryo 3: Nested @for ile \$index Hesaplama

```
@for (group of groups(); track group.id; let groupIdx = $index) {
  @for (item of group.items; track item.id; let itemIdx = $index) {
    @let globalIdx = groupIdx * 100 + itemIdx;

    <div [id]='item-' + globalIdx">
      {{ groupIdx }}.{{ itemIdx }} - {{ item.name }}
    </div>
  }
}
```

4

@let vs computed() — Hangisi Ne Zaman?

Konu	@let	computed()
Yer	Template'te	Class'ta
Scope	Template bloğunda local	Component-wide
Test edilebilir	Hayır	Evet (unit test)
Reusable	Hayır (lokalize)	Evet (her yerden okunabilir)
Performance	Aynı (her ikisi memoize)	Aynı
Kullanım	Küçük, lokal dönüşümler	Büyük, paylaşılan logic

Örnek: @let Uygun

```
@let fullName = user().firstName + ' ' + user().lastName;  
<!-- Sadece bu template'te kullanılıyor, basit -->  
<h1>{{ fullName }}</h1>
```

Örnek: computed() Uygun

```
// Birden fazla component'te kullanılacak
// Veya unit test gerekli
// Veya logic karmaşık

export class UserComponent {
  user = input.required<User>();

  fullName = computed(() => {
    const u = this.user();
    return `${u.firstName} ${u.lastName}`;
  });

  initials = computed(() => {
    const u = this.user();
    return `${u.firstName[0]}${u.lastName[0]}`;
  });

  // Template'te:
  // {{ fullName() }}
  // {{ initials() }}
}
```



Pratik Tavsiye

Karar verirken sor: "Bu logic başka yerde de gerekli olur mu?". Evet ise `computed()`, hayır ise `@let`.

5 @let Sınırlamaları

❑ Sonradan Değiştirilemez

```
@let count = 5;

<button (click)="count = count + 1"> <!-- ❑ ERROR -->
  Increment
</button>
```

`@let` read-only. Değiştirmek istiyorsan `signal()` kullan.

❑ Sadece Template'te

```
// ❑ Class'ta @let yok!
export class MyComponent {
  // @let isn't a JavaScript thing
}
```

Class'ta normal const veya computed kullan.

✓ Block Scope

```
<div>
  @let x = 1;
  <p>{{ x }}</p> <!-- ✓ OK -->
</div>

<div>
  <!-- x burada YOK -->
  <p>{{ x }}</p> <!-- ✖ ERROR -->
</div>
```

□ Mini Quiz

- **@let ne zaman geldi?**

Cevap: Angular v18 (Mayıs 2024). Modern Angular'ın template özelliklerinden biri.

- **@let ile computed() arasındaki temel fark nedir?**

Cevap: @let template'te local, sadece o template'te kullanılabilir. computed() class'ta, component-wide ve test edilebilir.

- **@let sonradan değiştirilebilir mi?**

Cevap: Hayır. Read-only. Değiştirilebilen değer için signal() kullan.

- **@let async pipe ile nasıl kullanılır?**

Cevap: @let user = (user\$ | async); şeklinde. Observable'ı unwrap eder ve template'te kullanılabilir hale getirir.

- **@let'in scope'u ne?**

Cevap: Tanımlandığı template bloğunda lokal. @if, @for gibi blokların içine girdiğinde dışına çıkar.

- **Karar verirken @let mi computed() mu nasıl seçilir?**

Cevap: Logic başka yerde gerekli mi? Evet → computed(). Hayır, sadece template'te → @let.

⚠ Anti-Patterns

- ✗ Her yerde @let yerine computed kullanmak — Küçük şeyler için @let yeterli
- ✗ Karmaşık logic için @let — computed() ile class'a taşı
- ✗ @let'i değiştirmeye çalışmak — Read-only, signal kullan
- ✗ Aynı @let'i her template'te yeniden yazmak — Reuse gerekiyorsa computed()

□ Bu Dersten Çıkarman Gerekenler

- ✓ @let syntax'ını biliyorum
- ✓ @let ile computed() farkını net anlıyorum
- ✓ @let scope'unu kavradım (template-local, read-only)
- ✓ Hangi durumda @let, hangi durumda computed() seçeceğimi biliyorum
- ✓ @let ile async data unwrap yapabiliyorum
- ✓ @let limitlerini (read-only, template-only) biliyorum

MODÜL 4 — DERS 4.6

input(), output(), model() Functions

Component'ler arası veri akışının modern yolu. Eski @Input, @Output decorator'larının yerine signal-based function'lar. Type-safe, daha temiz, daha güçlü.



Hedef

Component'lerin dışarıdan veri alma (input), dışarıya event yayma (output), two-way binding sağlama (model) yollarını modern syntax ile öğreneceğiz.

1

input() — Component'e Veri Geçme

Temel Kullanım

```
@Component({
  selector: 'app-user-card',
  template: `
    <h1>{{ user().name }}</h1>
    <p>{{ user().email }}</p>
  `
})
export class UserCardComponent {
  // Optional input - default değer
  user = input<User>({ name: '', email: '' });

  // Required input - mutlaka verilmeli
  userId = input.required<number>();

  // Optional input - alias ile
  showEmail = input(true, { alias: 'show-email' });

  // Transform ile
  age = input(0, {
    transform: (value: string | number) => +value
  });
}
```

Kullanımı

```
<!-- Parent template -->
<app-user-card
  [user]="currentUser()"
  [userId]="42"
  [show-email]="true"
  age="25"           <!-- transform sayesinde string OK -->
/>
```

Önemli Detaylar

- `input()` bir Signal döner
- Template'te `user()` şeklinde çağrılır
- Class'ta `this.user()` şeklinde okunur
- Read-only — component içinde değiştirilemez

Eski @Input vs Modern input()

```
// ESKİ
@Component({...})
export class UserCardComponent {
  @Input() user!: User;           // Required ama compile-time garan
  ti yok
  @Input() showEmail = true;
  @Input('show-email') showEmail2 = true; // Alias
}

// MODERN
@Component({...})
export class UserCardComponent {
  user = input.required<User>();   // Required - compile-time enforc
  ed
  showEmail = input(true);
  showEmail2 = input(true, { alias: 'show-email' });
}
```



Modern Avantajlar

- Signal-based — Reactivity built-in
- Type-safe required — Compile-time enforced
- Transform desteği — Input'u dönüştürme built-in
- Read-only — Yanlılıkla değiştirilmesi engellenir

2 output() — Event Yayma

Temel Kullanım

```
@Component({
  selector: 'app-user-card',
  template: `
    <button (click)="onSelect()">Select</button>
    <button (click)="onDelete()">Delete</button>
  `
})
export class UserCardComponent {
  user = input.required<User>();

  // Output event'ler
  selected = output<User>();
  deleted = output<number>(); // User id'si gönderiyoruz
  closed = output();         // Veri yok, sadece event

  onSelect() {
    this.selected.emit(this.user());
  }

  onDelete() {
    this.deleted.emit(this.user().id);
  }
}
```

Kullanımı

```
<!-- Parent template -->
<app-user-card
  [user]="user"
  (selected)="onUserSelected($event)"
  (deleted)="onUserDeleted($event)"
  (closed)="onCardClosed()"
/>
```

```
// Parent component
export class ParentComponent {
  onUserSelected(user: User) {
    console.log('Selected:', user.name);
  }

  onUserDeleted(userId: number) {
    this.userService.delete(userId);
  }

  onCardClosed() {
    console.log('Card closed');
  }
}
```

Eski @Output vs Modern output()

```
// ESKİ
@Component({...})
export class UserCardComponent {
  @Output() selected = new EventEmitter<User>();
}

// MODERN
@Component({...})
export class UserCardComponent {
  selected = output<User>();
}
```

Avantajlar:

- ✓ Daha kısa syntax
- ✓ EventEmitter'a manuel import gereksiz
- ✓ Aynı API — emit() method aynı
- ✓ Type inference daha iyi

3 model() — Two-way Binding

Modül 4.3'te [(ngModel)]'i gördük. Custom component için kendi two-way binding'ini yapmak istersen model() kullanırsın.

Temel Kullanım

```
@Component({
  selector: 'app-counter-input',
  template: `
    <button (click)="decrement()">-</button>
    <span>{{ value() }}</span>
    <button (click)="increment()">+</button>
  `
})
export class CounterInputComponent {
  // model() ile two-way input
  value = model.required<number>();

  // model.required<T> veya model(defaultValue)

  increment() {
    this.value.update(v => v + 1);
  }

  decrement() {
    this.value.update(v => v - 1);
  }
}
```

Kullanımı

```
// Parent template
<app-counter-input [(value)]="count" />

// Parent class
export class ParentComponent {
  count = signal(0);
  // Child component'te value değişince count otomatik güncellenir
  // count değişince child'da value otomatik güncellenir
}
```

input + output ile Manuel Yapsaydık?

```
// Eski yöntem - manuel iki yönlü
@Component({...})
export class CounterInputComponent {
  value = input.required<number>();
  valueChange = output<number>(); // ← Mecburi naming

  increment() {
    this.valueChange.emit(this.value() + 1);
  }
}

// Parent'ta:
<app-counter-input
  [value]="count"
  (valueChange)="count = $event"
/>

// Veya kısa form:
<app-counter-input [(value)]="count" />
```

`model()` bu boilerplate'i otomatik yapar. `valueChange` output'unu kendi yaratır.

4

Pratik Örnek — Üçü Bir Arada

```
// user-edit.ts - Custom editable user component
@Component({
  selector: 'app-user-edit',
  imports: [FormsModule],
  template: `
    <div class="user-edit">
      <input [(ngModel)]="name" placeholder="Name" />
      <input [(ngModel)]="email" placeholder="Email" />

      <div>
        <button (click)="onSave()">Save</button>
        <button (click)="onCancel()">Cancel</button>
      </div>

      @if (isEditing()) {
        <p class="status">Editing...</p>
      }
    </div>
  `,
})
export class UserEditComponent {
  // input - dışarıdan başlangıç değeri
  initialUser = input.required<User>();

  // model - iki yönlü bağlanan name ve email
  name = model<string>('');
  email = model<string>('');

  // Local state
  isEditing = signal(false);

  // output - event'ler
  saved = output<User>();
  cancelled = output();

  constructor() {
    // input değişince, name/email'i set et
    effect(() => {
      const u = this.initialUser();
      this.name.set(u.name);
      this.email.set(u.email);
    });
  }

  onSave() {
    this.saved.emit({
      ...this.initialUser(),
    });
  }
}
```



```
        name: this.name(),
        email: this.email(),
    });
    this.isEditing.set(false);
}

onCancel() {
    this.name.set(this.initialUser().name);
    this.email.set(this.initialUser().email);
    this.cancelled.emit();
}
}

// Parent kullanım
<app-user-edit
    [initialUser]="currentUser()"
    [(name)]="editedName"
    [(email)]="editedEmail"
    (saved)="onUserSaved($event)"
    (cancelled)="onEditCancelled()"
/>
```

□ Mini Quiz

- **input() ile @Input() arasındaki temel fark nedir?**

Cevap: input() signal döner (reactive, otomatik track edilebilir), @Input() plain property. input.required() compile-time enforced required, @Input() not null assertion (!) gerektirir.

- **input.required() ne işe yarar?**

Cevap: Required input tanımlar. Compile-time'da kontrol edilir — kullanıcı bu input'u geçmeden component'i kullanamaz. Hata: "Required input 'user' not provided".

- **output() ile EventEmitter arasındaki fark nedir?**

Cevap: output() daha kısa syntax. Aynı emit() API'sini kullanır. EventEmitter import etmeye gerek yok. Type inference daha iyi.

- **model() ne için kullanılır?**

Cevap: Custom component için two-way binding. şeklinde kullanılır. valueChange output'unu otomatik yaratır.

- **input transform ne işe yarar?**

Cevap: Gelen değeri dönüştürmek için. Örn. string number'a: transform: (v: string|number) => +v. Tip dönüşümü built-in.

- **Template'te input'lar nasıl okunur?**

Cevap: Signal gibi: this.user() veya {{ user() }}. Fonksiyon çağırısı yapılır, parantezsiz signal object'i gösterir.

⚠ Anti-Patterns

- ✗ Yeni projede @Input/@Output kullanmak — input()/output() tercih et
- ✗ input()'u component içinde set etmeye çalışmak — Read-only
- ✗ required input'u optional ile karıştırmak — input.required() vs input()
- ✗ Manuel valueChange yazmak — model() bu boilerplate'i otomatik yapar
- ✗ output ismini "xxxChange" yapmamak (two-way için) — model() bu sorunu çözer
- ✗ input() yazıp template'te parantezsiz okumak — user yerine user() yaz

□ Bu Dersten Çıkarman Gerekenler

- ✓ input() ile component'e veri aktarmayı biliyorum
- ✓ input.required() ile zorunlu input'lar tanımlayabiliyorum
- ✓ output() ile event yaymayı biliyorum
- ✓ model() ile two-way binding yapabiliyorum
- ✓ input transform fonksiyonlarını kullanabiliyorum

- ✓ Eski @Input/@Output decorator'larından modern alternatiflerine geçiş yapabiliyorum
- ✓ Üçünü (input, output, model) tek component'te birleştirebiliyorum

❏ Mini Proje: Angular ToDo App

Modül 4'ün asıl ödülü: Modül 2.6'da yaptığımız TypeScript ToDo Manager'ın Angular versiyonu! Bu sefer gerçek UI ile, modern Angular ile.



Hedef

Tüm Modül 4 konseptlerini birleştir: component anatomy, standalone, data binding, modern control flow, @let, input/output/model. Her şey gerçek bir projede kullanılacak.

1

Proje Özellikleri

- ✓ Todo ekleme (form ile)
- ✓ Todo silme
- ✓ Tamamlandı işaretleme (toggle)
- ✓ Filtreleme (all/active/completed)
- ✓ Priority seçme (low/medium/high)
- ✓ İstatistikler (toplam/aktif/tamamlanan)
- ✓ Tamamlananları toplu silme

Component Hiyerarşisi

App	← Root
├─ TodoForm	← Yeni todo ekleme
├─ TodoFilter	← Filtre butonları
├─ TodoStats	← İstatistikler
└─ TodoList	← Todo'ları gösterir
└─ TodoItem	← Tek bir todo (xN)

Önemli Tasarım Kararları

- Smart vs Dumb component: App smart (state owner), diğerleri dumb (props in/events out)
- Single source of truth: Tüm state App'te
- Immutable updates: Signal'lerde array spread, mutation yok
- Type-safe: Interface'ler, generic store gibi pattern'ler (Modül 2)

2 Proje Yapısı

```
angular-todo-app/  
├── src/  
│   ├── app/  
│   │   ├── models/  
│   │   │   └── todo.model.ts      ← Interfaces, types  
│   │   ├── services/  
│   │   │   └── todo.service.ts    ← Business logic  
│   │   ├── components/  
│   │   │   ├── todo-form/  
│   │   │   │   ├── todo-form.ts  
│   │   │   │   ├── todo-form.html  
│   │   │   │   └── todo-form.scss  
│   │   │   ├── todo-filter/  
│   │   │   │   └── (3 dosya)  
│   │   │   ├── todo-stats/  
│   │   │   │   └── (3 dosya)  
│   │   │   ├── todo-list/  
│   │   │   │   └── (3 dosya)  
│   │   │   └── todo-item/  
│   │   │       └── (3 dosya)  
│   │   ├── app.config.ts  
│   │   ├── app.routes.ts  
│   │   ├── app.ts  
│   │   ├── app.html  
│   │   └── app.scss  
│   ├── main.ts  
│   └── styles.scss  
├── angular.json  
├── package.json  
└── tsconfig.json
```

3 Adım Adım Yapım

Adım 1: Yeni Proje Oluştur

```
ng new angular-todo-app --style=scss --ssr=no  
  
# Sorulara: SCSS, No SSR, Yes Zoneless, No Tailwind  
  
cd angular-todo-app  
ng serve --open
```

Adım 2: Model Oluştur

```
# src/app/models/todo.model.ts
ng g i models/todo # Interface generator
```

```
// src/app/models/todo.model.ts
export type Priority = 'low' | 'medium' | 'high';
export type FilterMode = 'all' | 'active' | 'completed';

export interface Todo {
  readonly id: string;
  title: string;
  completed: boolean;
  priority: Priority;
  readonly createdAt: Date;
}

export type CreateTodoInput = Omit<Todo, 'id' | 'createdAt'>;

export interface TodoStats {
  total: number;
  active: number;
  completed: number;
  byPriority: Record<Priority, number>;
}
```

⚡ Modül 2 Tekrarı

Bu kod tamamen Modül 2 konularını kullanıyor: interface, readonly, union types, Omit, Record. Modül 2.6'daki TypeScript Bootcamp ile aynı pattern.

Adım 3: TodoService Oluştur

```
ng g s services/todo
```

```
// src/app/services/todo.service.ts
import { Injectable, signal, computed } from '@angular/core';
import {
  Todo, CreateTodoInput, FilterMode, Priority, TodoStats
} from '../models/todo.model';

@Injectable({ providedIn: 'root' })
export class TodoService {
  // Private state (write)
  private _todos = signal<Todo[]>([]);
  private _filter = signal<FilterMode>('all');

  // Public state (read-only)
  todos = this._todos.asReadonly();
  filter = this._filter.asReadonly();

  // Computed - filtrelenmiş todos
  filteredTodos = computed(() => {
    const allTodos = this._todos();
    const mode = this._filter();

    switch (mode) {
      case 'all':      return allTodos;
      case 'active':   return allTodos.filter(t => !t.completed);
      case 'completed': return allTodos.filter(t => t.completed);
    }
  });

  // Computed - statistics
  stats = computed<TodoStats>(() => {
    const allTodos = this._todos();
    const active = allTodos.filter(t => !t.completed).length;

    const byPriority: Record<Priority, number> = {
      low: 0,
      medium: 0,
      high: 0,
    };

    allTodos.forEach(t => { byPriority[t.priority]++; });

    return {
      total: allTodos.length,
      active,
      completed: allTodos.length - active,
      byPriority,
    };
  });
}
```

```
});

// Actions
addTodo(input: CreateTodoInput): void {
  const newTodo: Todo = {
    id: crypto.randomUUID(),
    ...input,
    createdAt: new Date(),
  };
  this._todos.update(todos => [...todos, newTodo]);
}

toggleTodo(id: string): void {
  this._todos.update(todos =>
    todos.map(t =>
      t.id === id ? { ...t, completed: !t.completed } : t
    )
  );
}

deleteTodo(id: string): void {
  this._todos.update(todos => todos.filter(t => t.id !== id));
}

clearCompleted(): void {
  this._todos.update(todos => todos.filter(t => !t.completed));
}

setFilter(mode: FilterMode): void {
  this._filter.set(mode);
}
}
```

Adım 4: TodoItem Component

```
ng g c components/todo-item
```



```
// todo-item.ts
import { Component, ChangeDetectionStrategy, input, output } from '@angular/core';
import { Todo } from '../../models/todo.model';

@Component({
  selector: 'app-todo-item',
  templateUrl: './todo-item.html',
  styleUrls: ['./todo-item.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class TodoItemComponent {
  todo = input.required<Todo>();
  toggled = output<string>();
  deleted = output<string>();

  onToggle() {
    this.toggled.emit(this.todo().id);
  }

  onDelete() {
    this.deleted.emit(this.todo().id);
  }
}
```

```
<!-- todo-item.html -->
<div class="todo-item" [class.completed]="todo().completed">
  <input
    type="checkbox"
    [checked]="todo().completed"
    (change)="onToggle()"
  />

  <span class="title">{{ todo().title }}</span>

  <span class="priority" [class]="p- + todo().priority">
    {{ todo().priority }}
  </span>

  <button class="delete" (click)="onDelete()">x</button>
</div>
```

Adım 5: TodoList Component

```
ng g c components/todo-list
```

```
// todo-list.ts
import { Component, ChangeDetectionStrategy, input, output } from '@angular/core';
import { Todo } from '../../models/todo.model';
import { TodoItemComponent } from '../todo-item/todo-item';

@Component({
  selector: 'app-todo-list',
  imports: [TodoItemComponent],
  templateUrl: './todo-list.html',
  styleUrls: ['./todo-list.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class TodoListComponent {
  todos = input.required<readonly Todo[]>();
  toggled = output<string>();
  deleted = output<string>();
}
```

```
<!-- todo-list.html -->
@if (todos().length > 0) {
  <div class="todo-list">
    @for (todo of todos(); track todo.id) {
      <app-todo-item
        [todo]="todo"
        (toggled)="toggled.emit($event)"
        (deleted)="deleted.emit($event)"
      />
    }
  </div>
} @else {
  <p class="empty">Henüz todo yok! Yeni bir tane ekle.</p>
}
```

Adım 6: TodoForm Component

```
ng g c components/todo-form
```

```
// todo-form.ts
import { Component, ChangeDetectionStrategy, output, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Priority, CreateTodoInput } from '../../models/todo.model';

@Component({
  selector: 'app-todo-form',
  imports: [FormsModule],
  templateUrl: './todo-form.html',
  styleUrls: ['./todo-form.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class TodoFormComponent {
  title = signal('');
  priority = signal<Priority>('medium');

  todoAdded = output<CreateTodoInput>();

  onSubmit() {
    const trimmedTitle = this.title().trim();
    if (!trimmedTitle) return;

    this.todoAdded.emit({
      title: trimmedTitle,
      priority: this.priority(),
      completed: false,
    });

    // Reset
    this.title.set('');
    this.priority.set('medium');
  }
}
```

```
<!-- todo-form.html -->
<form class="todo-form" (submit)="onSubmit(); $event.preventDefault()">
  <input
    type="text"
    [(ngModel)]="title"
    name="title"
    placeholder="Yeni todo..."
  />

  <select [(ngModel)]="priority" name="priority">
    <option value="low">Low</option>
    <option value="medium">Medium</option>
    <option value="high">High</option>
  </select>

  <button type="submit" [disabled]="!title().trim()">
    Ekle
  </button>
</form>
```

Adım 7: TodoFilter Component

```
// todo-filter.ts
import { Component, ChangeDetectionStrategy, input, output } from '@angular/core';
import { FilterMode } from '../models/todo.model';

@Component({
  selector: 'app-todo-filter',
  templateUrl: './todo-filter.html',
  styleUrls: ['./todo-filter.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class TodoFilterComponent {
  current = input.required<FilterMode>();
  filterChanged = output<FilterMode>();

  filters: FilterMode[] = ['all', 'active', 'completed'];
}
```

```
<!-- todo-filter.html -->
<div class="filter-buttons">
  @for (filter of filters; track filter) {
    <button
      [class.active]="current() === filter"
      (click)="filterChanged.emit(filter)"
    >
      {{ filter }}
    </button>
  }
</div>
```

Adım 8: TodoStats Component

```
// todo-stats.ts
import { Component, ChangeDetectionStrategy, input } from '@angular/core';
import { TodoStats } from '../models/todo.model';

@Component({
  selector: 'app-todo-stats',
  templateUrl: './todo-stats.html',
  styleUrls: ['./todo-stats.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class TodoStatsComponent {
  stats = input.required<TodoStats>();
}
```

```
<!-- todo-stats.html -->
@let s = stats();
<div class="stats">
  <div class="stat"><span>Toplam:</span> {{ s.total }}</div>
  <div class="stat"><span>Aktif:</span> {{ s.active }}</div>
  <div class="stat"><span>Tamamlandı:</span> {{ s.completed }}</div>

  <div class="priorities">
    @if (s.byPriority.high > 0) {
      <span class="badge high">High: {{ s.byPriority.high }}</span>
    }
    @if (s.byPriority.medium > 0) {
      <span class="badge medium">Medium: {{ s.byPriority.medium }}</span>
    }
    @if (s.byPriority.low > 0) {
      <span class="badge low">Low: {{ s.byPriority.low }}</span>
    }
  </div>
</div>
```

Adım 9: App Component'ini Birleştir

```
// app.ts
import { Component, ChangeDetectionStrategy, inject } from '@angular/core';
import { TodoService } from '../services/todo.service';
import { TodoFormComponent } from '../components/todo-form/todo-form';
import { TodoFilterComponent } from '../components/todo-filter/todo-filter';
import { TodoStatsComponent } from '../components/todo-stats/todo-stats';
import { TodoListComponent } from '../components/todo-list/todo-list';

@Component({
  selector: 'app-root',
  imports: [
    TodoFormComponent,
    TodoFilterComponent,
    TodoStatsComponent,
    TodoListComponent,
  ],
  templateUrl: './app.html',
  styleUrls: ['./app.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class App {
  // Service'i inject et
  protected todoService = inject(TodoService);
}
```

```
<!-- app.html -->
<main class="todo-app">
  <h1>📌 Todo App</h1>

  <app-todo-form
    (todoAdded)="todoService.addTo($event)"
  />

  <app-todo-filter
    [current]="todoService.filter()"
    (filterChanged)="todoService.setFilter($event)"
  />

  <app-todo-stats [stats]="todoService.stats()" />

  <app-todo-list
    [todos]="todoService.filteredTodos()"
    (toggled)="todoService.toggleTodo($event)"
    (deleted)="todoService.deleteTodo($event)"
  />

  @if (todoService.stats().completed > 0) {
    <button class="clear-btn" (click)="todoService.clearCompleted()">
      Tamamlananları Temizle ({{ todoService.stats().completed }})
    </button>
  }
</main>
```


4 Bu Projede Ne Öğrendin?

Konsept	Nerede?
Component anatomy (M4.1)	6 component
Standalone (M4.2)	Tüm component'ler standalone
Property binding (M4.3)	[todo]="todo", [stats]="..."
Event binding (M4.3)	(toggled), (deleted), (filterChanged)
Two-way ngModel (M4.3)	Form input'ları
@if @else (M4.4)	Empty state, conditional render
@for + track (M4.4)	Todo list rendering
@let (M4.5)	Stats unwrap
input.required (M4.6)	Tüm child component'ler
output (M4.6)	Event emission
Interface, readonly (M2.1)	Todo model
Union types (M2.1)	Priority, FilterMode
Omit, Record (M2.4)	CreateTodoInput, TodoStats
signal, computed (M5'e bakış)	TodoService
Service + DI (M7'e bakış)	TodoService injection



Tebrikler!

Bu proje Modül 2 + Modül 3 + Modül 4'ün tümünü birleştiriyor. TypeScript bilgin, Angular CLI deneyimin, modern syntax bilgilerin — hepsi bir araya geldi. Bundan sonra her modülde bu temellerin üstüne inşa edeceğiz.

□ Kendi Pratiğin (Egzersizler)

Egzersiz 1: LocalStorage Persistence

```
// TodoService'e ekle:
import { effect } from '@angular/core';

constructor() {
  // İlk yükle
  const saved = localStorage.getItem('todos');
  if (saved) {
    try {
      const parsed = JSON.parse(saved);
      // Date string'leri Date'e çevir
      this._todos.set(parsed.map(t => ({
        ...t,
        createdAt: new Date(t.createdAt)
      })));
    } catch {}
  }

  // Değişiklikleri kaydet
  effect(() => {
    localStorage.setItem('todos', JSON.stringify(this._todos()));
  });
}
```

Egzersiz 2: Edit Mode

TodoItem'a double-click ile edit mode ekle.

Egzersiz 3: Sıralama

Priority'ye göre veya tarihe göre sıralama eklenebilir bir filter olarak.

Egzersiz 4: Animasyon

Todo eklendiğinde/silindiğinde CSS transition.

Egzersiz 5: Drag and Drop

Angular CDK ile sıralanabilir yap (ileri seviye).

☐ Modül 4 Tamamlandı!

Tebrikler! Modül 4'ü başarıyla tamamladın. Artık component'lerin tüm anatomisini biliyorsun. Modern Angular'ın temel template syntax'ında ustalaştın.

Neler Öğrendin?

Ders 4.1 — Component Anatomisi

Component'in üç parçasını, @Component decorator'ının tüm ayarlarını, naming convention'ları, lifecycle hook'ları kavradık.

Ders 4.2 — Standalone Components

NgModule'siz dünyaya geçişi, imports array'i ile composition'u, lazy loading'i öğrendik.

Ders 4.3 — Data Binding 4 Çeşidi

Interpolation, property, event, two-way binding'lerin tümünü ve \$event ile event handling'i gördük.

Ders 4.4 — Modern Control Flow

@if, @for, @switch syntax'ını ve eski *ngIf, *ngFor'a göre avantajlarını öğrendik. track'in neden zorunlu olduğunu anladık.

Ders 4.5 — @let

Template-local değişkenleri ve computed() ile farkını kavradık.

Ders 4.6 — input(), output(), model()

Modern signal-based component API'sini, two-way binding'in nasıl yaratıldığını gördük.

Mini Proje — Angular Todo App

Tüm bu konuları birleştirip gerçek bir uygulama yaptık.

☐ Sıradaki Modül: Signals & Reactivity

Modül 5 — Modern Angular'ın en güzel özelliği: signals'ın derinliklerine ineceğiz. signal(), computed(), effect(), linkedSignal(), resource() — hepsi bir araya gelecek. Reactivity'nin sırlarını çözeceğiz.